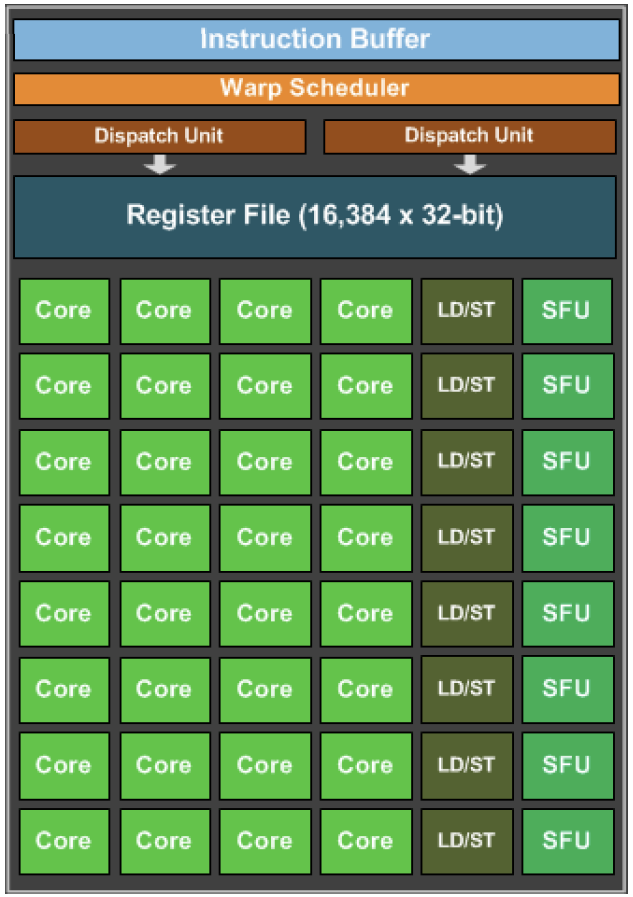
CSE113: Parallel Programming

- **Topics**:
  - GPUs



Announcements

- Last day to turn in HW 3 was last weekend, hope you get it in!
- HW 4 to be released. We will discuss it in class a little bit today.

Announcements

- Today GPUs. This is just the tip of the iceberg!

- Rest of quarter:
  - Memory models
  - Barriers
  - Concurrent Set

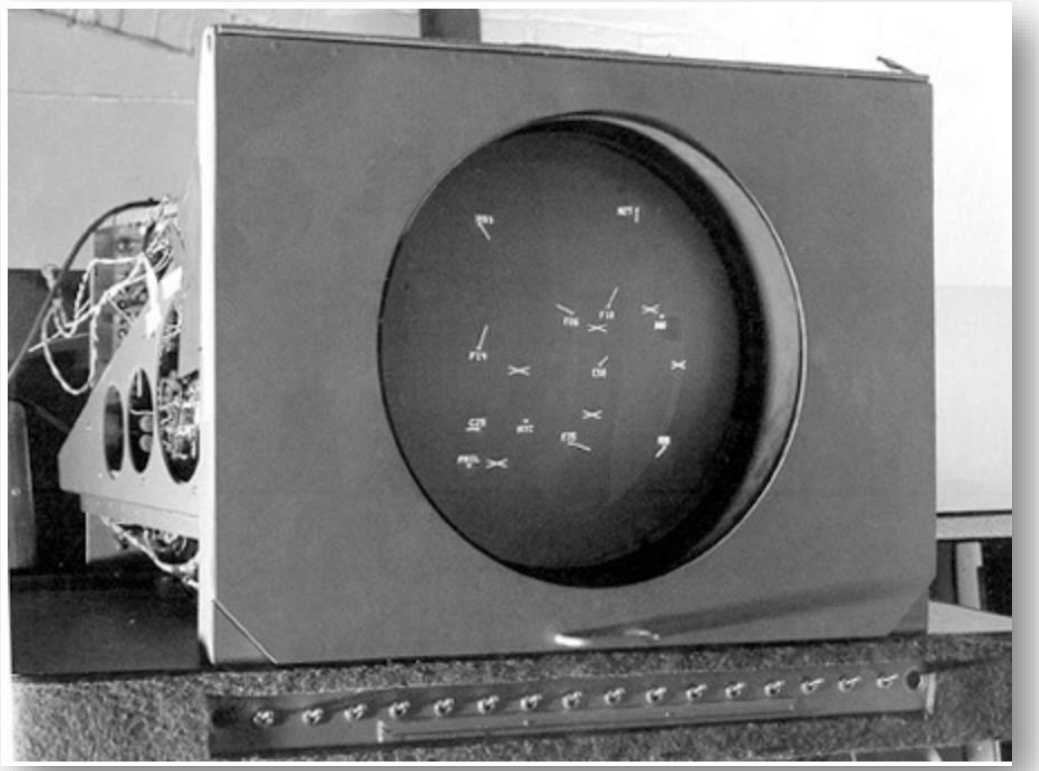
GPUs: a brief history

First chapter of CUDA by Example
https://www.techspot.com/article/650-history-of-the-gpu/

The very beginning

- **Specialized hardware** to accelerate graphics rendering

- One of the first real-time computers: **Whirlwind 1 at MIT** (1951)
  - **Flight simulator** for bombers
  - **Vector graphics**



Vector graphics are a form of computer graphics in which visual images are created directly from geometric shapes defined on a Cartesian plane, such as points, lines, curves and polygons.

Image from: https://ohiostate.pressbooks.pub/graphicshistory/chapter/2-1-whirlwind-and-sage/

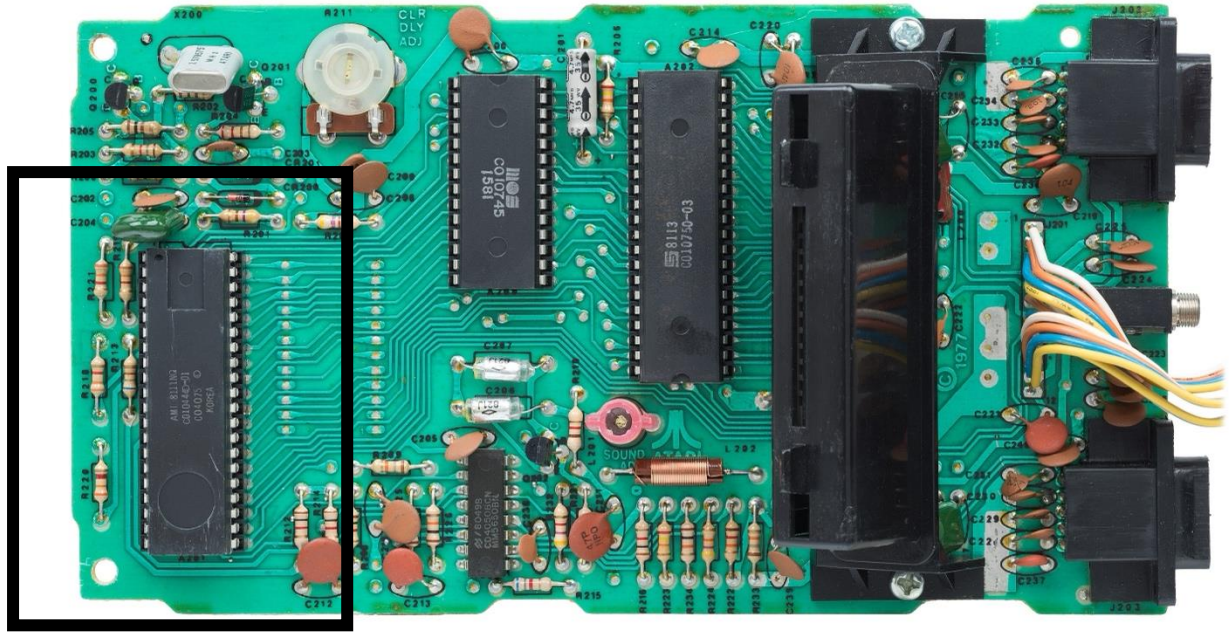
Specialization

- Next 30 years, specialized hardware for specialized software to display 2D graphics

- Specialized
  - Typically ran **specific programs**
  - Portability (i.e. multiple program) was not a top priority
  - Even the idea of portable ISAs for CPUs were not mainstream

Multi-program devices

- 1977: Television Interface Adapter
  - One of the first (and widely produced) portable GPUs





from: https://en.wikipedia.org/wiki/Television_Interface_Adaptor

River Raid Game



https://en.wikipedia.org/wiki/DirectX

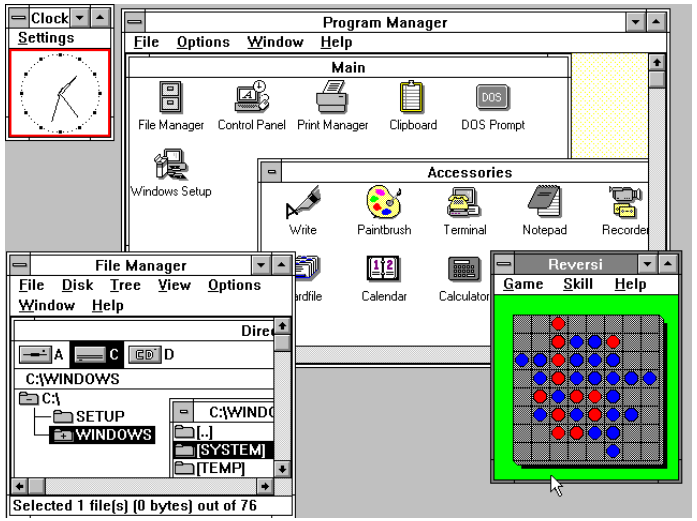
https://en.wikipedia.org/wiki/Microsoft_Windows

https://en.wikipedia.org/wiki/OpenGL

OS integration

- 1990s: Windows: a graphical operating systems, required chips to support 2D graphics.

- **New APIs** started appearing, to enable GUI programs



Windows 3 (1990)

1992

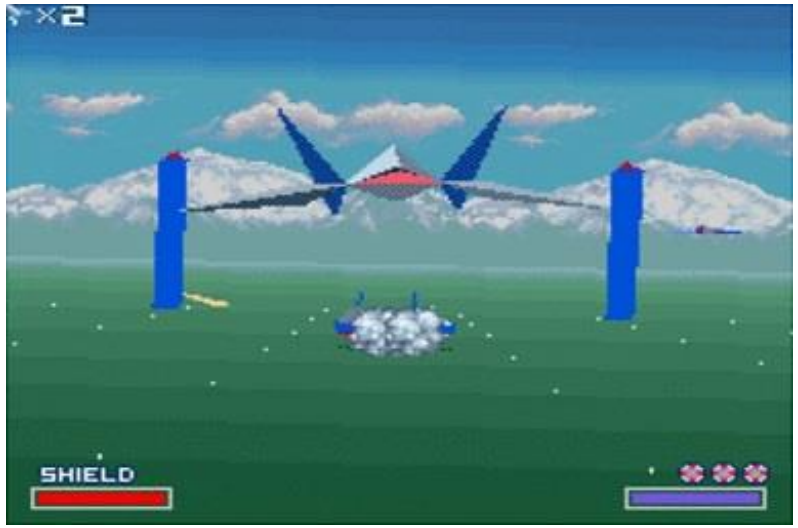
1995

3D graphics in consoles (1993)

- Super Nintendo was not powerful enough to draw 3D graphics
- **Shigeru Miyamoto (Super Mario developer)** really wanted a 3D flight simulator though
- Worked with a British software company to develop…

3D graphics in consoles (1993)

- Super Nintendo was not powerful enough to draw 3D graphics
- Shigeru Miyamoto (Super Mario developer) really wanted a 3D flight simulator though
- Worked with a British software company to develop…





https://en.wikipedia.org/wiki/Star_Fox_(1993_video_game)

3D graphics in consoles (1993)

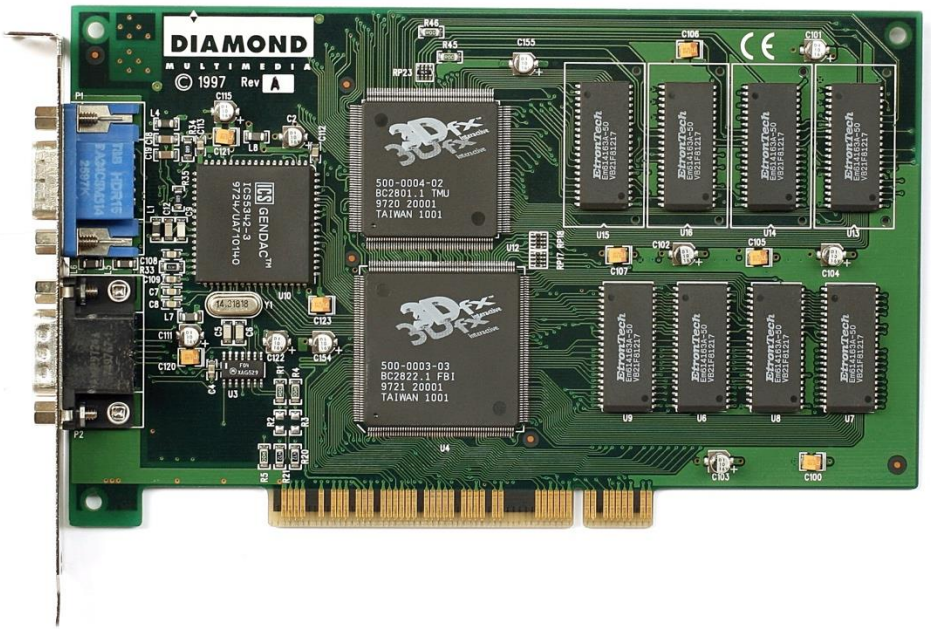
- Game **cartridges shipped with a “mini GPU”** on them:
  - the Super FX



https://twitter.com/gameminesocials/status/1322946537077526528?s=20

3D graphics acceleration

- 1996 : First **3D graphics accelerator**: Voodoo from **3Dfx**
  - Discrete GPU (separate from CPU)
  - Early **3D games: e.g. Tomb Raider**
  - Acquired by Nvidia in 2002



https://en.wikipedia.org/wiki/3dfx_Interactive

3D graphics acceleration

- 3D accelerators continued, many companies **competing**:
  - Nvidia
  - ATI (acquired by AMD)
  - 3Dfx
  - and more...

- Next milestone in 1999:
  - **Nvidia coins the term “GPU”**
  - Compare with modern website

https://web.archive.org/web/20030814003456/www.nvidia.com/object/gpu.html

Programmable 3D accelerators

- 2001: Microsoft **DirectX** 8 required programmable **vertex and pixel shaders**.

- 2001: First GPU to satisfy the requirement was Nvidia GeForce 3
  - Used on the original Xbox

- Programmers **started writing general programs** for these GPUs:
  - Present your data as a graphical input (e.g. Textures and Triangles)
  - Read the output after a series of “graphics” API calls

GPGPU Programming (General Purpose)

- 2006: Nvidia releases **CUDA**: programming language for their GPUs
  - Supported by GeForce 8 series.
  - Integrated vertex and pixel cores into “shader cores”
  - Support for IEEE floating point

- Soon after...

GPGPU Programming

- 2006: Nvidia releases CUDA: programming language for their GPUs
  - Supported by 8th generation CUDA devices.
  - Integrated vertex and pixel cores into “shader cores”
  - Support for IEEE floating point

- Soon after...

- 2008: The **Khronos** Group launches **OpenCL for cross vendor GPGPU**:
  - including AMD, Intel, Qualcomm

Khronos Group



- Started in 2000 **by Apple** as a standards body for graphics API to **counter Microsoft's DirectX**:
  - A way to unify APIs across many different vendors
  - At the time: ATI, Nvidia, Intel, Sun Microsystems (and a few others)
  - Now: Many companies, including AMD, Nvidia, Intel, Qualcomm, ARM, Google

  - **OpenGL** is maybe the biggest standard they maintain (**for graphics**)
  - **OpenCL** is big **for compute**
  - **Vulkan** unifies **compute + graphics** (mostly graphics)
- **Apple deprecated Khronos** group standards to support **Metal** in 2018
  - iPhone used to have **GPUs from a company called Imagination, a British company**. Apple took their engineers.
- Our laptops may have GPUs from **Intel or AMD**. Now apple laptops has **apple GPUs**.

https://en.wikipedia.org/wiki/Khronos_Group

Where are we now?

- **Nvidia CUDA** is widely used, driving many HPC and ML applications
  - Nvidia heavily invested in its software stack and made it accessible, and this made it widely used.
  - Nvidia GPUs are **not designed power efficient**, and are **not used much in mobile devices**.
- **OpenCL** is used to program **other GPUs** (although it is not as widely used)
- **Metal** is used for **Apple devices**
- **Vulkan has momentum**, but mostly in graphics

- New GPGPU programming languages are on the horizon:
  - **WebGPU** - **a javascript interface to unite Metal, Vulkan and DirectX**
  - It's ambitious! Ships in Chrome, Firefox and Safari.

GPU Shortages?

- **Cryptocurrency**:
  - 2018 reported tripling of GPU prices and shortages due to increase demand from miners.
  - Still happening with lots of market fluctuations.
  - Still plenty of GPUs in your phone, laptop, etc. ☺

GPUs now

- **Nvidia is #2 most valuable company** (after Apple) in the world at roughly $4.94 trillion,
  - More valuable than Microsoft, Alphabet and Amazon
- Graphics are still a valuable domain, but **AI** seems to be more valuable
  - Valuable to learn general purpose GPU programming (GPGPU)
- **Many companies** make their own GPUs:
  - AMD, Apple, Qualcomm, ARM, Intel, Imagination, Broadcom
- **Many startups** in the Valley making new AI chips similar to GPUs

Teaching GPU programming

- This is difficult!
- Nvidia GPUs have the most straightforward programming model (**CUDA**).
- It is extremely difficult to get a class of students access to Nvidia GPUs these days.
  - AWS? Expensive and often oversubscribed w.r.t. GPUs
  - Department? ML folks get priority and super computing clusters are painful

Plan

- The GPU programming lectures will use **CUDA**
  - It is widely used
  - The programming model is straightforward

- Homework will use **WebGPU**, because it is widely supported
  - *There are more non-Nvidia GPUs in this room than Nvidia GPUs*

Plan

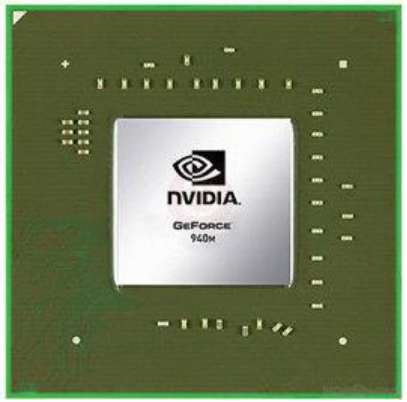
- The homework uses Javascript as its ”CPU” language, and WebGPU as its ”GPU” language.

- We have provided generous skeletons for the homework. We can go over some javascript, but it is a high-level language and should not be hard to figure out what you need to do.

- The WebGPU portion is straightforward, and I will provide a mapping directly from what we talked about to what you need.

Programming a GPU

Tiny GPU in an embedded system





The CPU



Nvidia Jetson Nano (whole chip, CPU + GPU)
2 Billion transistors
10 TDP (Thermal Design Power)
Est. $99

similar transistor count

Intel i7-9700K
2.16 Billion transistors
95 TDP
Est. $316

https://www.techpowerup.com/gpu-specs/geforce-940m.c2648
https://www.alibaba.com/product-detail/Intel-Core-i7-9700K-8-Cores_62512430487.html
https://www.prolast.com/prolast-elevated-boxing-rings-22-x-22/

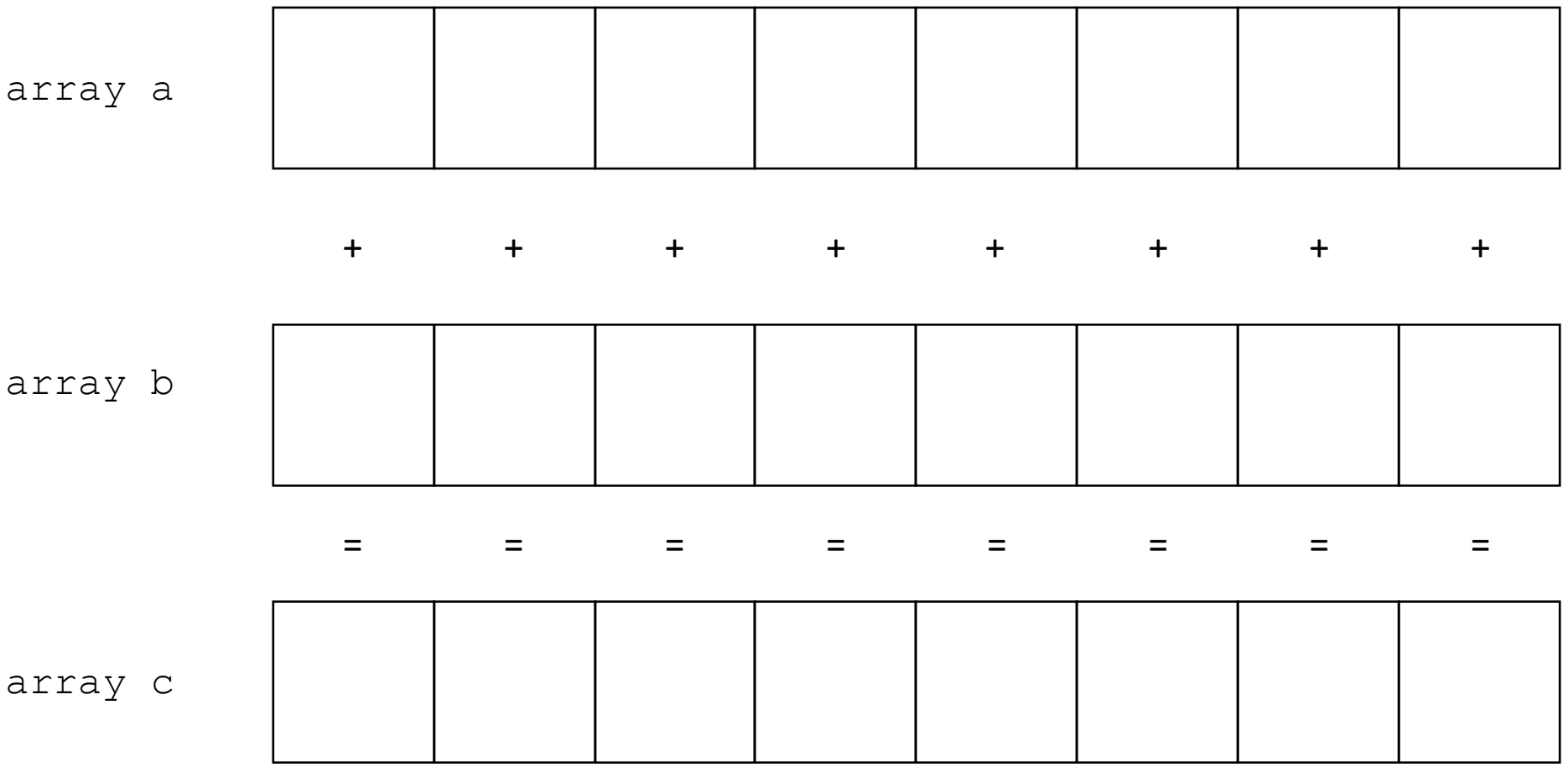
Programming a GPU

- The problem: Vector addition

Embarrassingly parallel

Computation can easily be divided into threads

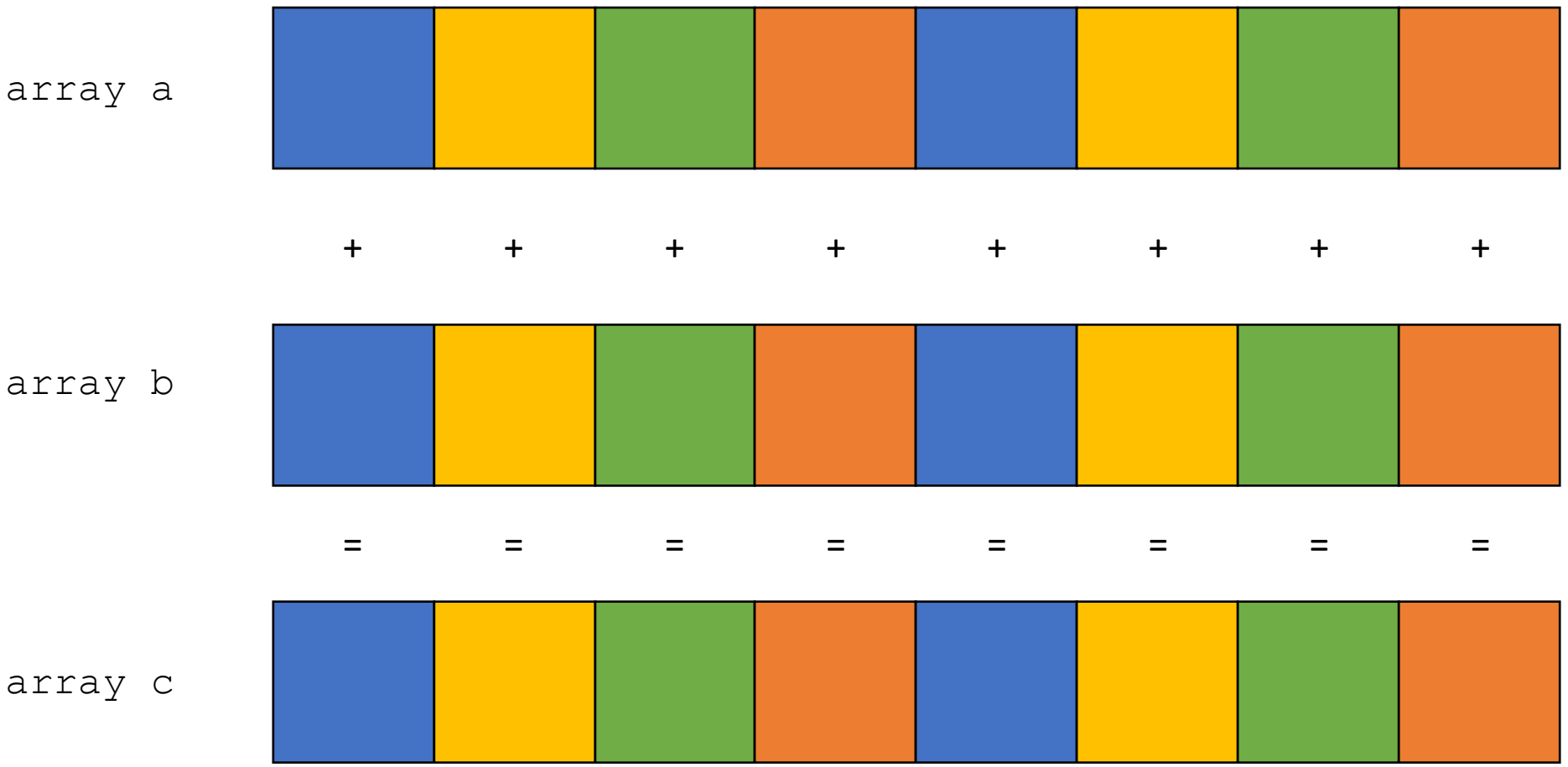
Thread 0 - Blue
Thread 1 - Yellow
Thread 2 - Green
Thread 3 - Orange



Parallel Schedules

Computation can easily be divided into threads

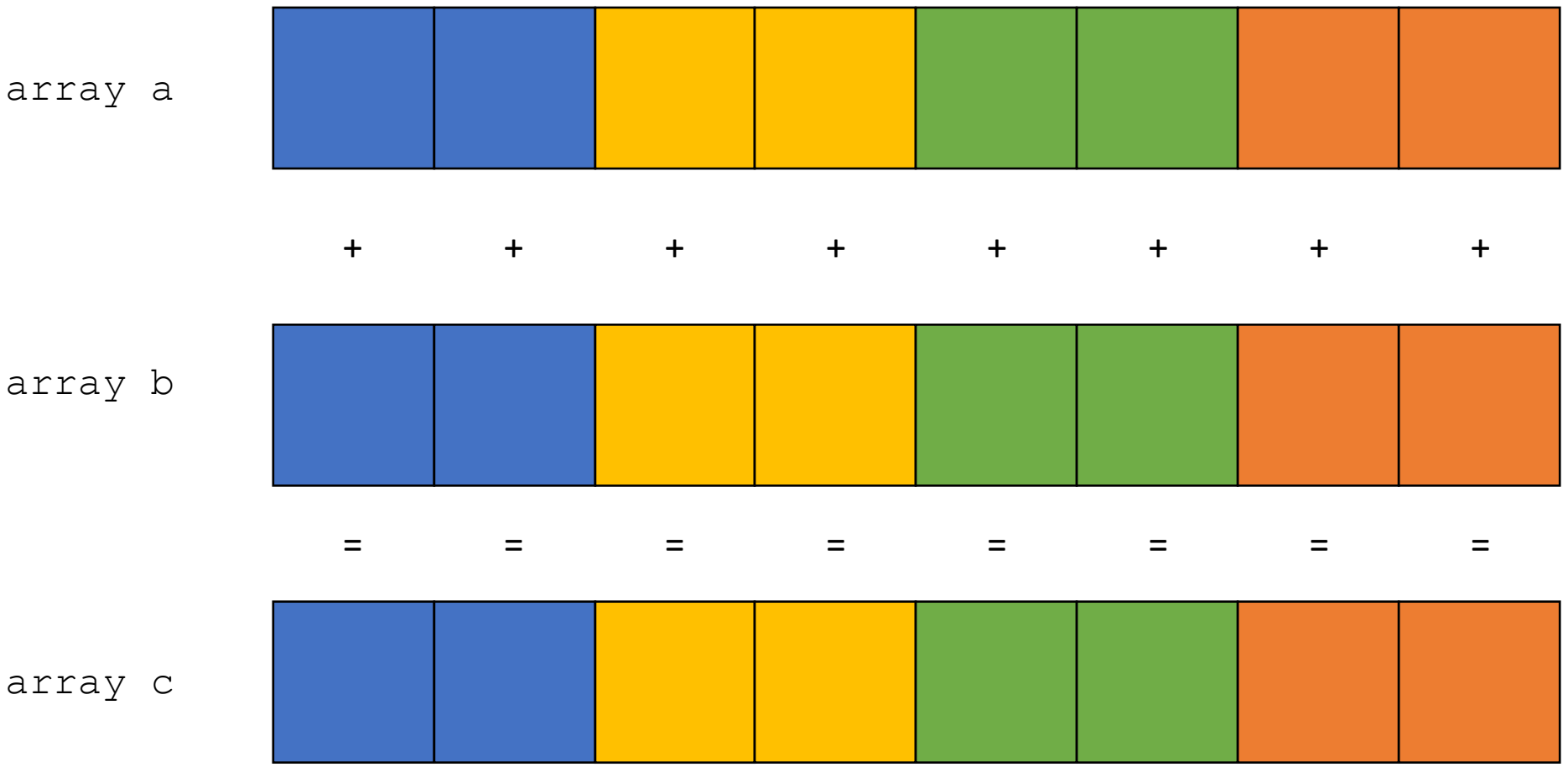
Thread 0 - Blue
Thread 1 - Yellow
Thread 2 - Green
Thread 3 - Orange



Parallel Schedules

Computation can easily be divided into threads

Thread 0 - Blue
Thread 1 - Yellow
Thread 2 - Green
Thread 3 - Orange



Programming a GPU

- The problem: Vector addition

- Who can do it faster?

Let’s set up the CPU

```
void vec_add(double *a, double *b, double *c, int size,
int tid, int num_threads) {
  int chunk = size / num_threads;
  int start = chunk * tid;
  int end = start + chunk;
  for (int j = 0; j < OUTER; j++) {
    for (int i = start; i < end; i++) {
      a[i] = b[i] + c[i];
    }
  }
}
```

```
for (int i = 0; i < num_threads; i++)
  thread_array[i] = thread(vec_add, a, b, c, SIZE, i,
num_threads);

for (int i = 0; i < num_threads; i++)
  thread_array[i].join();
```

Now for the GPU

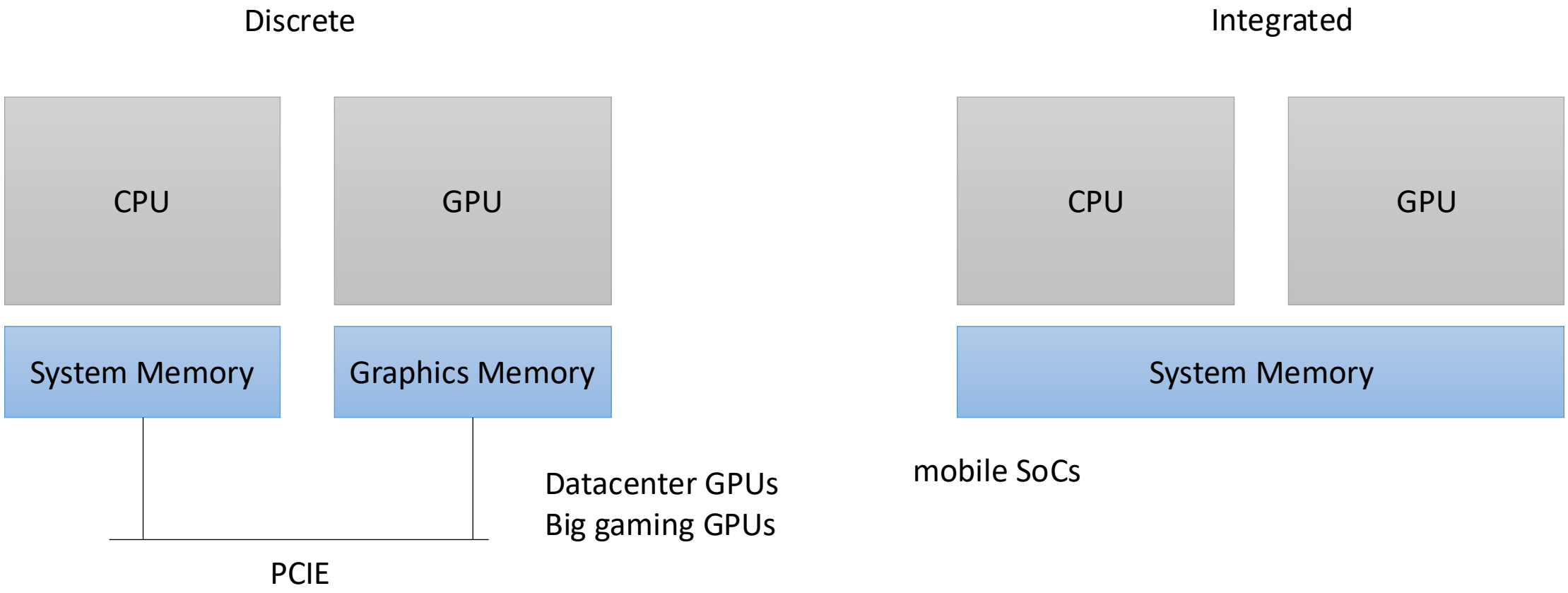
- It’s going to take a bit of work....

GPU set up

- We need to allocate and initialize memory

GPU set up

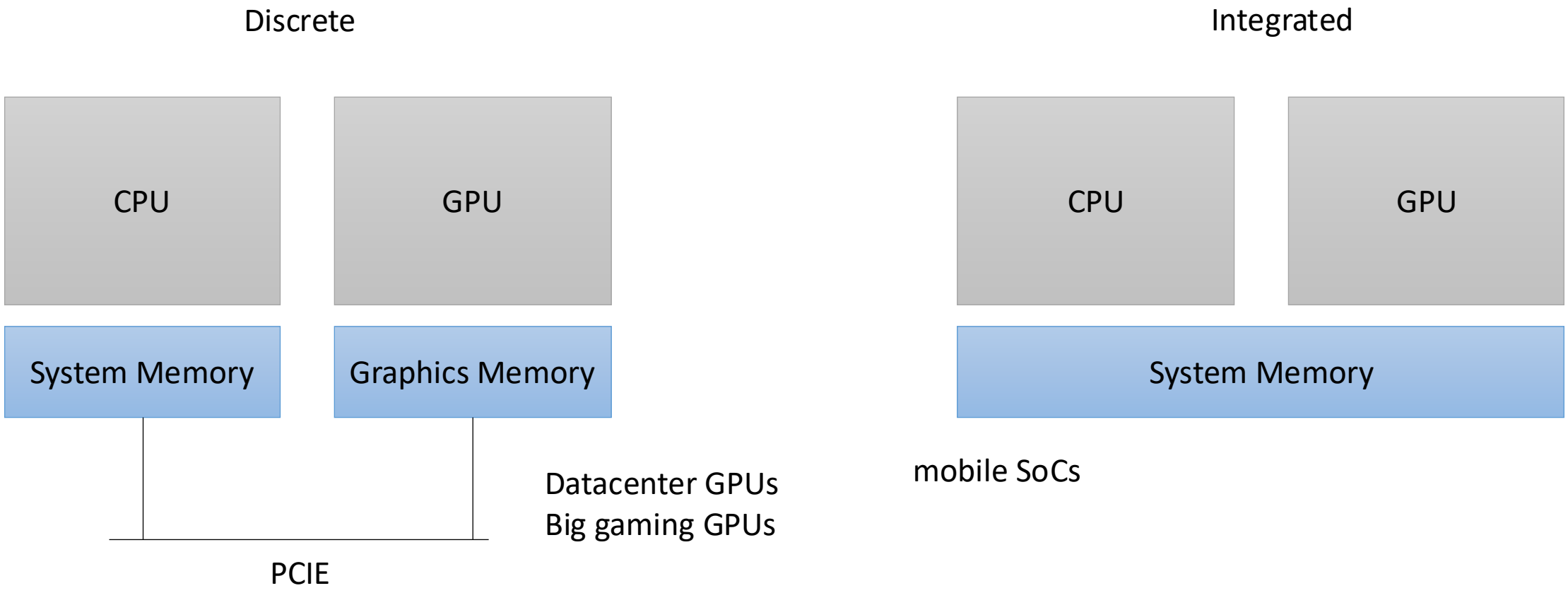
- GPUs come in two flavors



GPU set up

Pros and cons of each?

- GPUs come in two flavors

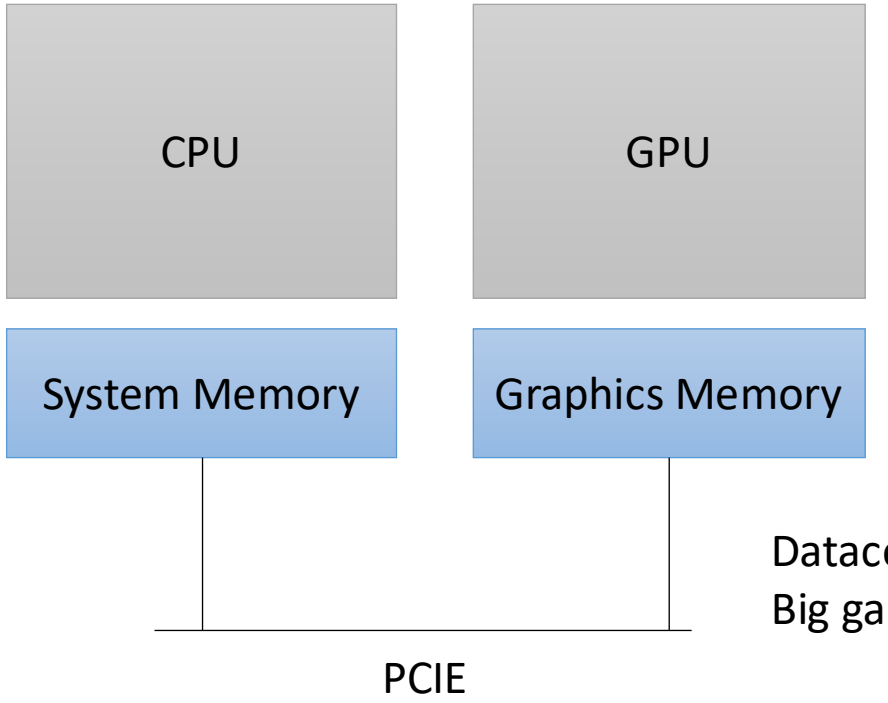


GPU set up

- GPUs come in two flavors

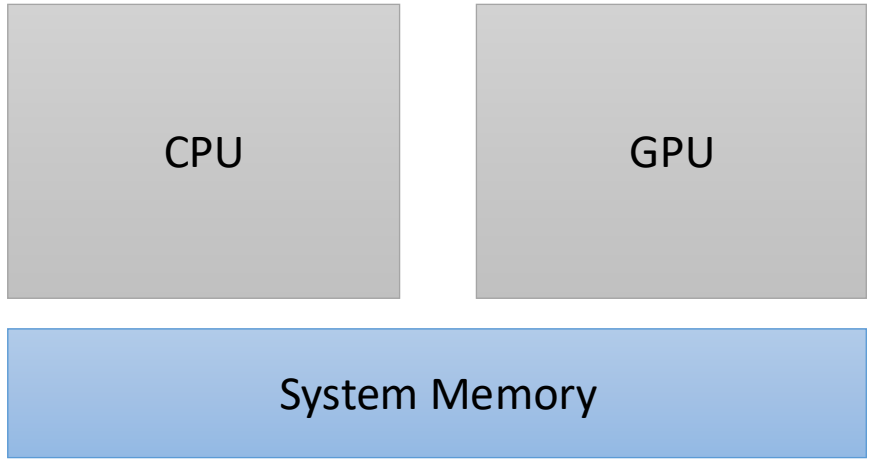
Pros and cons of each?
*Different types of memory for discrete
*Swappable for discrete
*More energy efficient for integrated
*Better memory utilization for integrated

Discrete



Datacenter GPUs
Big gaming GPUs

Integrated



mobile SoCs

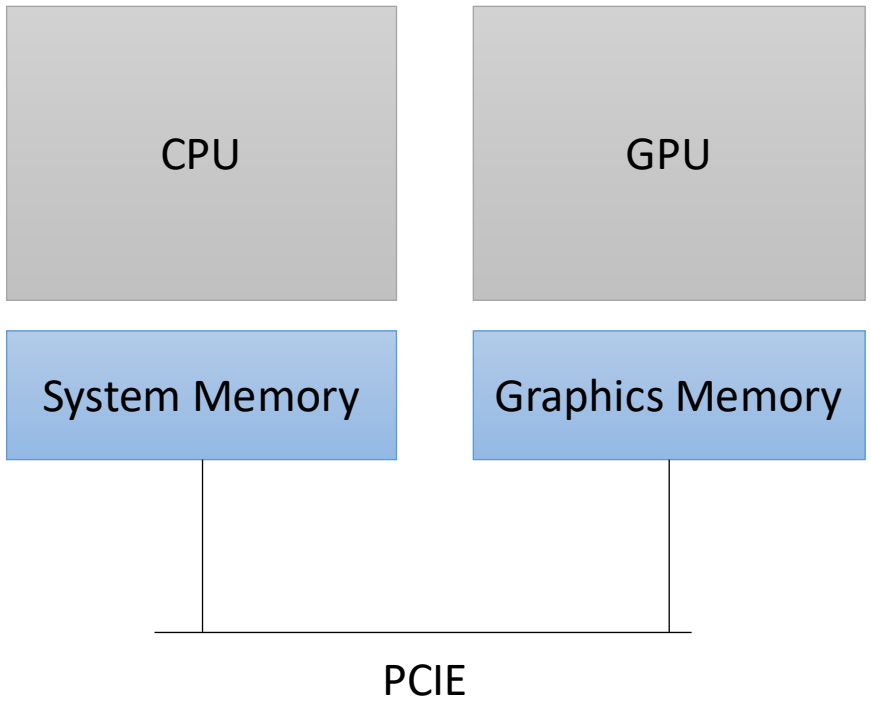
GPU set up

- GPUs come in two flavors

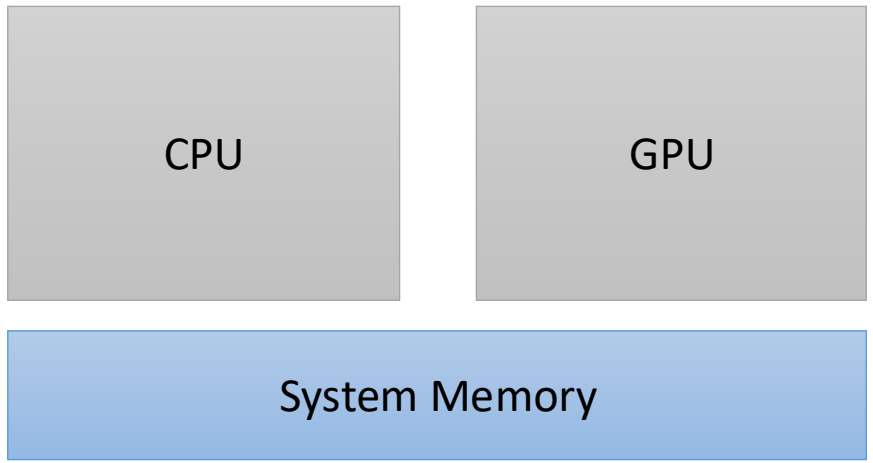
Although mobile GPUs share the system memory, Most still require you to program as if they didn't have shared memory.

Why?

Discrete



Integrated



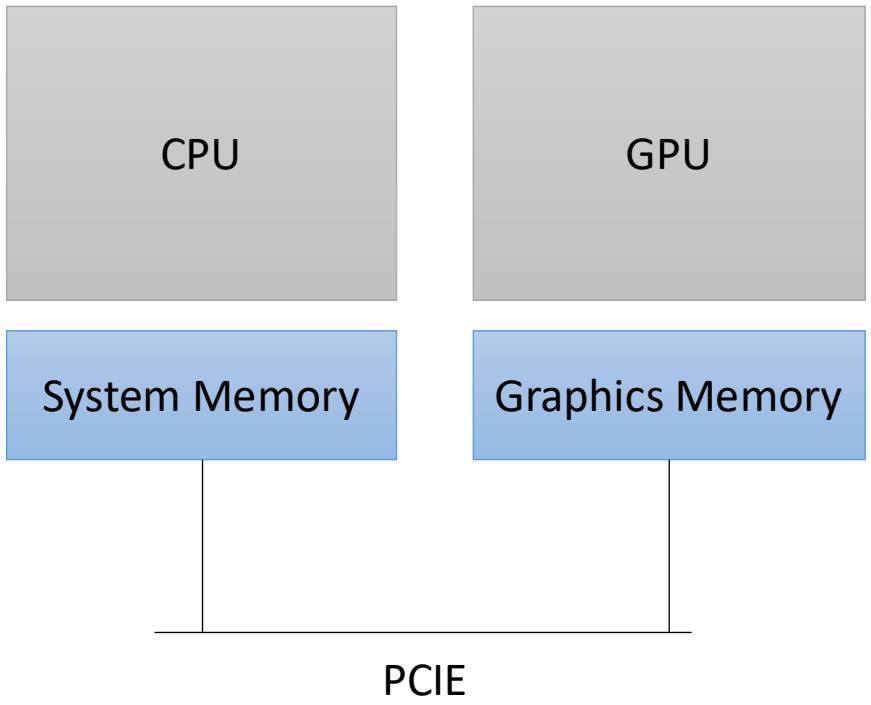
GPU set up

- GPUs come in two flavors

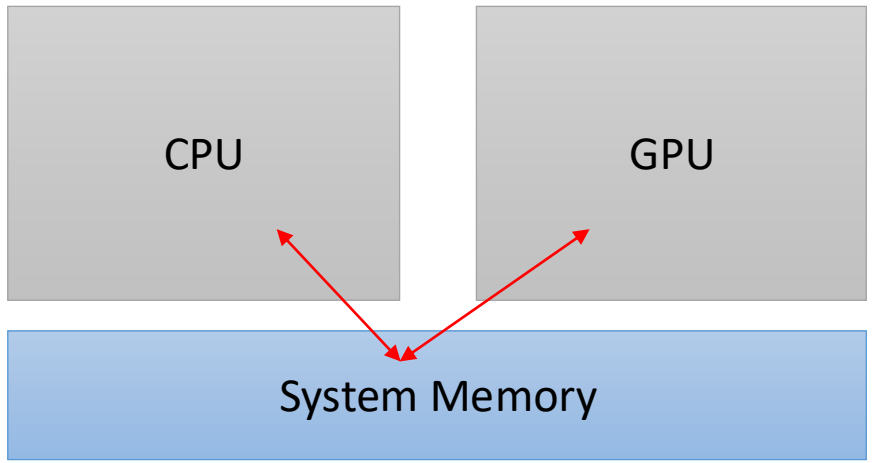
Although mobile GPUs share the system memory, Most still require you to program as if they didn't have shared memory.

Why?

Discrete



Integrated



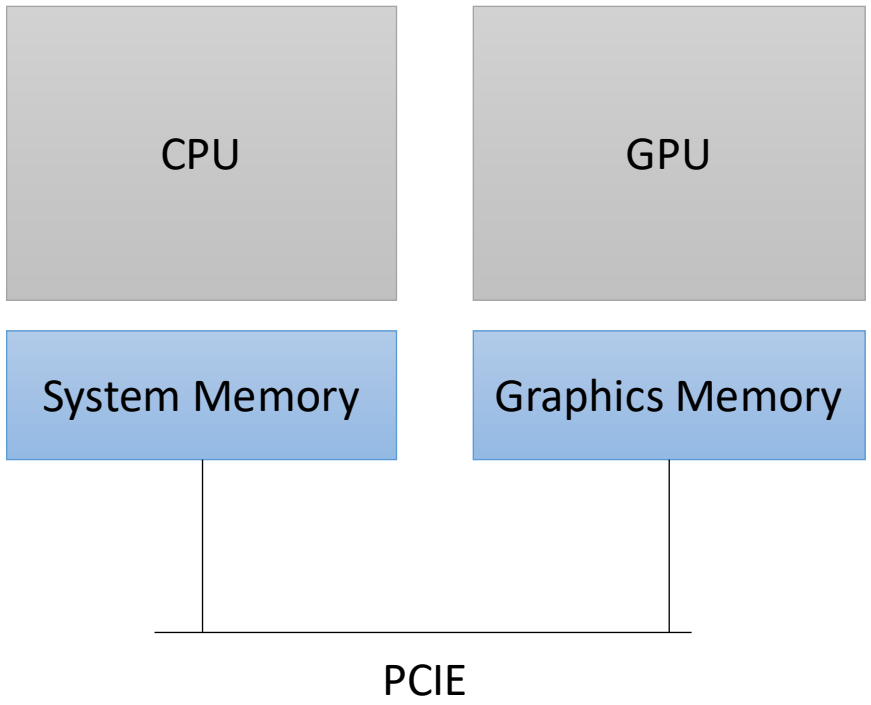
GPU set up

- GPUs come in two flavors

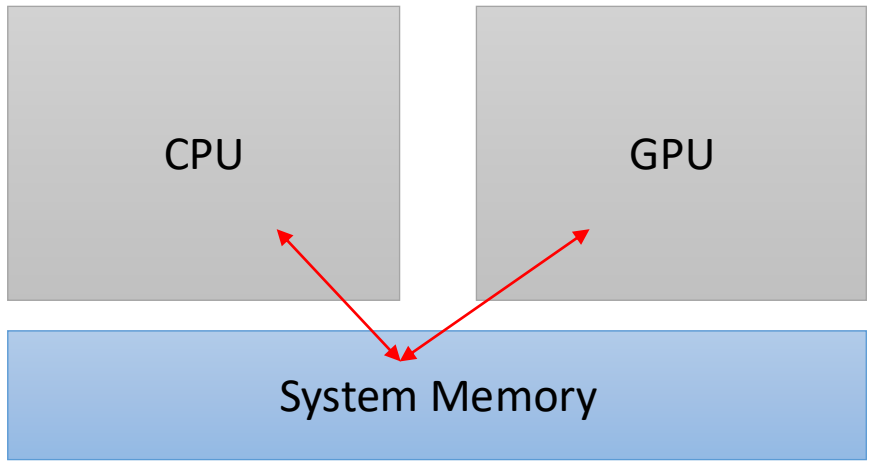
Although mobile GPUs share the system memory, Most still require you to program as if they didn't have shared memory.

Why?

Discrete



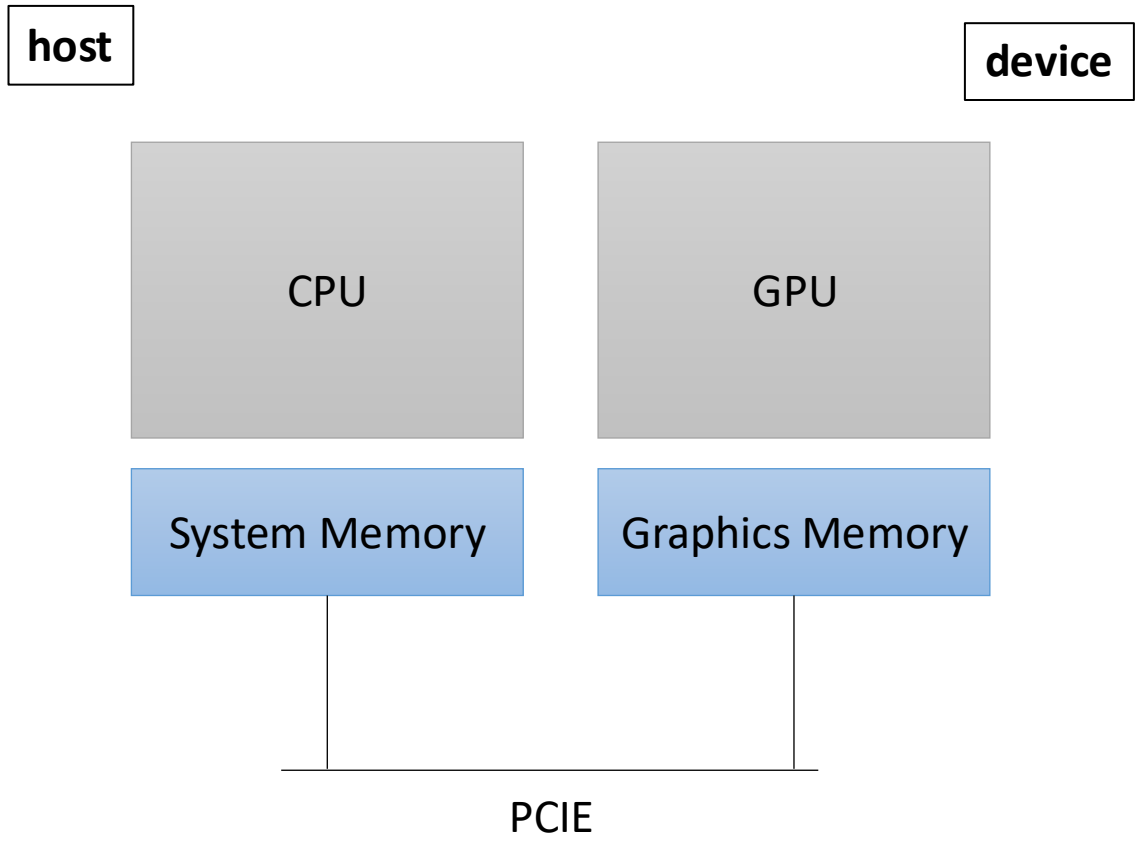
Integrated



In many cases, CPU-GPU communication is not fully supported. Coherence, fences, and RMWs might not be supported.

GPU set up

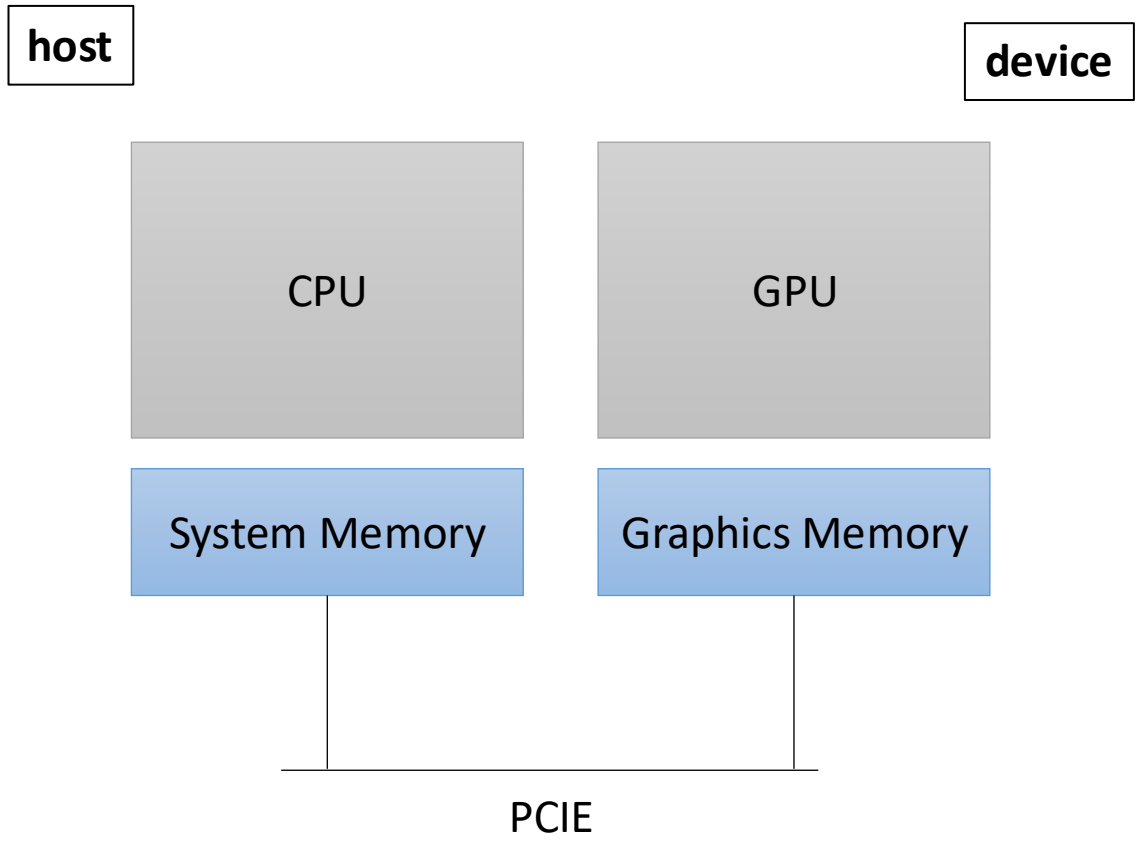
- Our heterogeneous, parallel, programming model



GPU set up

- Our heterogeneous, parallel, programming model

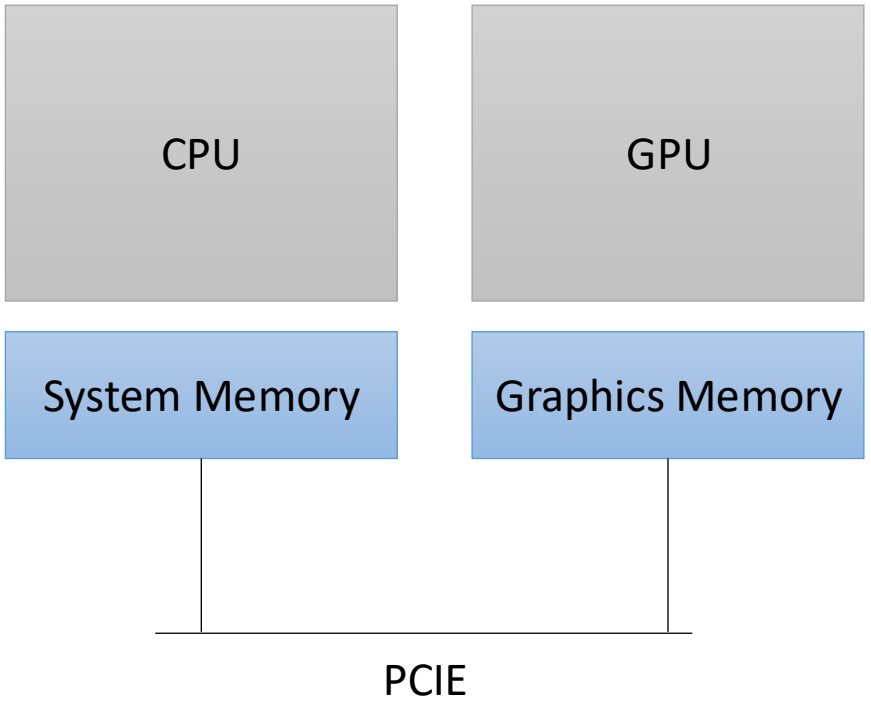
The host (CPU) will write a C++-like program that allocates and sets up memory on the GPU. The host will then call a GPU program called a kernel.



GPU set up

How do we allocate memory on a CPU?

- Our heterogeneous, parallel, programming model

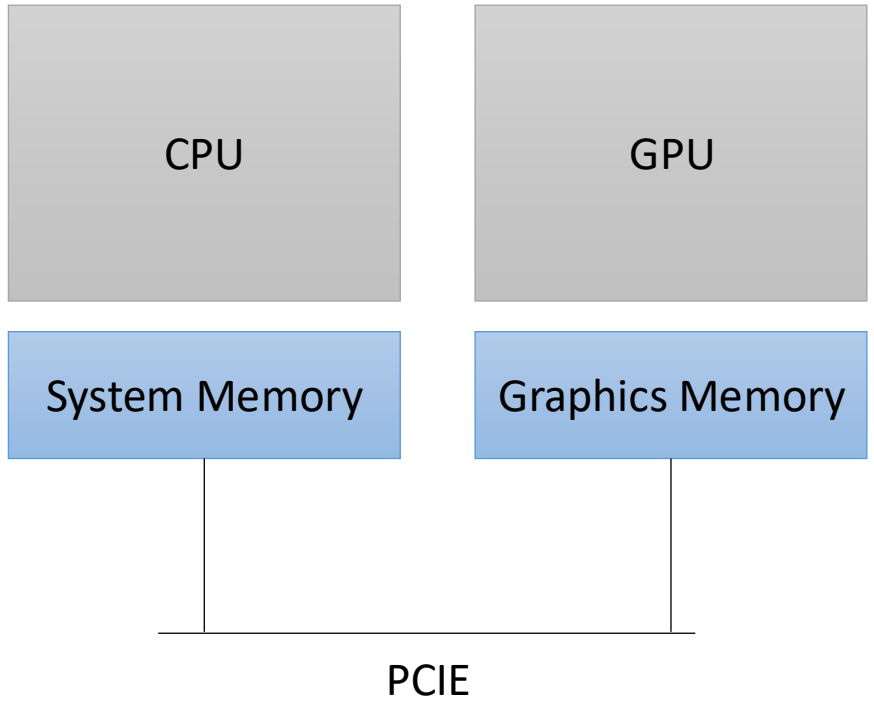


GPU set up

How do we allocate CPU memory on the host?

- Our heterogeneous, parallel, programming model

```
int *x = (int*) malloc(sizeof(int)*SIZE);
```

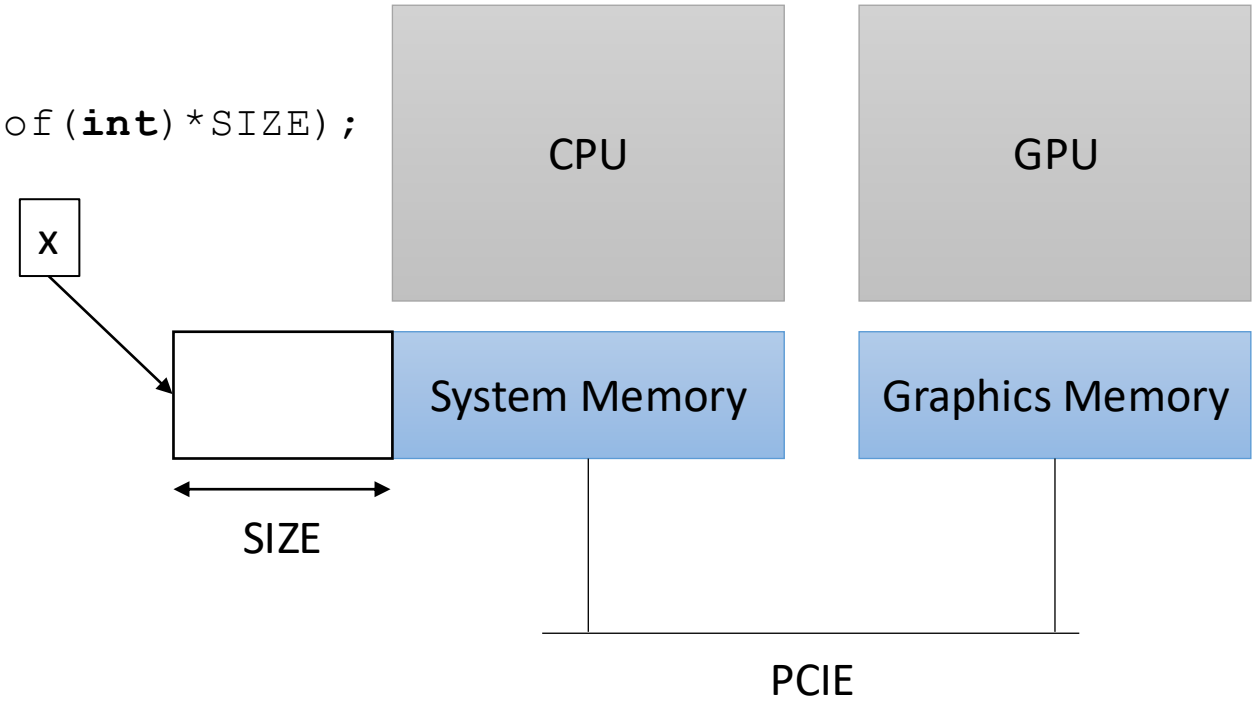


GPU set up

How do we allocate CPU memory on the host?

- Our heterogeneous, parallel, programming model

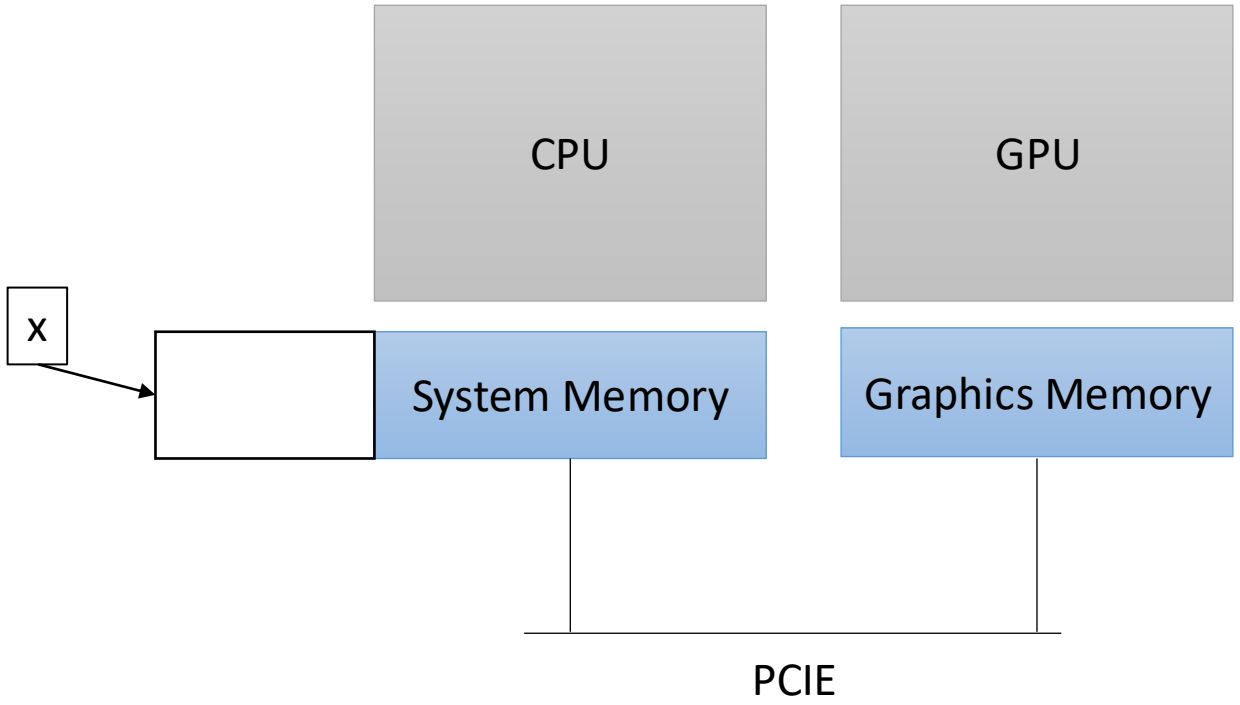
```
int *x = (int*) malloc(sizeof(int)*SIZE);
```



GPU set up

We need to allocate GPU memory on the host

- Our heterogeneous, parallel, programming model

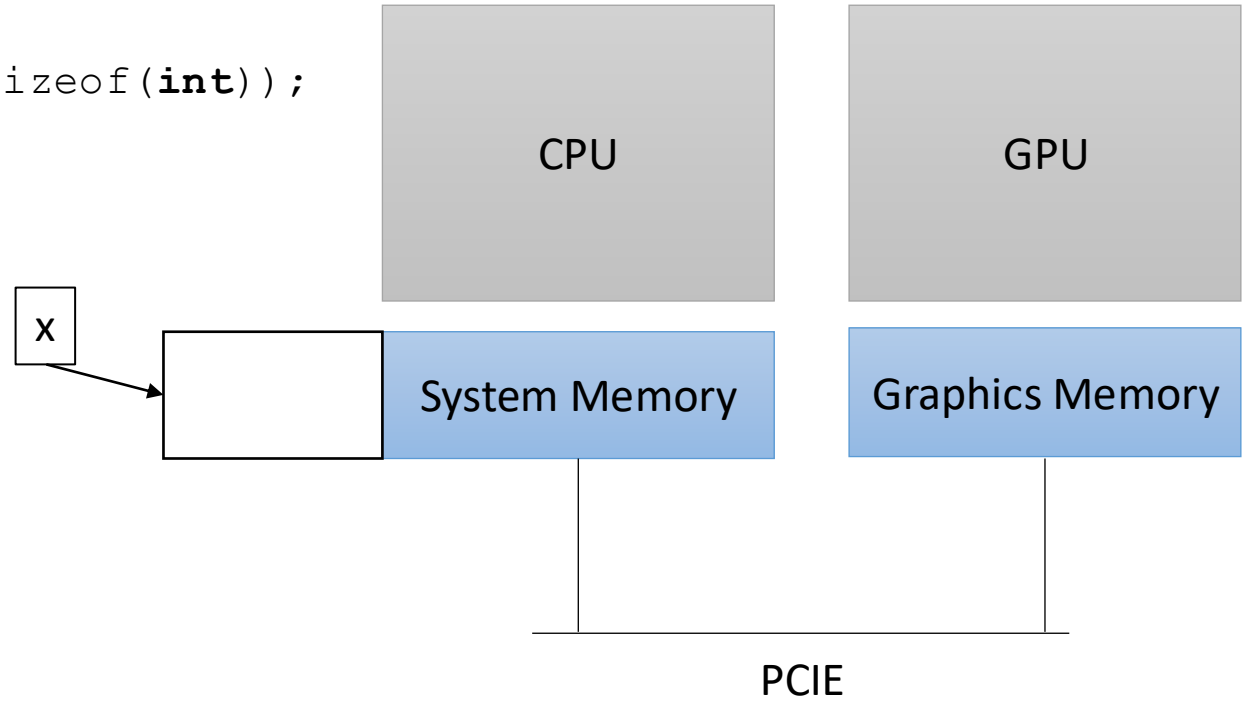


GPU set up

We need to allocate GPU memory on the host

- Our heterogeneous, parallel, programming model

```
int *d_x;
cudaMalloc(&d_x, SIZE*sizeof(int));
```

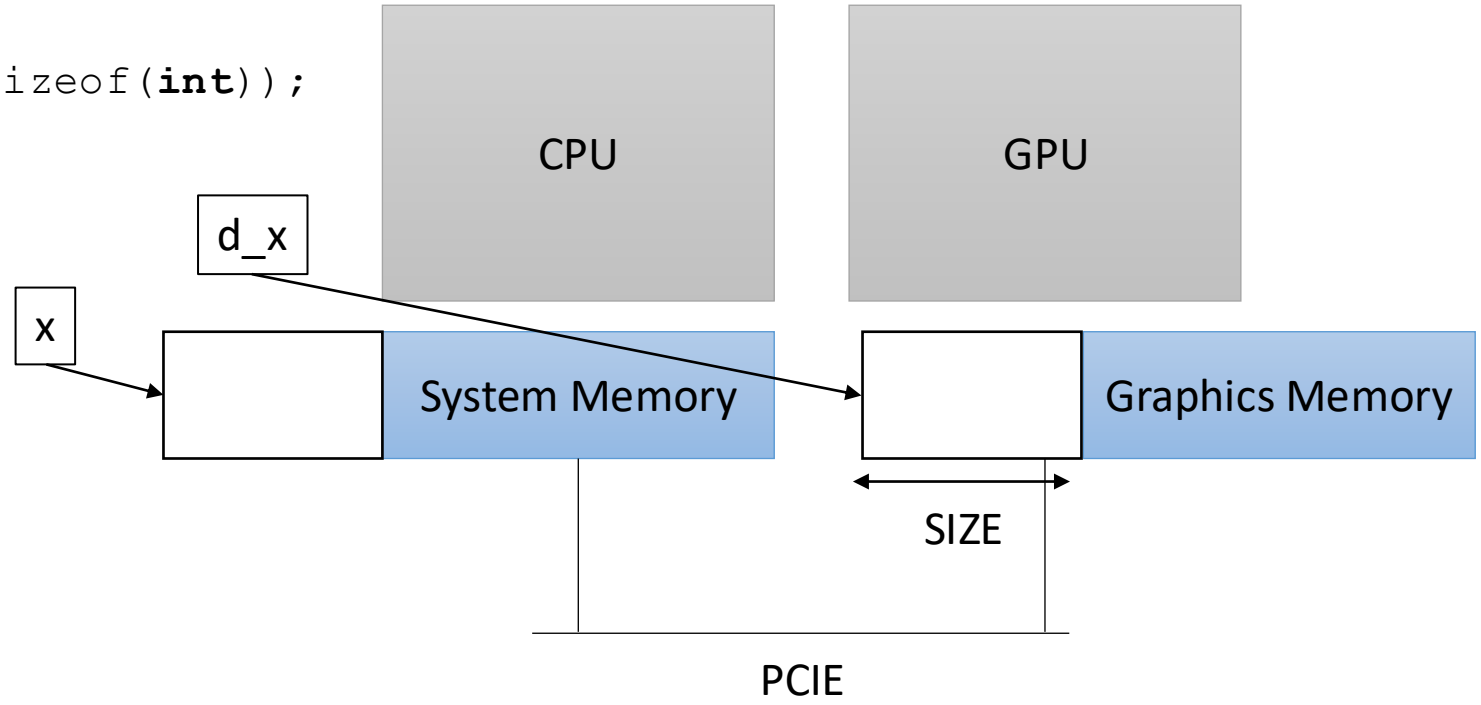


GPU set up

We need to allocate GPU memory on the host

- Our heterogeneous, parallel, programming model

```
int *d_x;
cudaMalloc(&d_x, SIZE*sizeof(int));
```

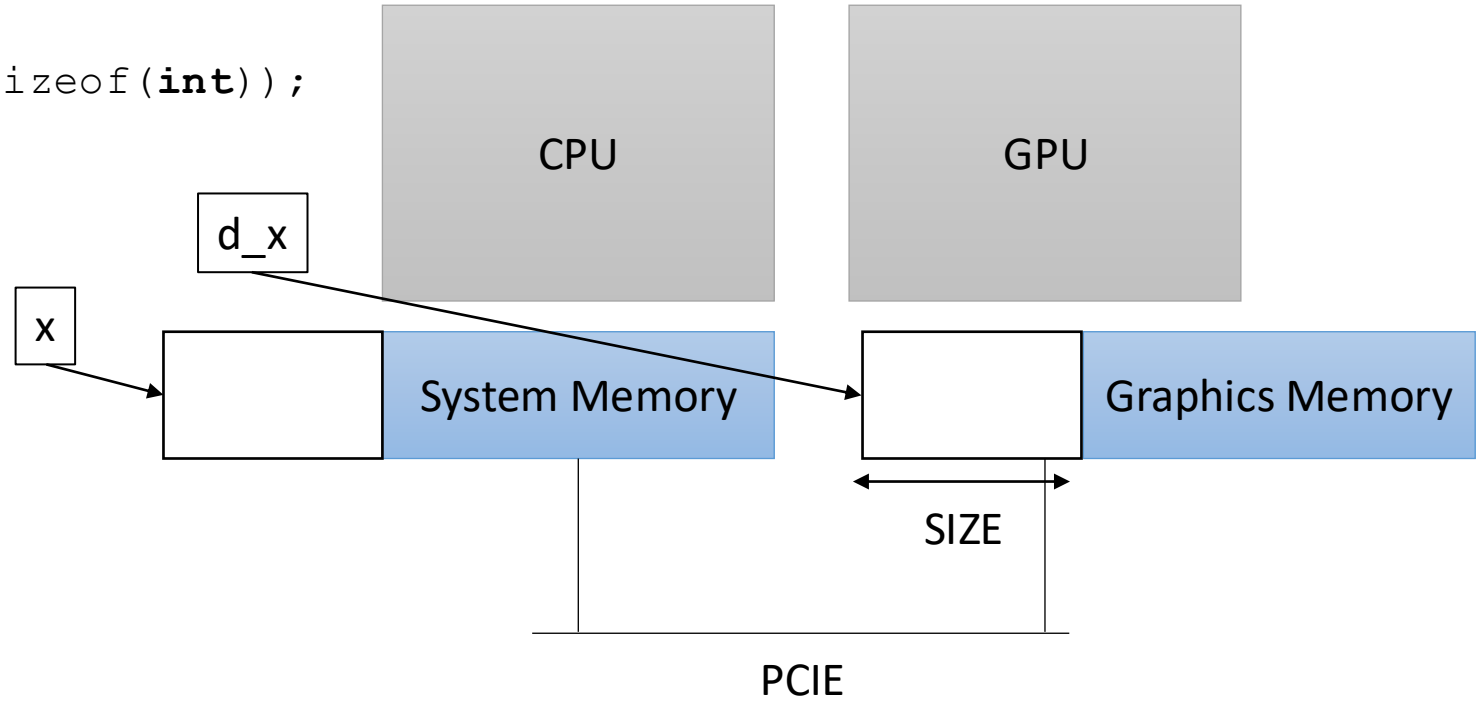


GPU set up

We need to allocate GPU memory on the host

- Our heterogeneous, parallel, programming model

```
int *d_x;
cudaMalloc(&d_x, SIZE*sizeof(int));
```

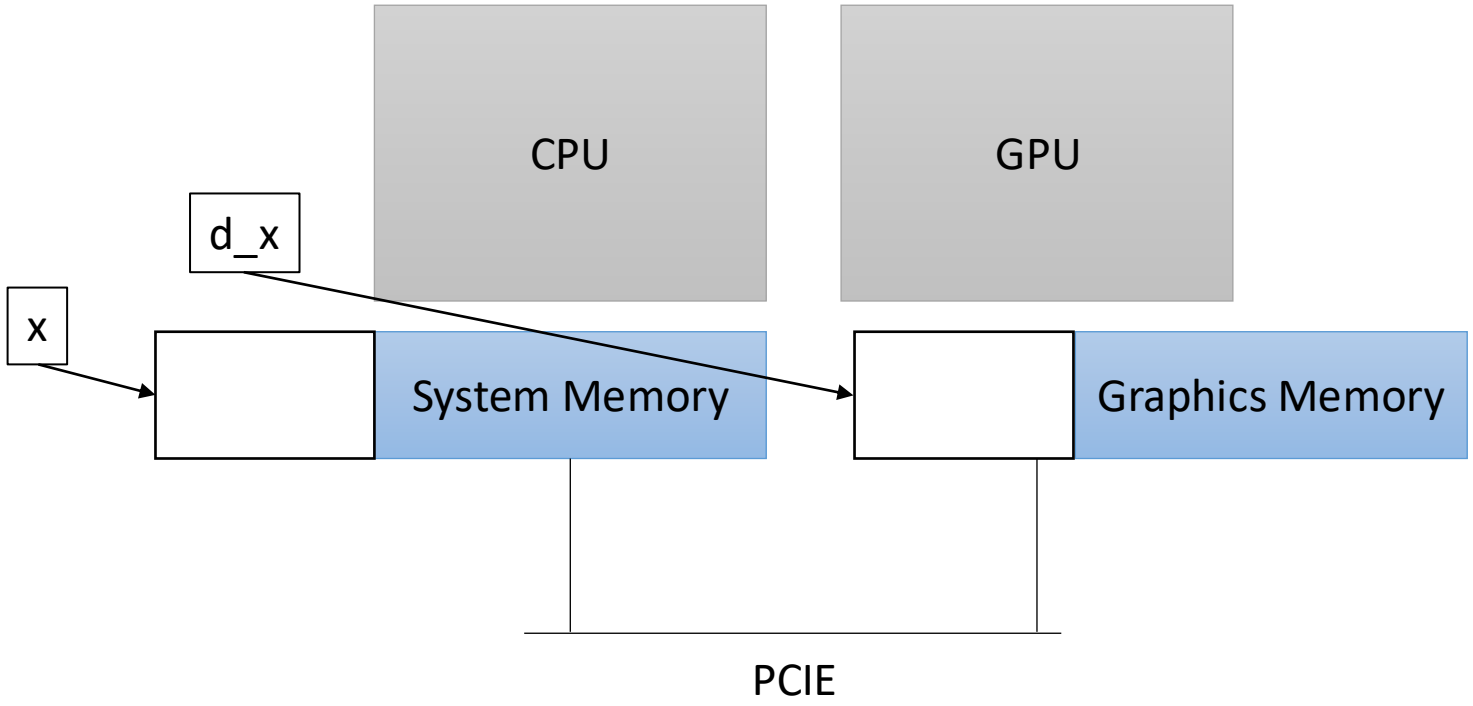


`d_x` is a pointer, in the CPU program, that points to memory on the GPU.

We can pass the pointer around, but the CPU cannot access the data i.e. `d_x[0]` gives an error!

GPU set up

- Our heterogeneous, parallel, programming model

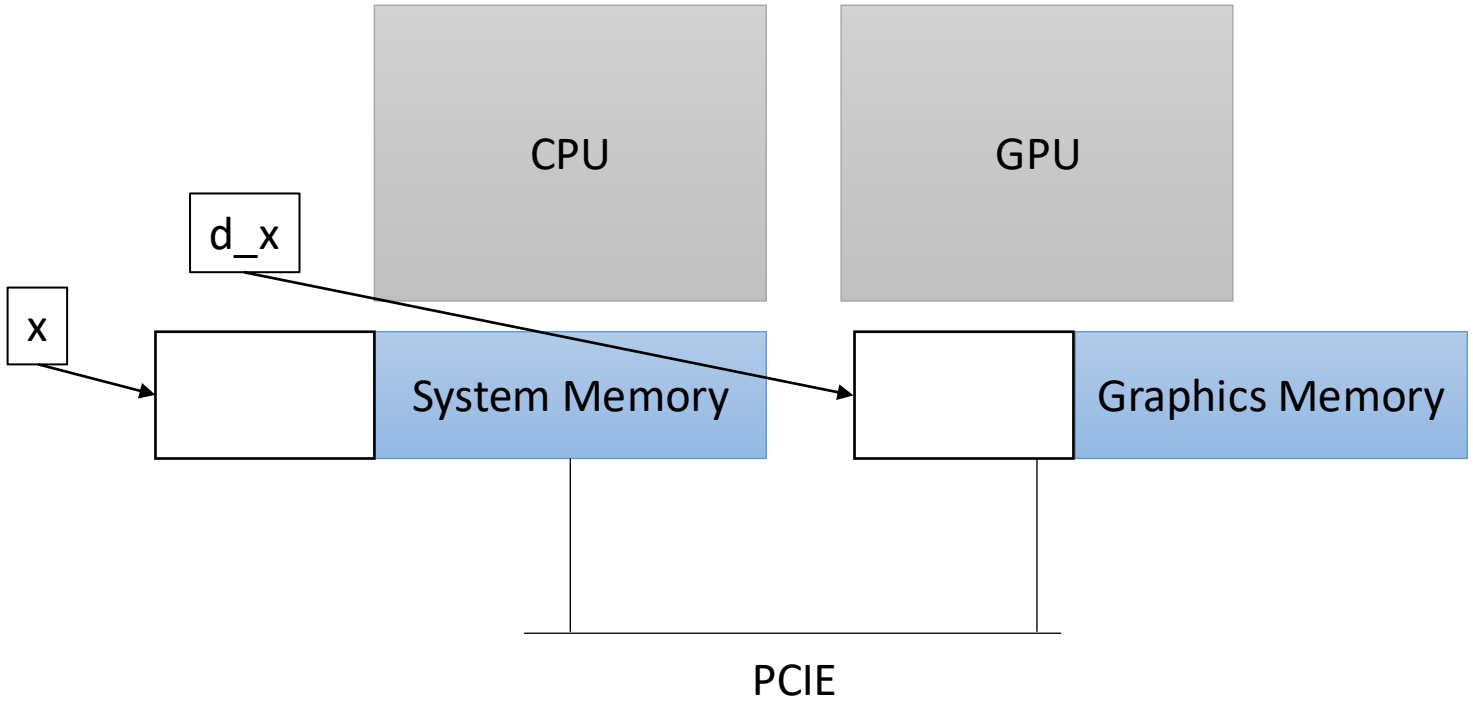


GPU set up

- Our heterogeneous, parallel, programming model

```
cudaFree(d_x)
```

device memory is scarce, and a program that allocates inside a loop without freeing runs out of GPU memory quickly.

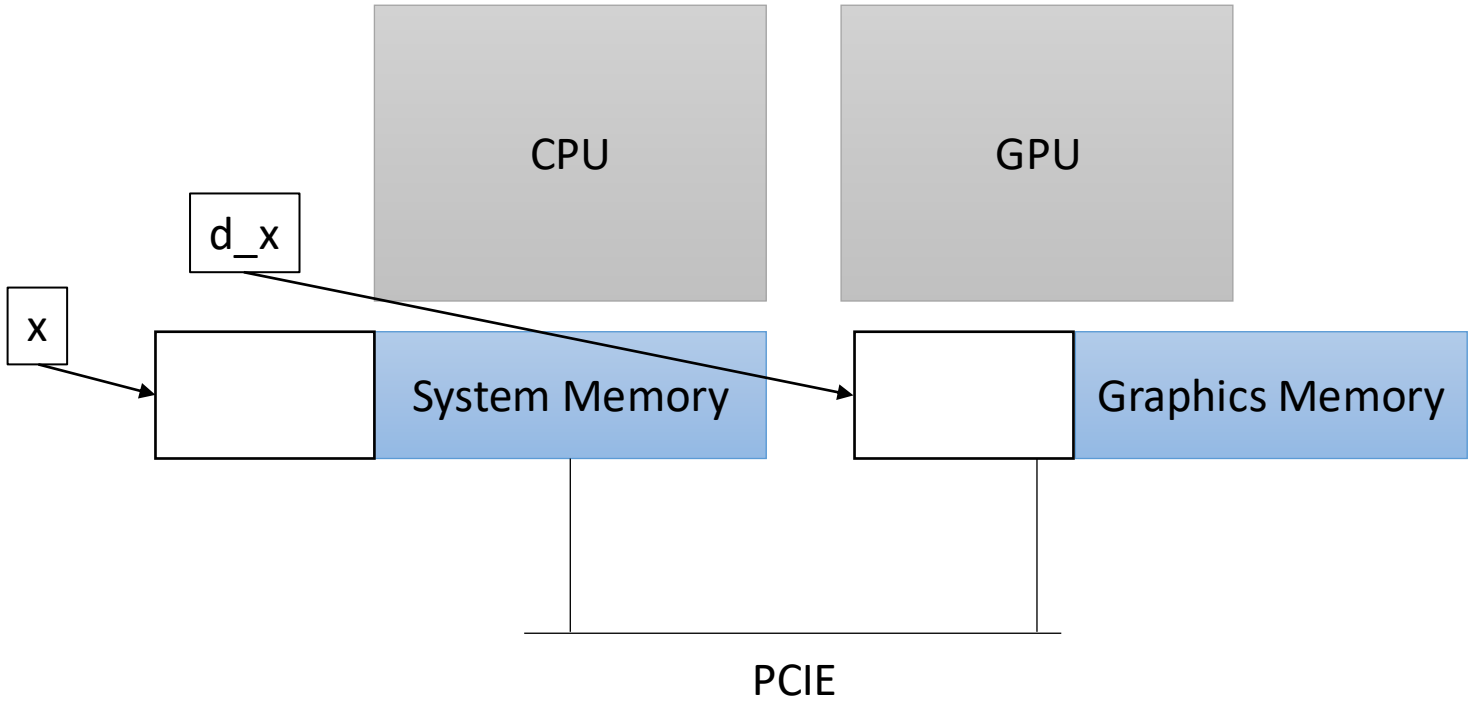


GPU set up

- Our heterogeneous, parallel, programming model

If we can't access d_x on the CPU, how do we initialize the memory?

GPU has no access to input devices e.g. disk



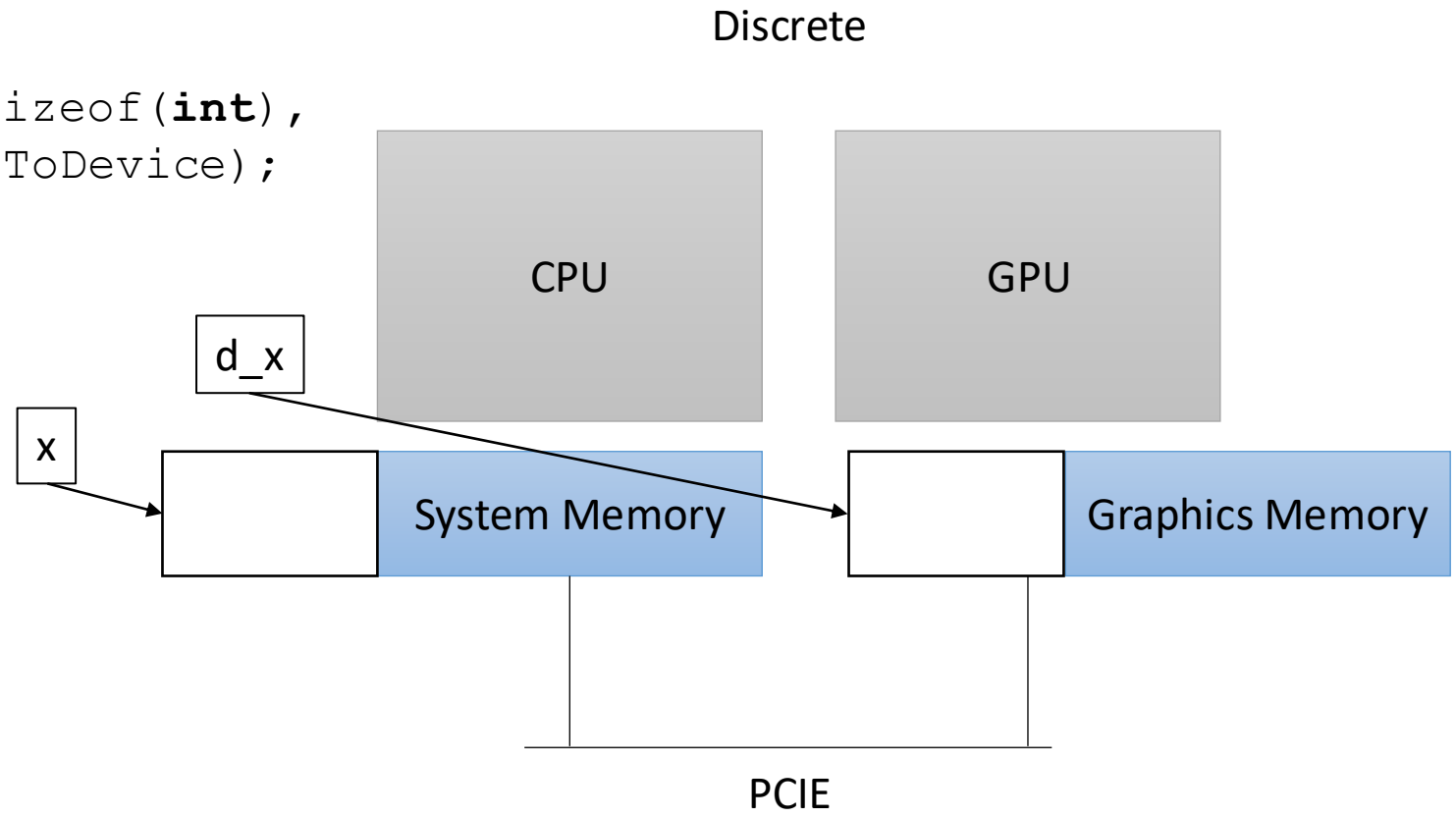
GPU set up

- Our heterogeneous, parallel, programming model

If we can't access d_x on the CPU, how do we initialize the memory?

GPU has no access to input devices e.g. disk

```
//initialize x on host
cudaMemcpy(d_x, x, SIZE*sizeof(int),
           cudaMemcpyHostToDevice);
```



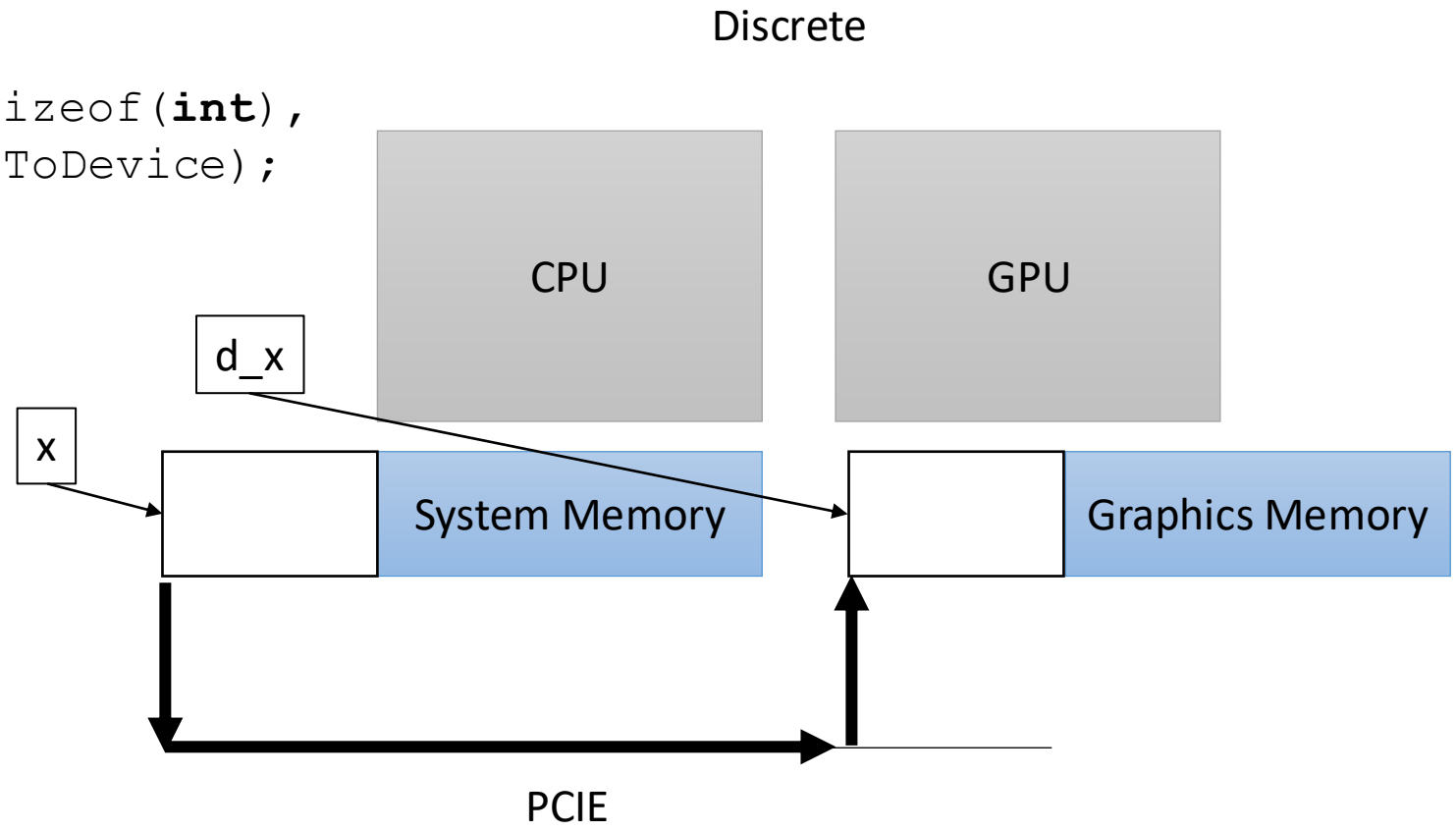
GPU set up

- Our heterogeneous, parallel, programming model

If we can't access d_x on the CPU, how do we initialize the memory?

GPU has no access to input devices e.g. disk

```
//initialize x on host
cudaMemcpy(d_x, x, SIZE*sizeof(int),
           cudaMemcpyHostToDevice);
```



How does this look in code?

```
float *a = new float[SIZE];
float *b = new float[SIZE];
float *c = new float[SIZE];

for (int i = 0; i < SIZE; i++) {
  a[i] = 0.0f;
  b[i] = i;
  c[i] = 1.0f;
}

float *d_a, *d_b, *d_c;
int e = 0;
e = cudaMalloc(&d_a, SIZE*sizeof(float));
e |= cudaMalloc(&d_b, SIZE*sizeof(float));
e |= cudaMalloc(&d_c, SIZE*sizeof(float));

e |= cudaMemcpy(d_a, a, SIZE*sizeof(float), cudaMemcpyHostToDevice);
e |= cudaMemcpy(d_b, b, SIZE*sizeof(float), cudaMemcpyHostToDevice);
e |= cudaMemcpy(d_c, c, SIZE*sizeof(float), cudaMemcpyHostToDevice);
```

The GPU Program

- Write a special function in your C++ code.
  - Called a Kernel
  - Use the new keyword `__global__`
  - Keywords in
    - OpenCL `__kernel`
    - Metal `kernel`

- Write it how you'd write any other function

The GPU Program

```
__global__ void vector_add(int * a, int * b, int * c, int size) {
    for (int i = 0; i < size; i++) {
      a[i] = b[i] + c[i];
   }
}
```

Quiz 11.

What keyword do we need to include for a function to be executed on the GPU using CUDA?

Select one

A. __kernel__

B. __global__

C. __this__

D. __gpu__

Scan to answer

forms.gle/9Bi6y8tjrNJW51iw6

chart: docs.google.com/forms/d/e/1FAIpQLSclk6ovyBEa3PJZYObGvoL2y5oHdGaCi2ABDs5X1cmriyDlJQ/viewanalytics

The GPU Program

```
__global__ void vector_add(int * a, int * b, int * c, int size) {
    for (int i = 0; i < size; i++) {
      a[i] = b[i] + c[i];
   }
}
```

calling the function

```
vector_add<<<1,1>>>(d_a, d_b, d_c, size);
```

The GPU Program

```
__global__ void vector_add(int * a, int * b, int * c, int size) {
    for (int i = 0; i < size; i++) {
      a[i] = b[i] + c[i];
   }
}
```

What in the world?
special new CUDA syntax. We will talk more soon

calling the function

```
vector_add<<<1,1>>>(d_a, d_b, d_c, size);
```

The GPU Program

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    for (int i = 0; i < size; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

Pass in pointers to memory on the device

calling the function

```
vector_add<<<1,1>>>(d_a, d_b, d_c, size);
```

Quiz 11.

The host (CPU) will write a C++-like program that allocates and sets up memory on the GPU. The host will then call a GPU program called a kernel. Is this correct?

Select one

A. True

B. False

Scan to answer

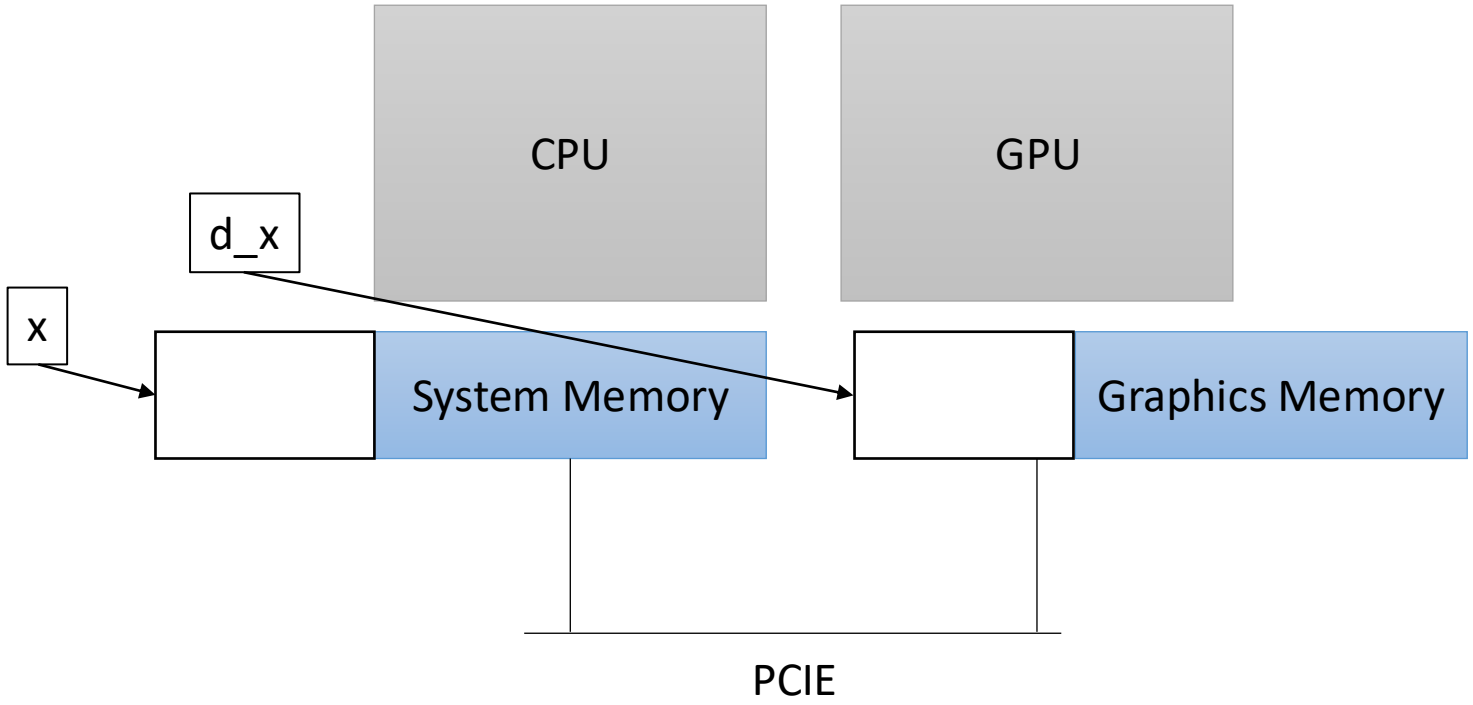
forms.gle/LjxX4hGcHhz4AfYg7

chart: docs.google.com/forms/d/e/1FAIpQLSfHfxGhnzTkZhUPARK6gMAjoYf8RmV8n-Xc4zJdaIioAr7ylQ/viewanalytics

GPU set up

- Our heterogeneous, parallel, programming model

Remember, GPU needs to access its own memory



The GPU Program

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    for (int i = 0; i < size; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

calling the function

```
vector_add<<<1,1>>>(d_a, d_b, d_c, size);
e |= cudaDeviceSynchronize();
```

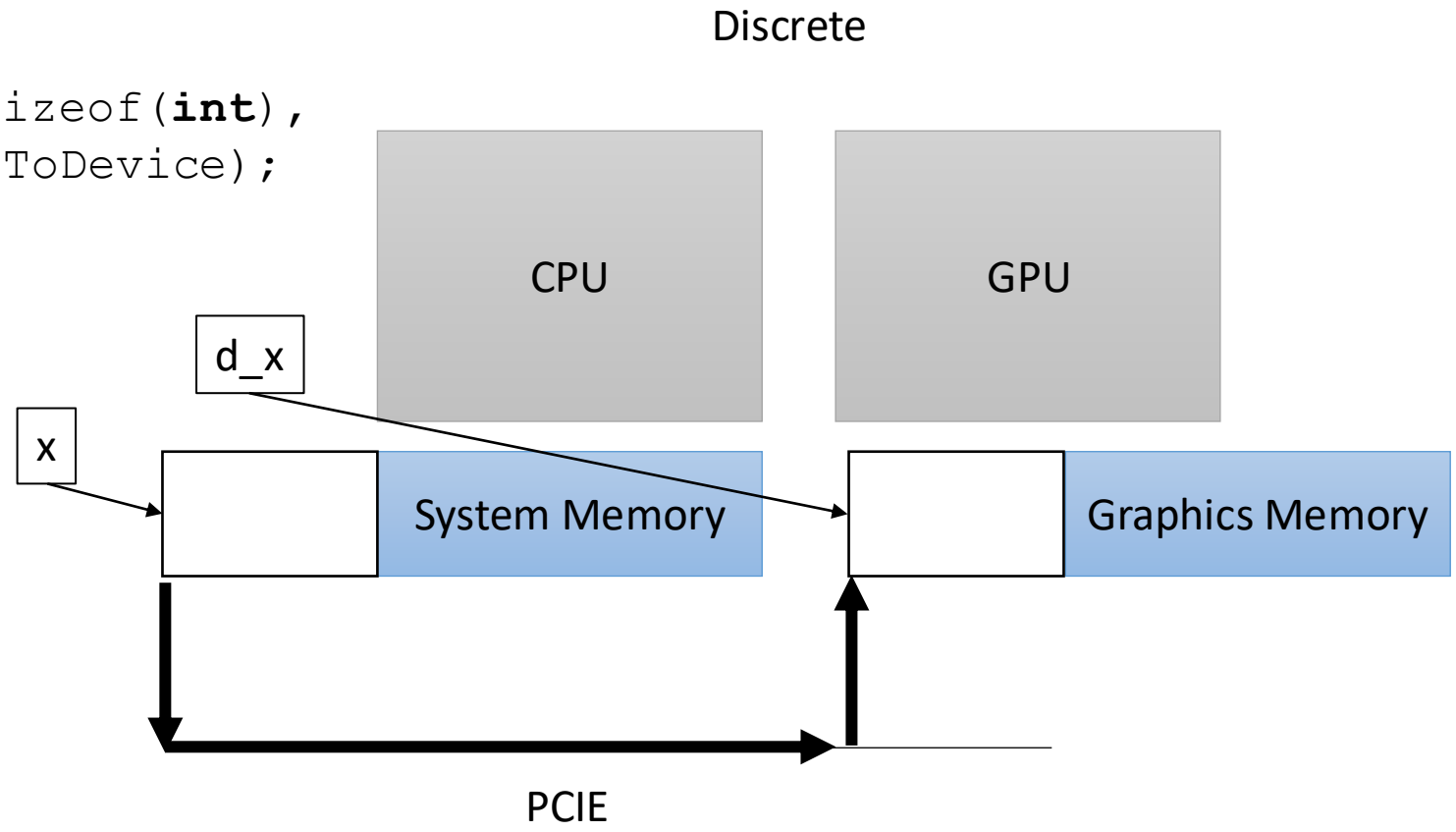
Constants can be passed in regularly

To make sure that it is finished

```
$ nvcc main.cu -o main
```

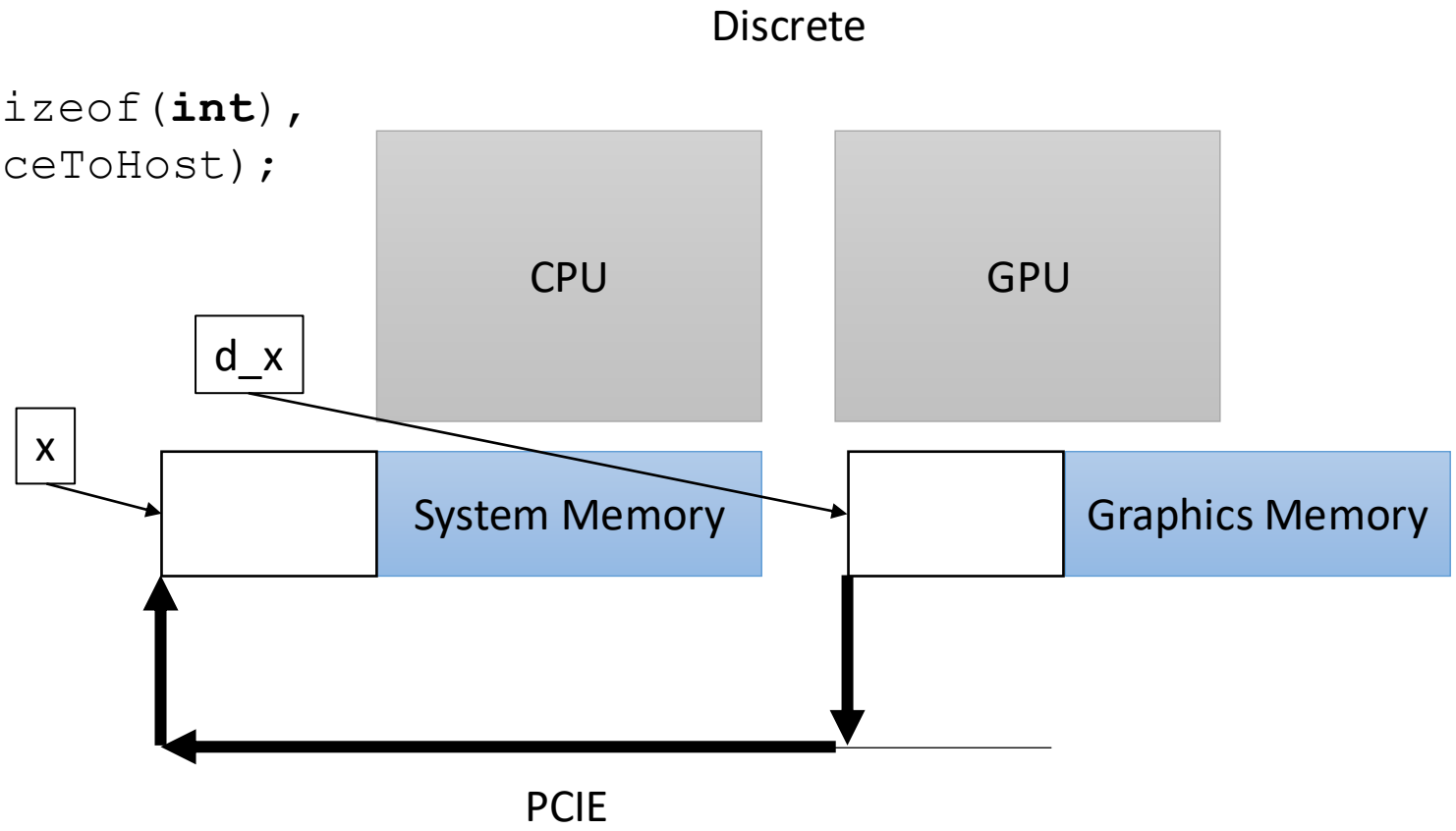
GPU set up

```
//initialize x on host
cudaMemcpy(d_x, x, SIZE*sizeof(int),
           cudaMemcpyHostToDevice);
```



GPU set up

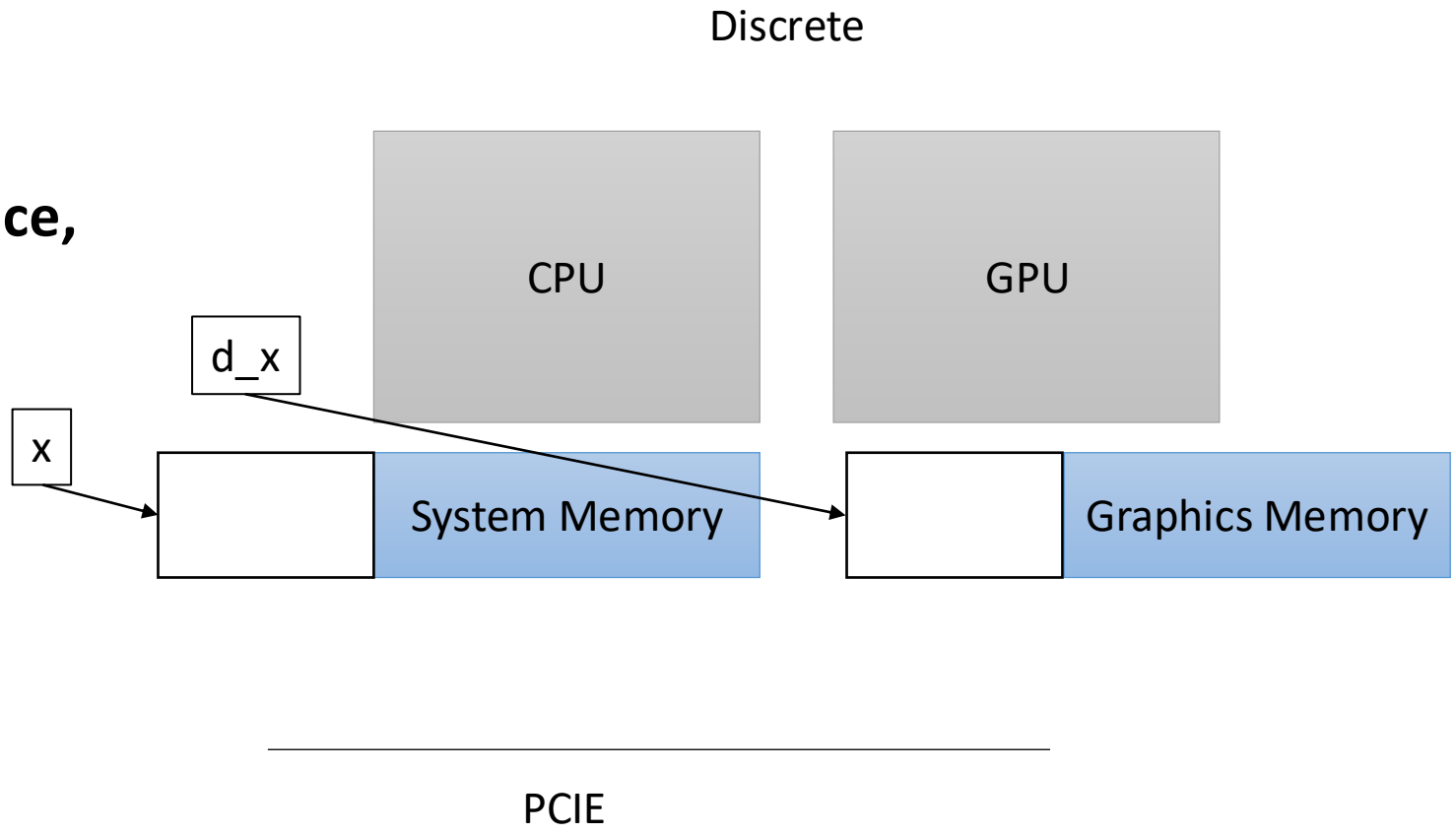
```
//initialize x on host
cudaMemcpy(x, d_x, SIZE*sizeof(int),
           cudaMemcpyDeviceToHost);
```



GPU set up

PCIe moves tens of GB/s while device memory moves TB/s,

Keep data on the device, copy rarely



Quiz 11.

How do we initialize memory for a variable we aim to use in the GPU computation?

Select one

A. Using cudaMemcpy

B. Using memcpy from C++

C. Just declaring a new variable



Scan to answer

forms.gle/RB5L3Q8BrcgwwaxB8

chart: docs.google.com/forms/d/e/1FAIpQLSc_FxDbMvqjPnhcKIB1RA0gGa6tezTQENINVbLqm5lAC62xiQ/viewanalytics

Kernel constraints

- What can and can’t you do in a GPU kernel?
  - Print?
  - File I/O?
  - C++ standard library? E.g. vectors?
  - Memory allocation?
  - Atomics?

Kernel constraints

- What can and can’t you do in a GPU kernel?
  - Print?
    CUDA now supports printfs directly in the kernel.
  - File I/O?
    You won't be able to implement file I/O directly in CUDA C++, as the GPU does not connect directly to the file system.
  - C++ standard library? E.g. vectors?
    You can't use the STL in CUDA, but you may be able to use the Thrust library to do what you want.
  - Memory allocation?
    In general allocating memory dynamically from the gpu is not possible in CUDA or other GPU APIs.
  - Atomics?
    Every Nvidia GPU that is a core 84 or higher supports CUDA 1.1 or higher, and thus supports atomic operations.

The GPU Program

- If the same GPU also drives the display:
  The operating system expects the GPU kernel to be fast to render graphics.
  It kills the kernel if it takes more than ~5 seconds.

- A headless or second GPU has no limit." Students on

The GPU Program



1000x slower on GPU?
It didn’t do so well...

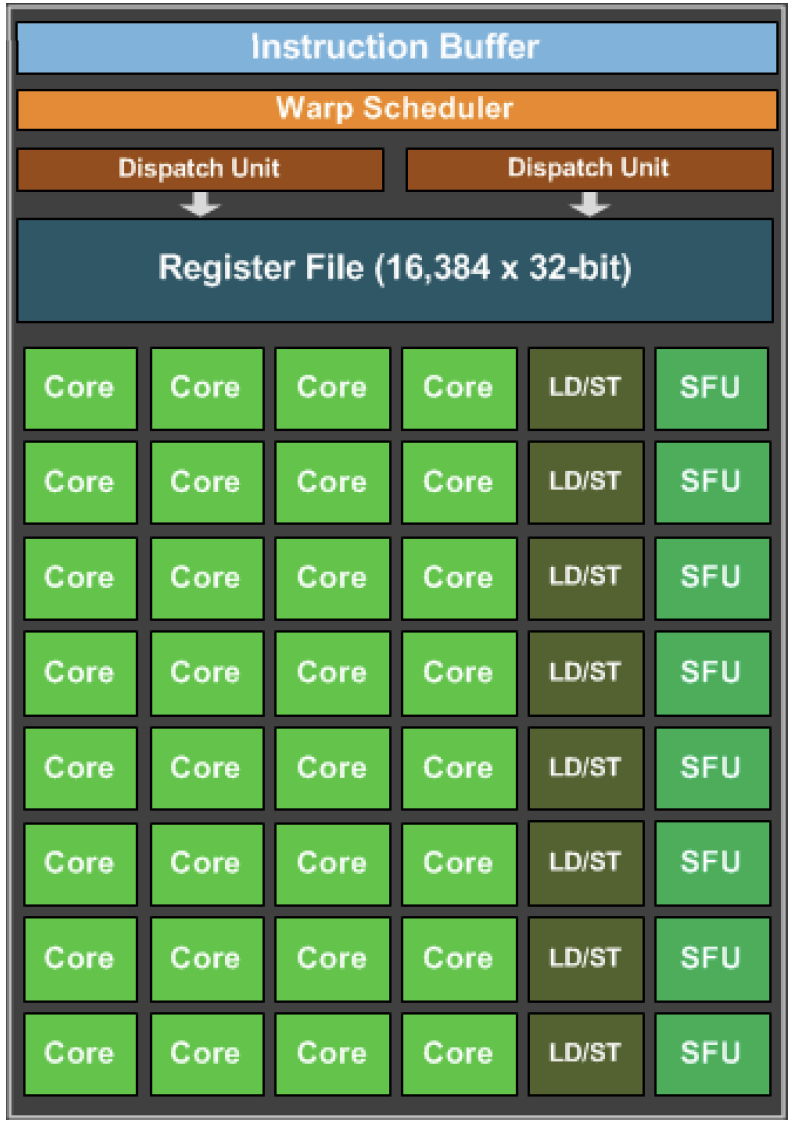
One GPU core is much slower than a single CPU core.

CPU spending silicon on caches, branch prediction and out-of-order logic,
GPU spending it on ALUs and a huge register file.

This saves energy and helps manage many cores.

First parallelization attempt

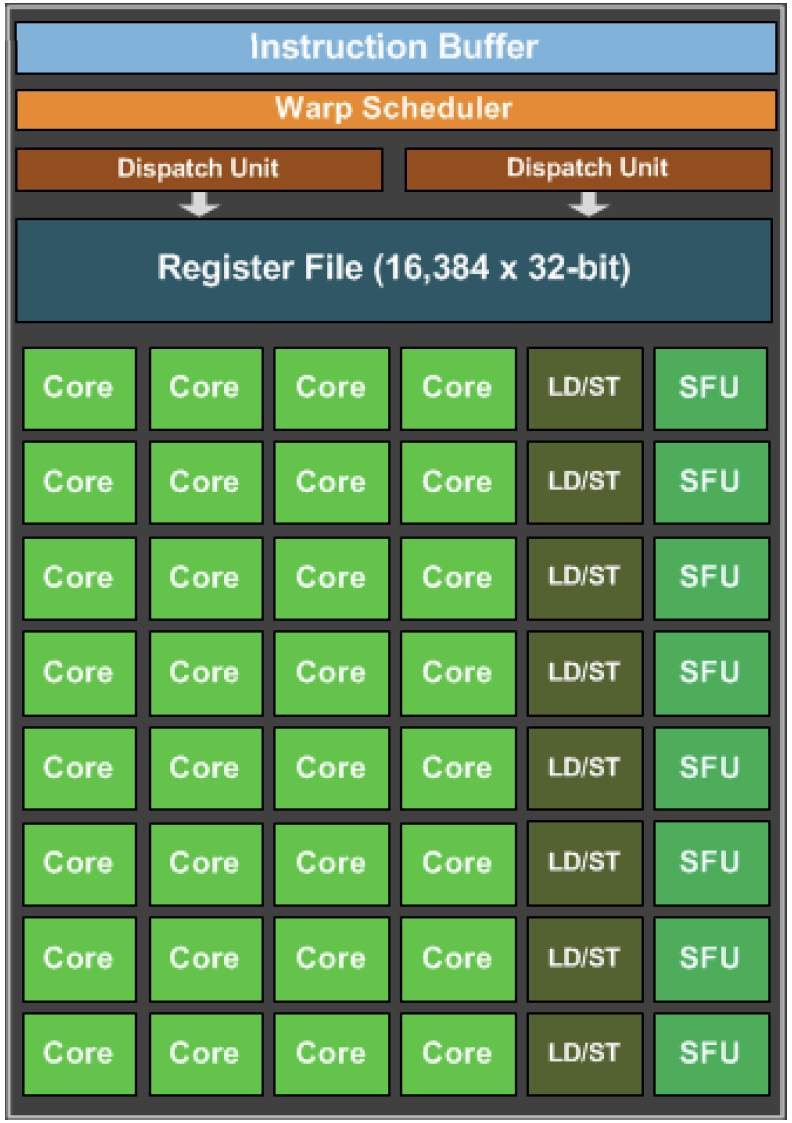
- Let’s look at some GPU documentation.
- The Maxwell whitepaper (Nvidia GPU microarchitecture, the 2014 generation.) shows a diagram of one of the GPU cores



https://www.techpowerup.com/gpu-specs/docs/nvidia-gtx-980.pdf

woah, 32 cores!

We should parallelize our application!



https://www.techpowerup.com/gpu-specs/docs/nvidia-gtx-980.pdf

First parallelization attempt

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    for (int i = 0; i < size; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

calling the function

```
vector_add<<<1,1>>>(d_a, d_b, d_c, size);
```

First parallelization attempt

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    for (int i = 0; i < size; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

calling the function

```
vector_add<<<1,32>>>(d_a, d_b, d_c, size);
```

number of threads to launch the program with

First parallelization attempt

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    int chunk_size = size/blockDim.x;
    int start = chunk_size * threadIdx.x;
    int end = start + chunk_size;
    for (int i = start; i < end; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

number of threads

calling the function

```
vector_add<<<1,32>>>(d_a, d_b, d_c, size);
```

First parallelization attempt

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    int chunk_size = size/blockDim.x;
    int start = chunk_size * threadIdx.x;
    int end = start + chunk_size;
    for (int i = start; i < end; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

number of threads
thread id

calling the function

```
vector_add<<<1,32>>>(d_a, d_b, d_c, size);
```

blockIdx.x is the block id.
blockDim.x is the number of threads per block.
threadIdx.x is the thread id within block.

First parallelization attempt

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    int chunk_size = size/blockDim.x;
    int start = chunk_size * threadIdx.x;
    int end = start + chunk_size;
    for (int i = start; i < end; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

calling the function

```
vector_add<<<1,32>>>(d_a, d_b, d_c, size);
```

blockIdx.x is the block id.
blockDim.x is the number of threads per block.
threadIdx.x is the thread id within block.

When you write <<<1, 32>>> the integers are converted to dim3(1,1,1) and dim3(32,1,1), so only the x field is non-trivial and threadIdx.x is the only one you read.

First parallelization attempt

That gives us about 14x speedup!

First parallelization attempt



Getting better but we have a long ways to go!

Quiz 11.

Using a few sentences, give examples of workloads that benefit from GPU parallelism.

Free response: a few sentences on the form

Scan to answer

forms.gle/xcAa74cFCvfMcVhT7

chart: docs.google.com/forms/d/e/1FAIpQLSfafsP0Ovn_31uhRJgjO4RqzZL0ir9-cI74cTtjh88RP87VAQ/viewanalytics

Quiz 11.

Using a few sentences, give examples of workloads that benefit from GPU parallelism.

Free response: a few sentences on the form

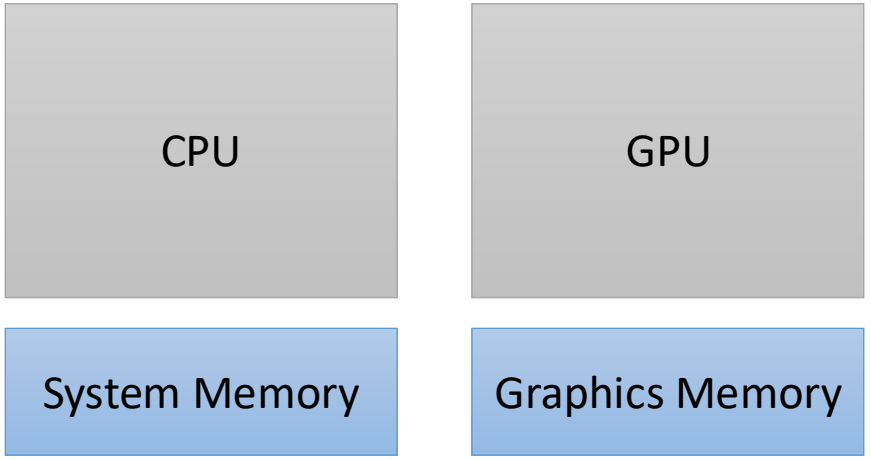
Data parallel.

Scan to answer

forms.gle/xcAa74cFCvfMcVhT7

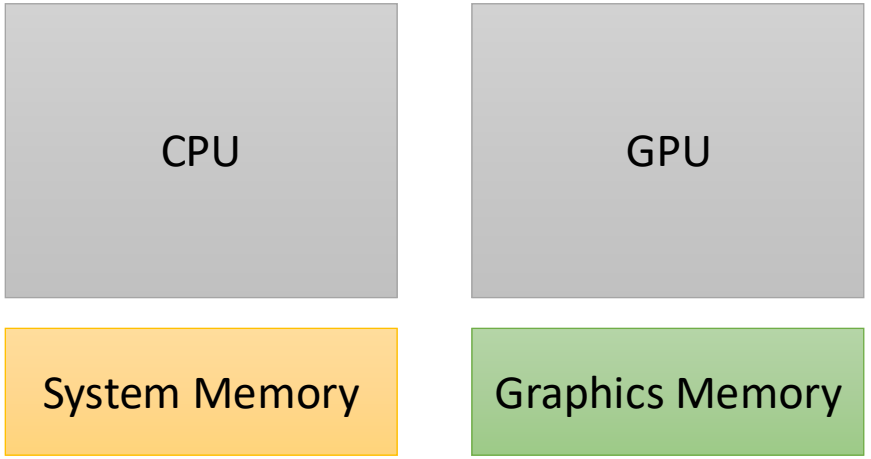
chart: docs.google.com/forms/d/e/1FAIpQLSfafsP0Ovn_31uhRJgjO4RqzZL0ir9-cI74cTtjh88RP87VAQ/viewanalytics

GPU Memory



GPU Memory

CPU Memory:
Fast: Low Latency
Easily saturated: Low Bandwidth
Scales well: up to 1 TB
DDR



GPU Memory:
Slow: High Latency
Hard to saturate: High Bandwidth
Doesn't scale: 80-192 GB
GDDR, HBM

2-lane straight highway driven on by sports cars

Different technologies

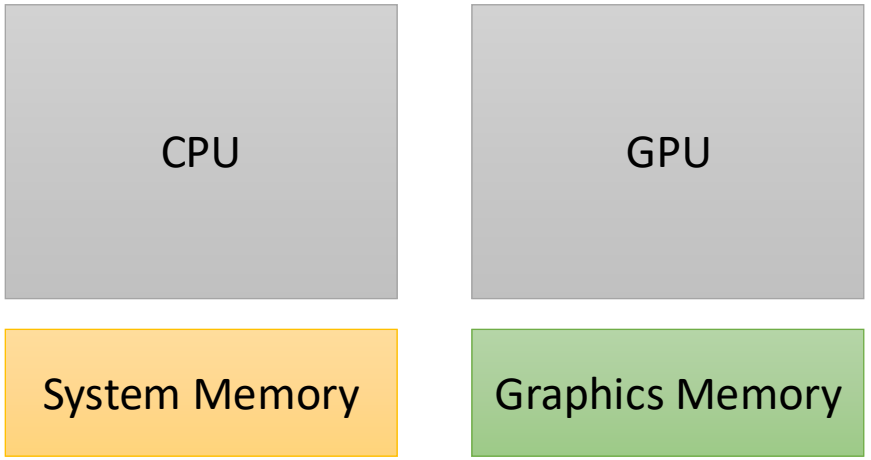
16-lane highway on a windy road driven by semi trucks

GPU Memory

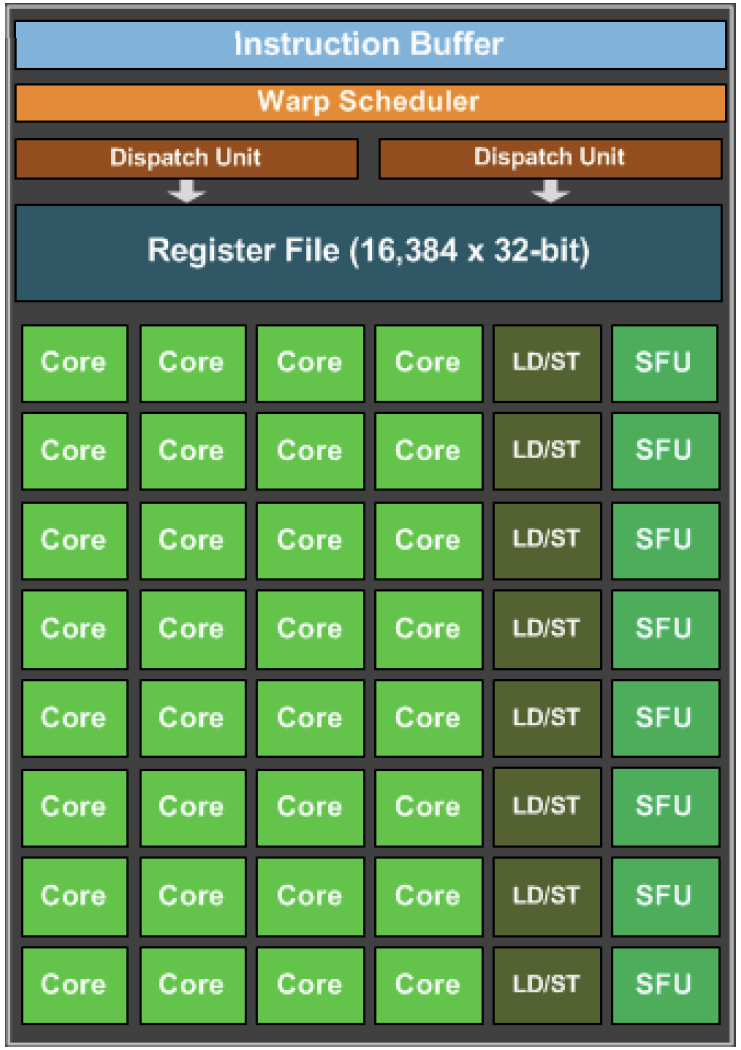
bandwidth:
~**1-3.3 TB/s** for GPU
~**300 GB/s** for CPUs

memory Latency:
~**600** cycles for GPU memory
~**200** cycles for CPU memory

Cache Latency:
~**28** cycles for L1 hit for GPU
~**4** cycles for L1 hit on CPUs



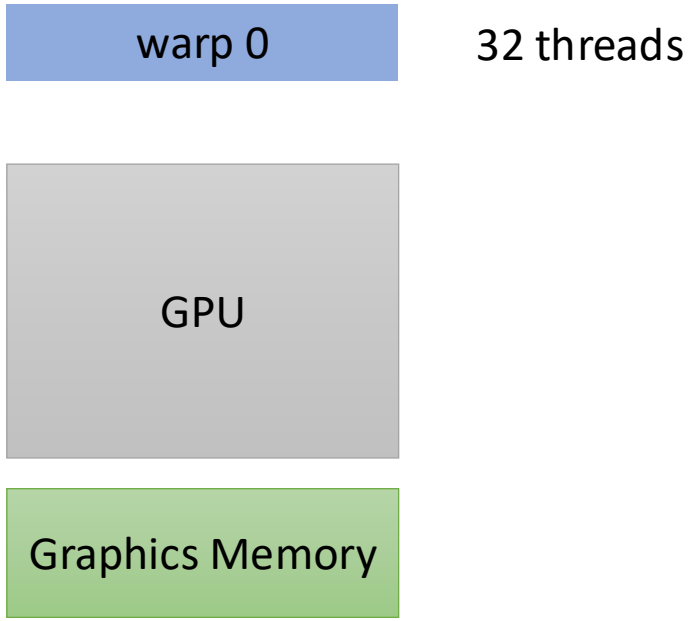
Warp scheduling and concurrency?



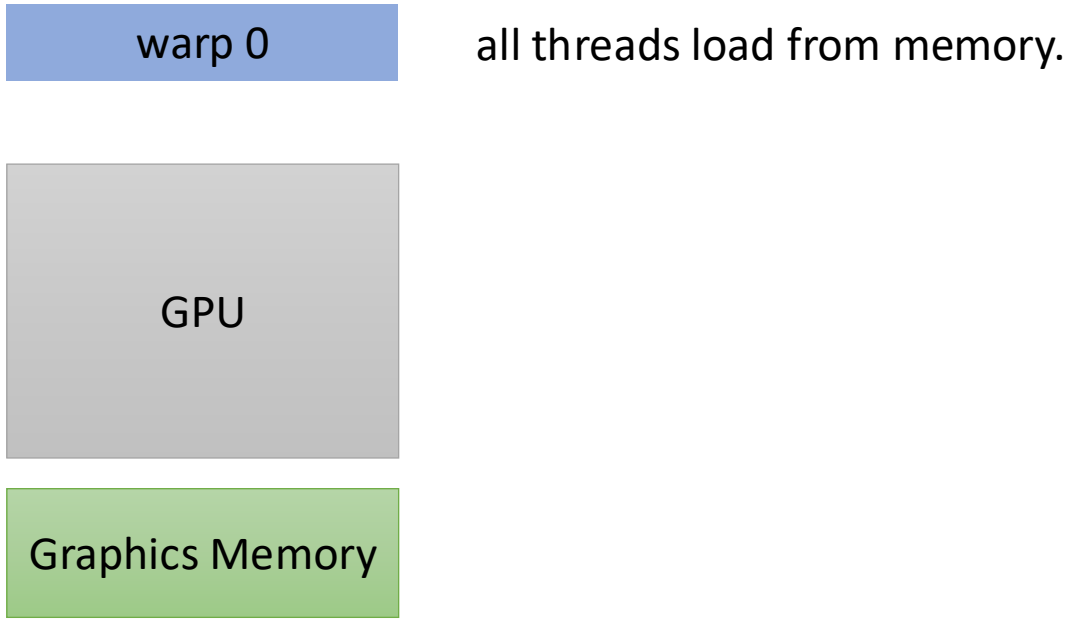
Blocks: multiple warms
Bound on number of warps: 32

Block = 32 warps * 32 blocks = 1024 threads max

Warp scheduling and concurrency?

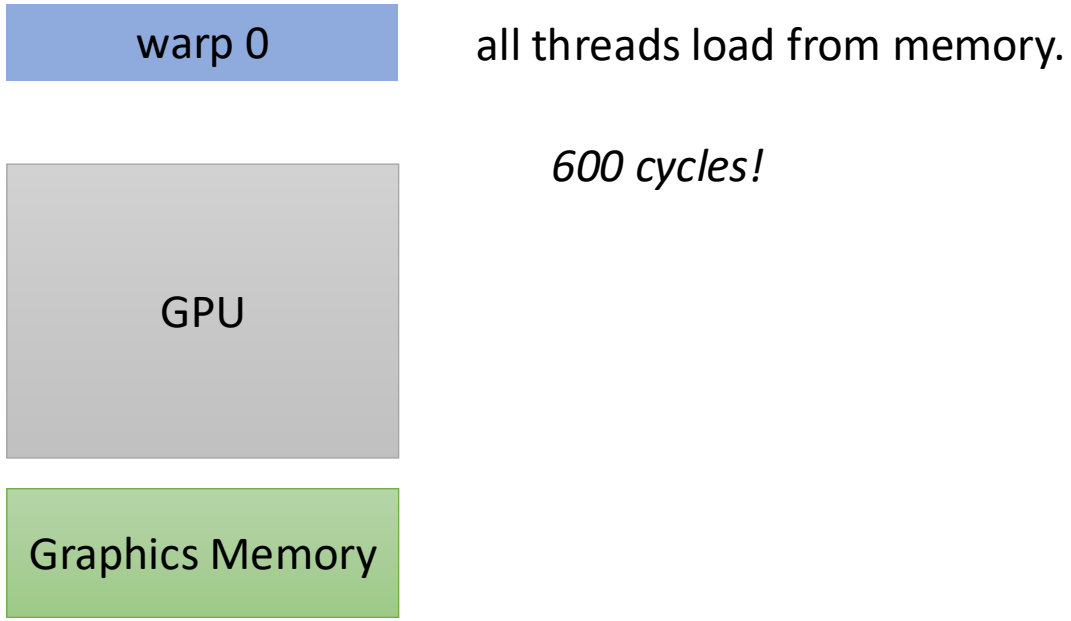


Warp scheduling and concurrency?



all threads load from memory.

Warp scheduling and concurrency?



all threads load from memory.

600 cycles!

Warp scheduling and concurrency?

warp 0

warp 1

warp 2

We can hide latency through scheduling and concurrency!

GPU

Graphics Memory



Warp scheduling and concurrency?

warp 1

warp 2

warp 0

GPU

Graphics Memory



We can hide latency through scheduling and concurrency!

Warp scheduling and concurrency?

memory access
600 cycles

warp 1

warp 2

warp 0

GPU

Graphics Memory



We can hide latency through scheduling and concurrency!

Warp scheduling and concurrency?

memory access
600 cycles

warp 1

warp 2

warp 0

GPU

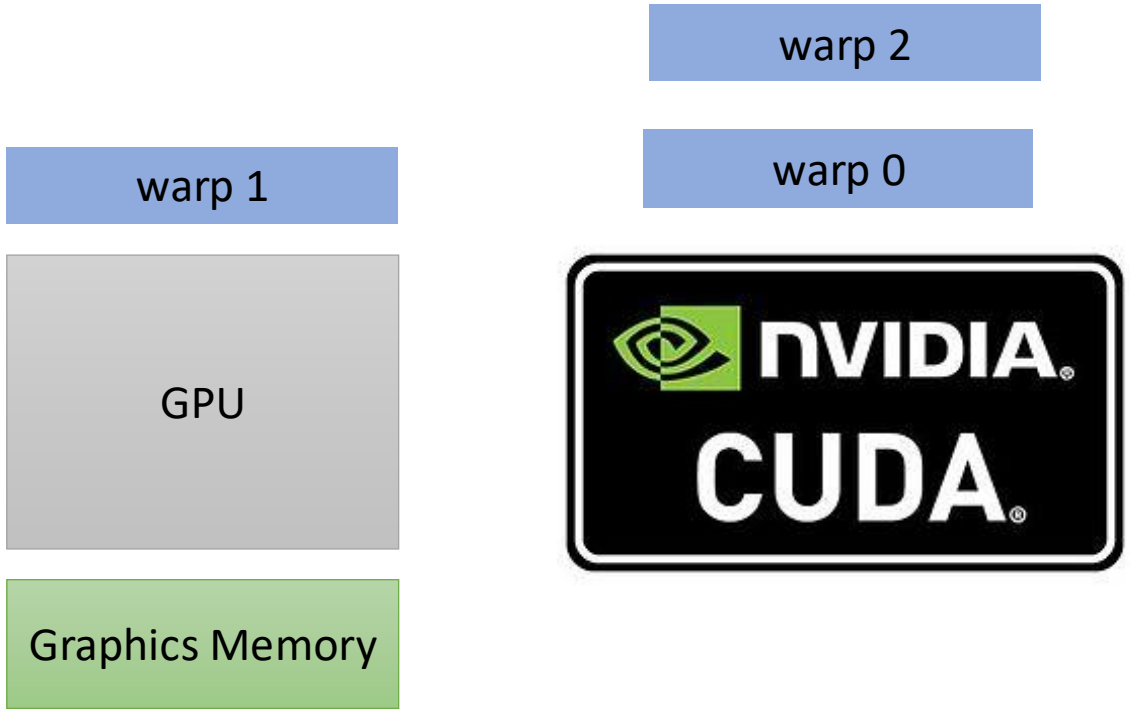
Graphics Memory



preempt warp 0
and put warp 1 on

We can hide latency through scheduling and concurrency!

Warp scheduling and concurrency?



We can hide latency through scheduling and concurrency!

Warp scheduling and concurrency?

memory access
600 cycles

warp 1

GPU

Graphics Memory

warp 2

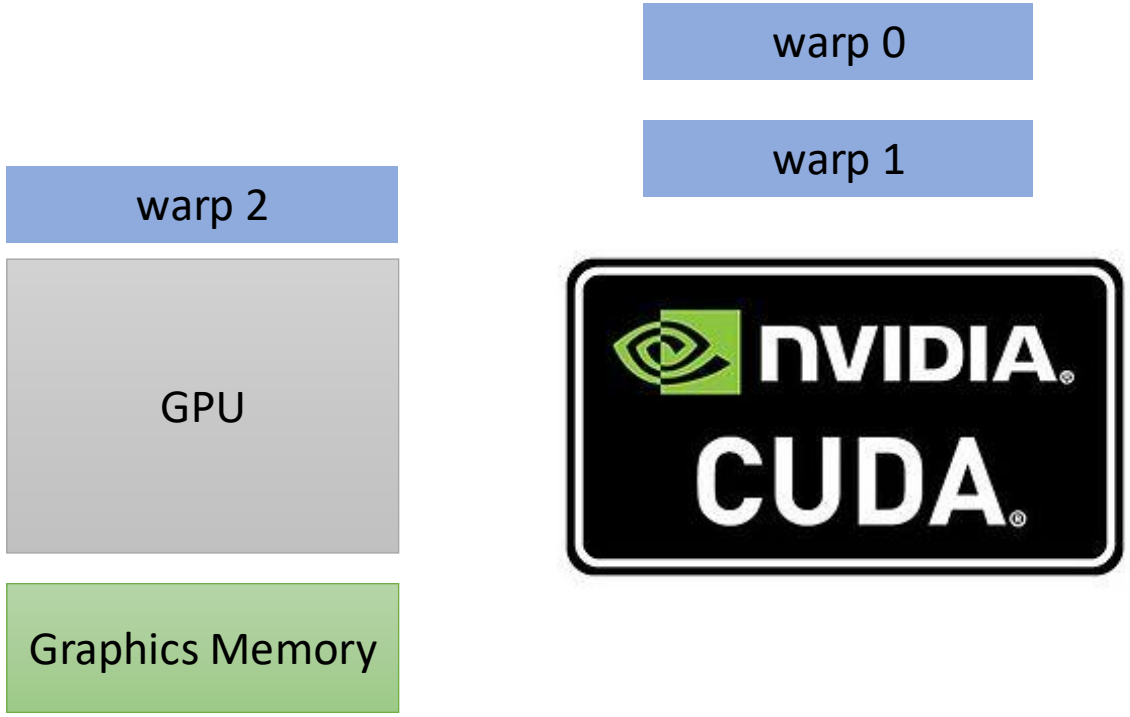
warp 0



preempt warp 1
and put warp 2 on

We can hide latency through scheduling and concurrency!

Warp scheduling and concurrency?



We can hide latency through scheduling and concurrency!

Warp scheduling and concurrency?

memory access
600 cycles

warp 2

GPU

Graphics Memory

warp 0

warp 1



preempt warp 2
and put warp 0 on

We can hide latency through scheduling and concurrency!

Warp scheduling and concurrency?

Hey, my memory has arrived!

warp 0

GPU

Graphics Memory

warp 1

warp 2



preempt warp 2
and put warp 0 on

We can hide latency through scheduling and concurrency!

Quiz 12.

What is a warp?

Select one

A. Group of 32 threads in a GPU

B. Group of 16 threads in a GPU

C. Group of 2 threads in a GPU

D. Group of some threads in a GPU

Scan to answer

forms.gle/vwqLu4z8L7qVbBdd9

chart: docs.google.com/forms/d/e/1FAIpQLSdtiYdVsGaa_pIzyv0wv39CZD-oXhojmrd3oNDMY8qagatO0g/viewanalytics

Quiz 12.

What is a warp?

Select one

A. Group of 32 threads in a GPU

B. Group of 16 threads in a GPU

C. Group of 2 threads in a GPU

D. Group of some threads in a GPU

Nvidia uses 32, AMD uses 64 or 32, Intel varies, and WebGPU calls the same thing a subgroup whose size you query at runtime.

Scan to answer

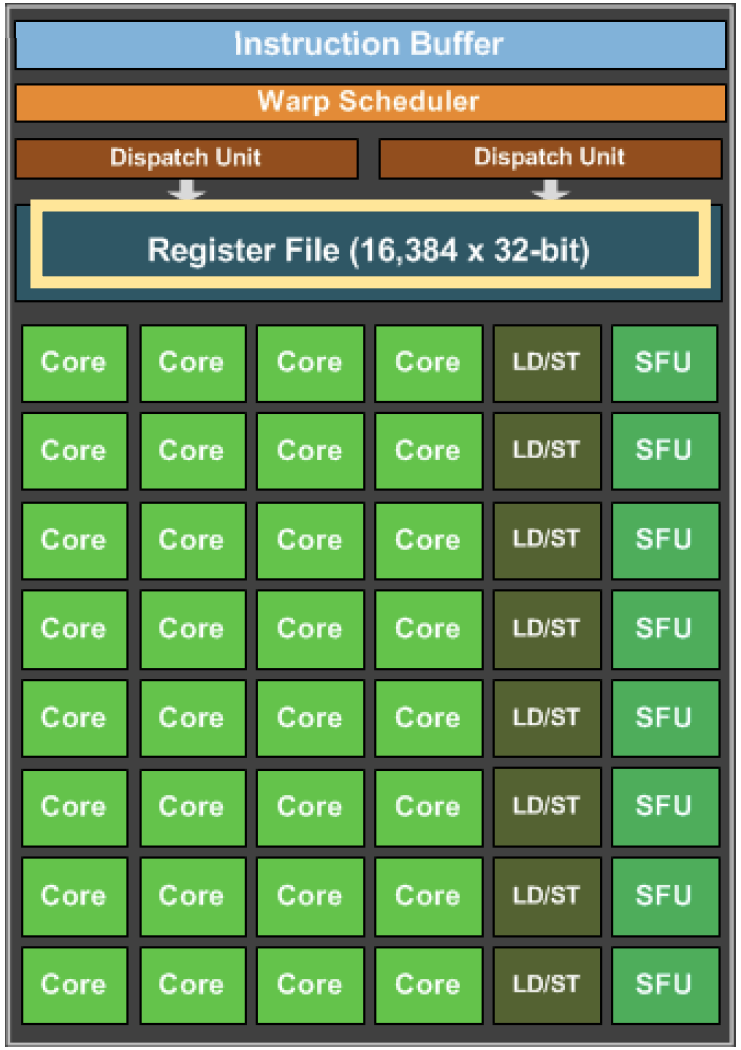
forms.gle/vwqLu4z8L7qVbBdd9

chart: docs.google.com/forms/d/e/1FAIpQLSdtiYdVsGaa_pIzyv0wv39CZD-oXhojmrd3oNDMY8qagatO0g/viewanalytics

Warp scheduling and concurrency?

But wait, I thought rescheduling was expensive?

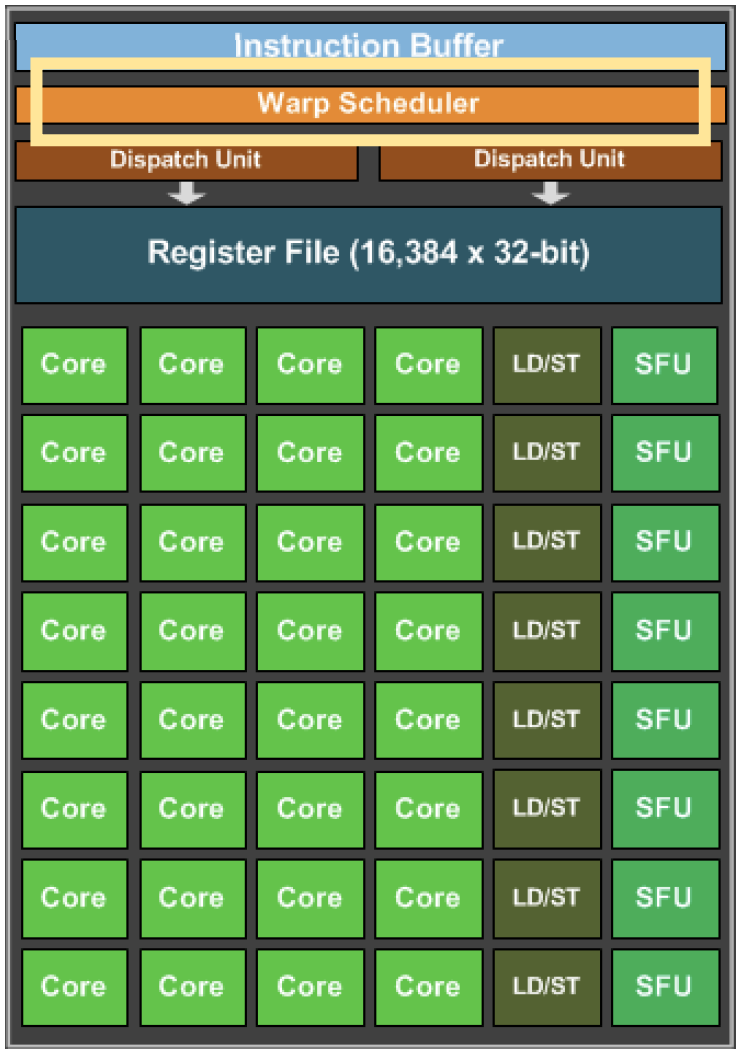
Warp scheduling and concurrency?



But wait, I thought rescheduling was expensive?

Registers all stay on chip. Large register file.

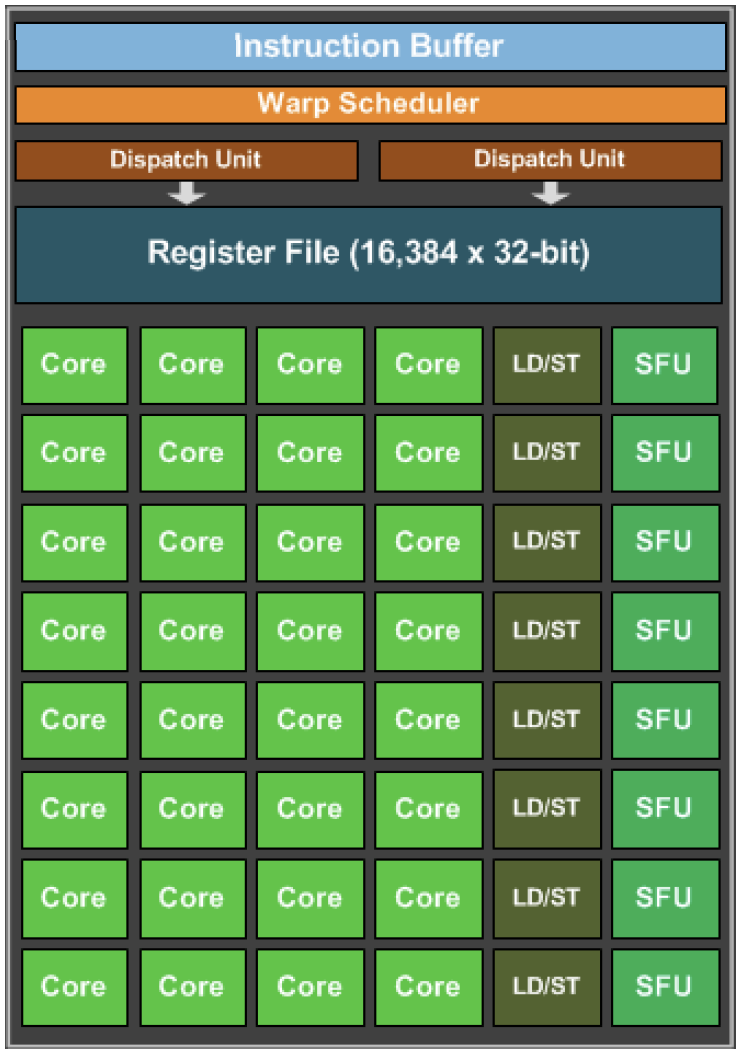
Warp scheduling and concurrency?



But wait, I thought rescheduling was expensive?

dedicated scheduler logic

Warp scheduling and concurrency?



But wait, I thought rescheduling was expensive?
Hardware multithreading, not OS-style preemption.

Go back to our program

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    int chunk_size = size/blockDim.x;
    int start = chunk_size * threadIdx.x;
    int end = start + chunk_size;
    for (int i = start; i < end; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

calling the function

Lets launch with 32 warps

```
vector_add<<<1,32>>>(d_a, d_b, d_c, size);
```

Go back to our program

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    int chunk_size = size/blockDim.x;
    int start = chunk_size * threadIdx.x;
    int end = start + chunk_size;
    for (int i = start; i < end; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

calling the function

Lets launch with 32 warps

```
vector_add<<<1,1024>>>(d_a, d_b, d_c, size);
```

Concurrent warps

This is about 2x faster.

Concurrent warps

This is about 2x faster.

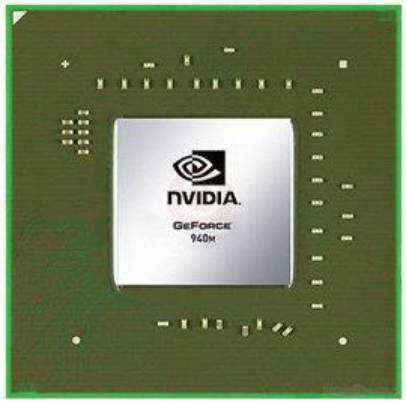


Getting better!

Recalling where we left off

Programming a GPU

Tiny GPU in an embedded system



Nvidia Jetson Nano (whole chip, CPU + GPU)
2 Billion transistors
10 TDP
Est. $99

Fight!



The CPU in my research workstation



Intel i7-9700K
2.16 Billion transistors
95 TDP
Est. $316

https://www.techpowerup.com/gpu-specs/geforce-940m.c2648
https://www.alibaba.com/product-detail/Intel-Core-i7-9700K-8-Cores_62512430487.html
https://www.prolast.com/prolast-elevated-boxing-rings-22-x-22/

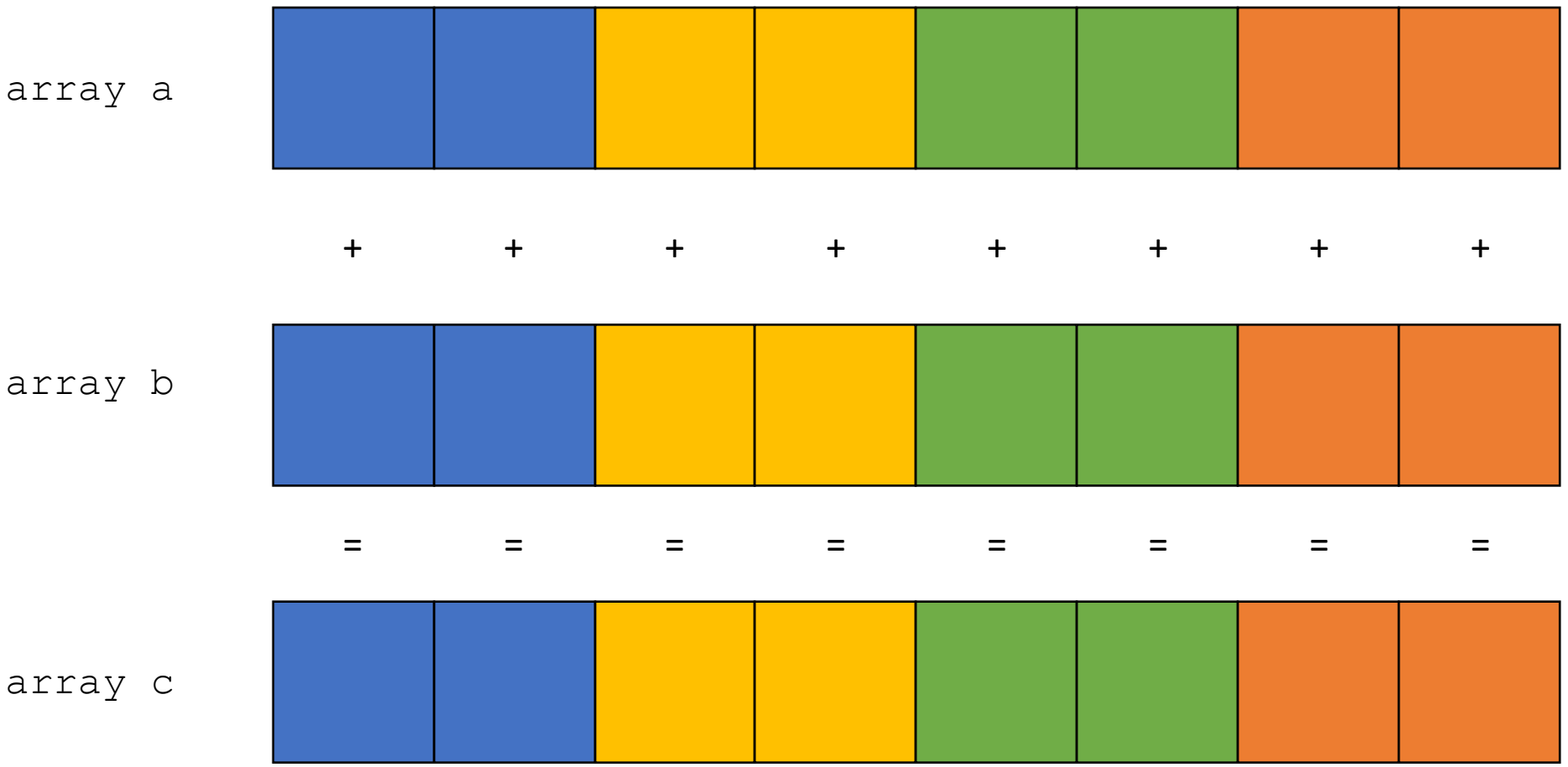
Programming a GPU

- The problem: Vector addition

Parallel Schedules (Chunked)

Computation can easily be divided into threads

Thread 0 - Blue
Thread 1 - Yellow
Thread 2 - Green
Thread 3 - Orange



Lets set up the CPU

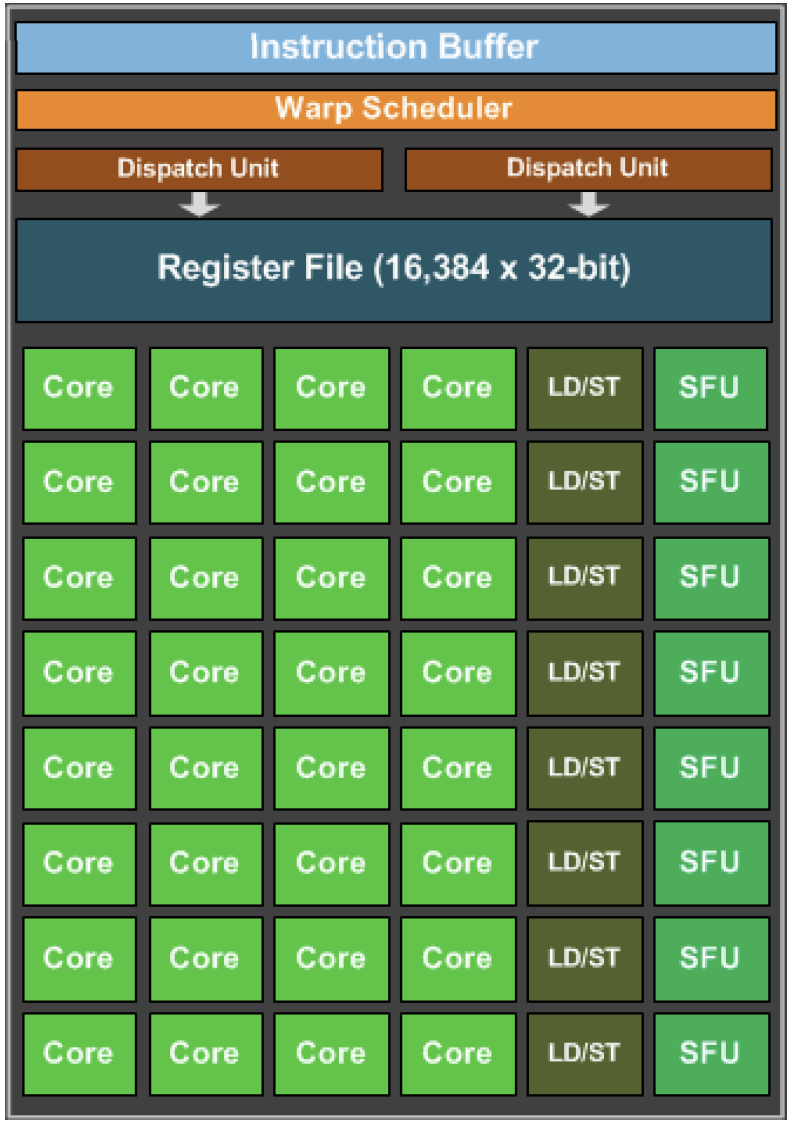
- CPU code

First GPU attempt

- Basic sequential GPU program
  - Issues?
  - Key insights?

woah, 32 cores!

We should parallelize our application!



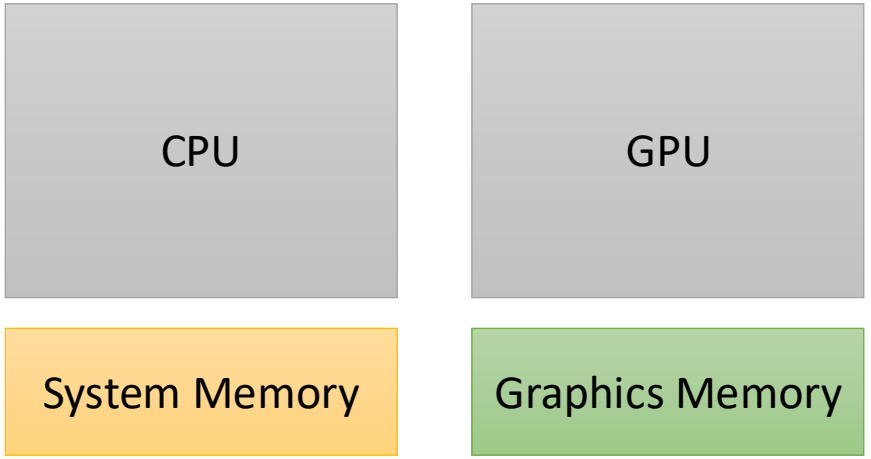
https://www.techpowerup.com/gpu-specs/docs/nvidia-gtx-980.pdf

Second GPU attempt

- Use 32 cores
  - Issues?
  - Key insights?

GPU Memory

CPU Memory:
Fast: Low Latency
Easily saturated: Low Bandwidth
Scales well: up to 1 TB
DDR



GPU Memory:
slow: High Latency
hard to saturate: High Bandwidth
doesn't scale: 32 GB
GDDR, HBM

2-lane straight highway driven on by sports cars

Different technologies

16-lane highway on a windy road driven by semi trucks

Warp scheduling and concurrency?

memory access
600 cycles

warp 1

GPU

Graphics Memory

warp 2

warp 0



preempt warp 1
and put warp 2 on

We can hide latency through scheduling and concurrency!

Go back to our program

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    int chunk_size = size/blockDim.x;
    int start = chunk_size * threadIdx.x;
    int end = start + chunk_size;
    for (int i = start; i < end; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

calling the function

```
vector_add<<<1,1024>>>(d_a, d_b, d_c, size);
```

Let's launch with 32 warps
32 warps * 32 threads = 1024 threads

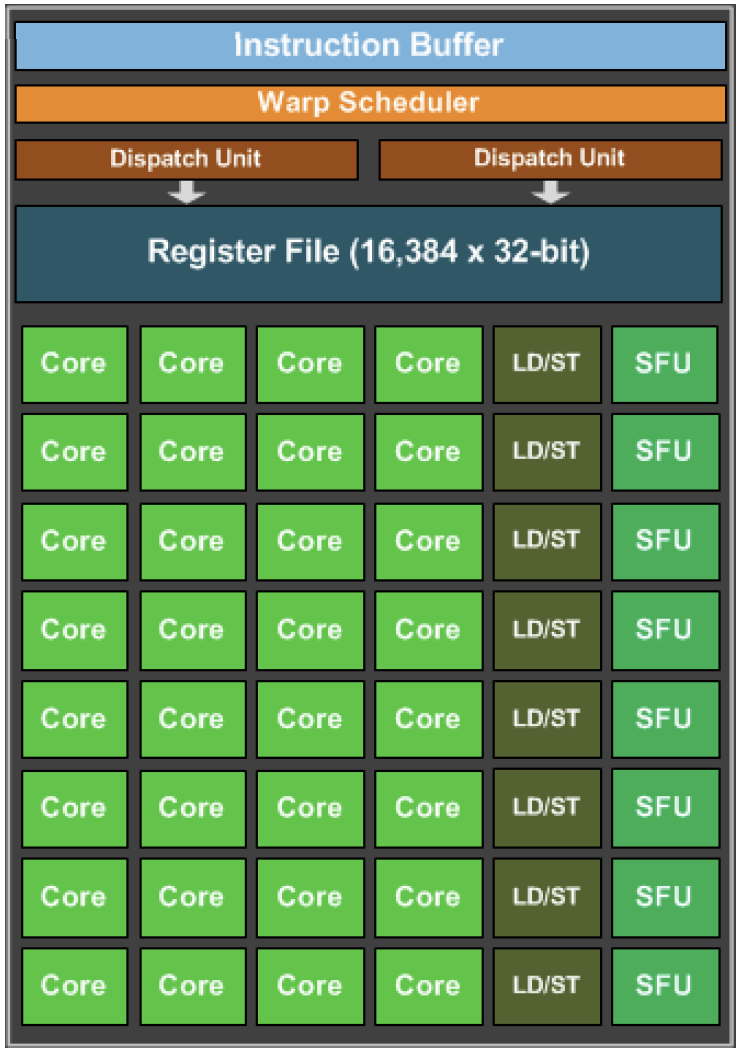
Concurrent warps

This is about 2x faster.

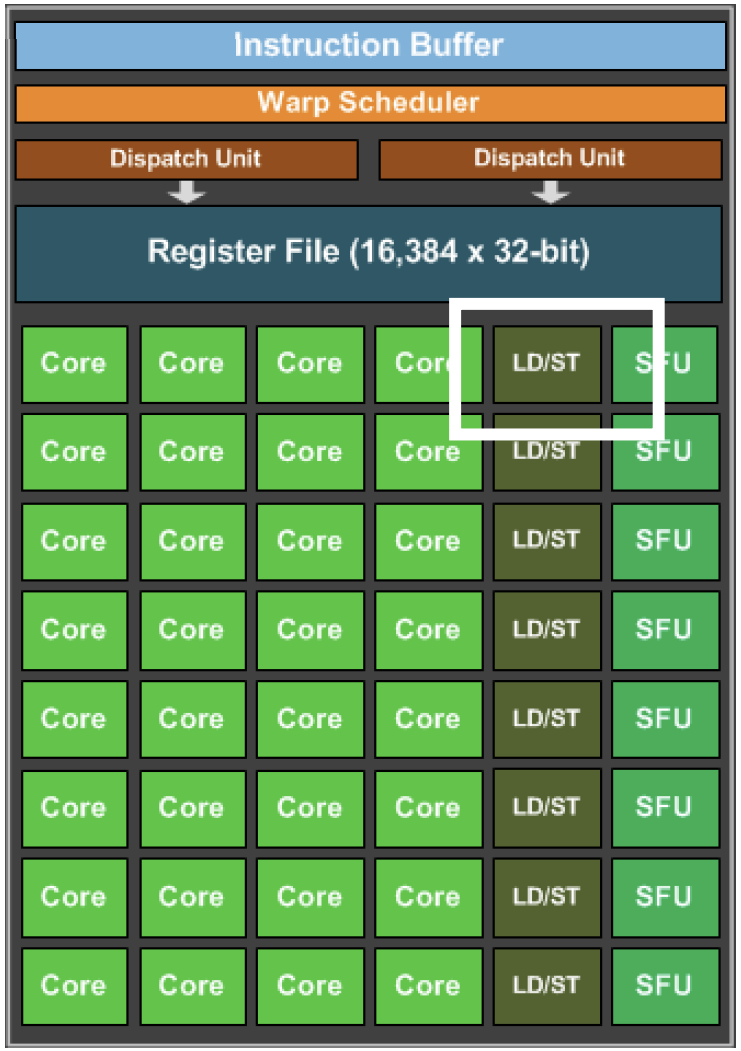


Getting better!

Optimizing memory accesses

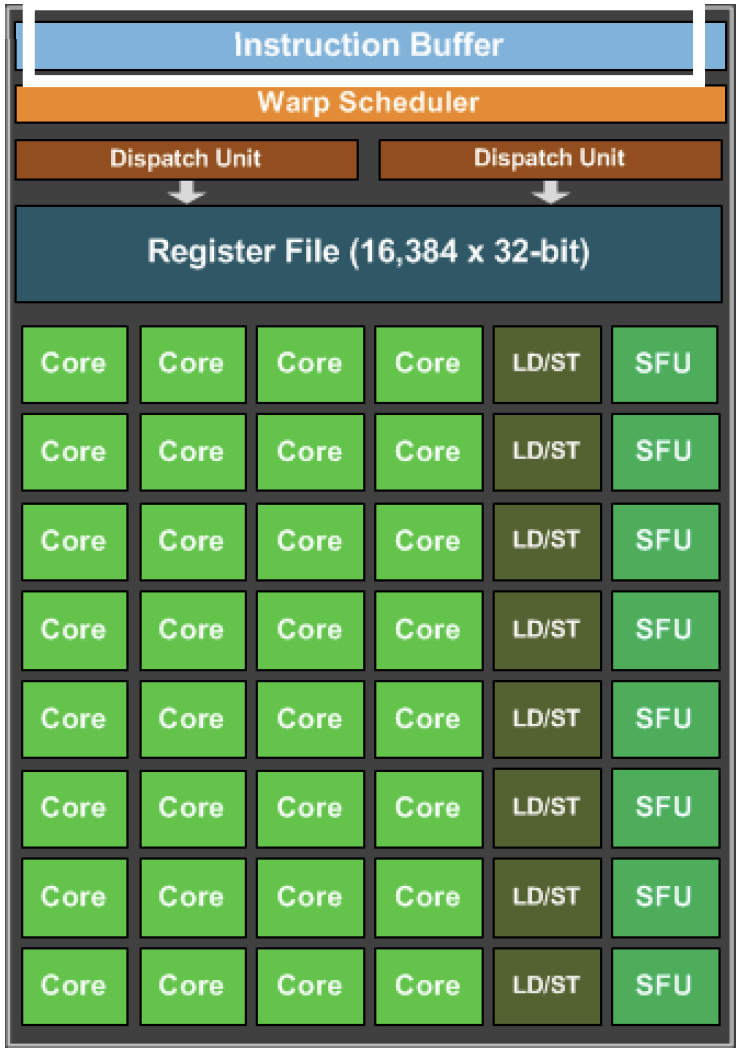


Optimizing memory accesses



This is the load/store unit. The hardware component responsible for issuing loads and stores.
Shared between 4 cores.

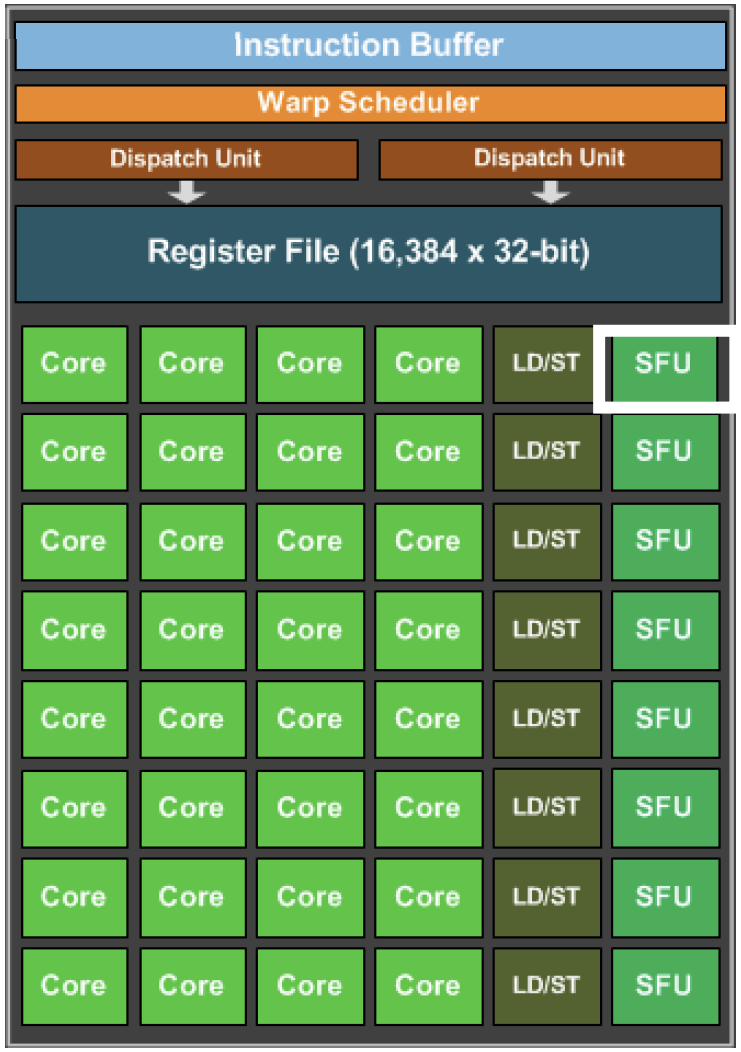
Optimizing memory accesses



This is the instruction cache
Shared between all cores of the warp.

This is the load/store unit. The hardware component responsible for issuing loads and stores.
Shared between 4 cores.

Optimizing memory accesses

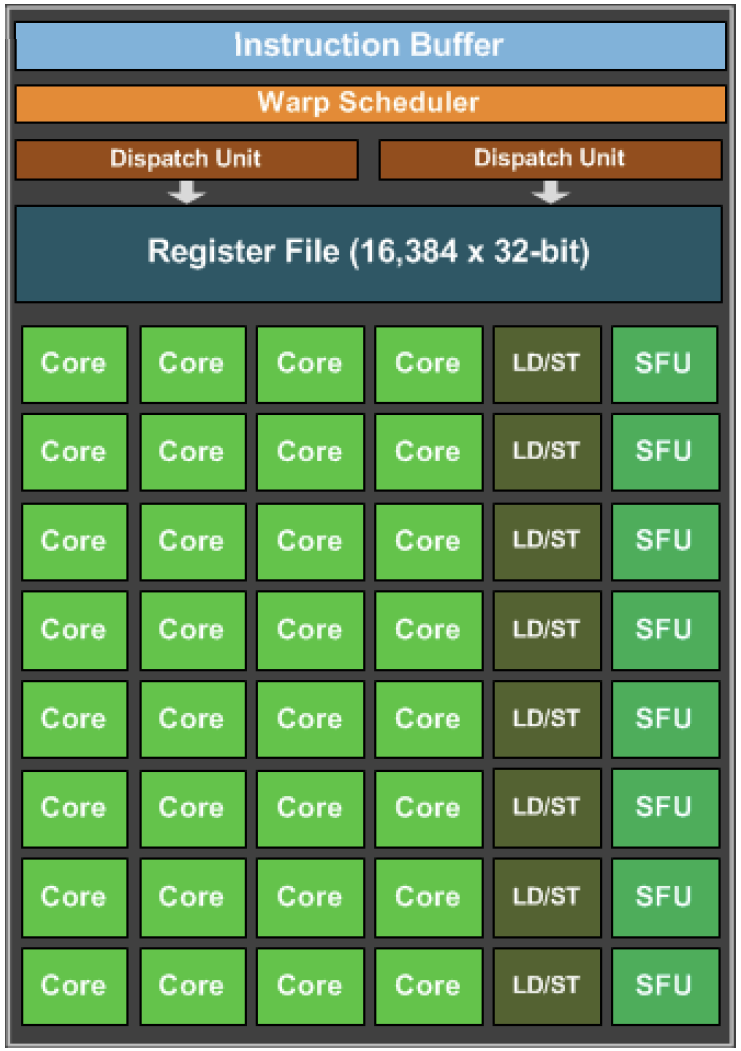


This is the instruction cache
Shared between all cores of the warp.
A dispatch unit issues instructions to a group of cores,
load/store units, or special function units (SFUs).

This is the load/store unit. The hardware component responsible for issuing loads and stores.
Shared between 4 cores.

SFU (The Special Function Unit) is an accelerator to perform tracendental and trigonometric operations such as sine and cosine.
These operations are not executed in parallel; an SFU is shared for those.

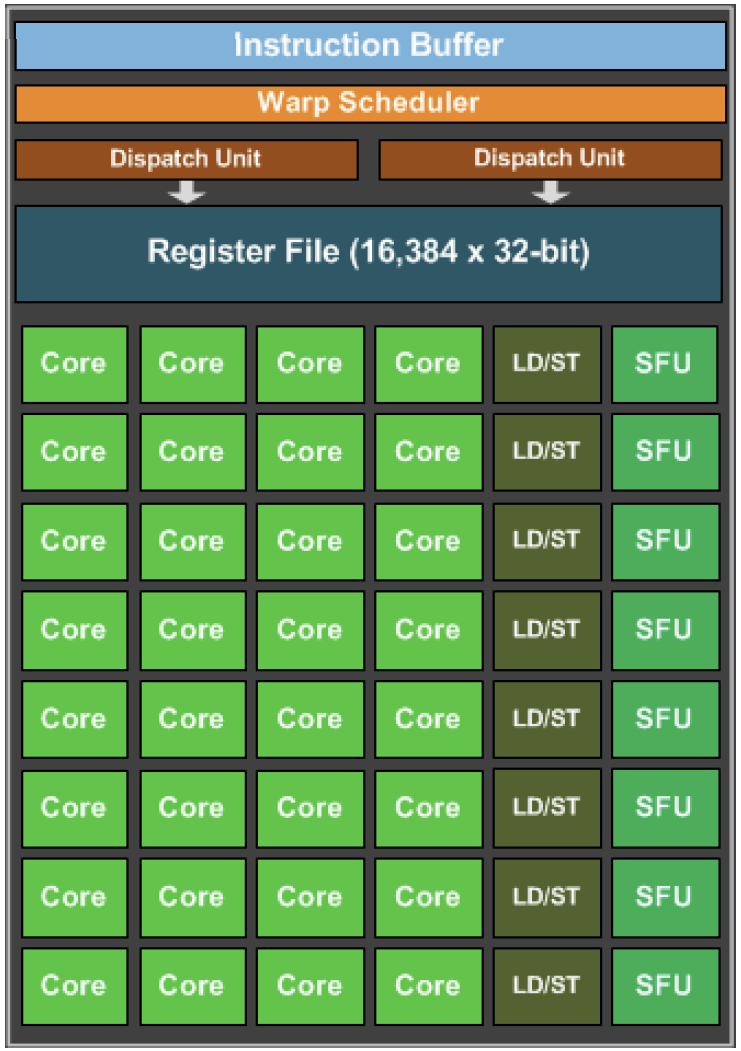
Warp execution



Groups of 32 threads are called a “warp”

They are executed in lock-step, i.e. they all execute the same instruction at the same time

Warp execution



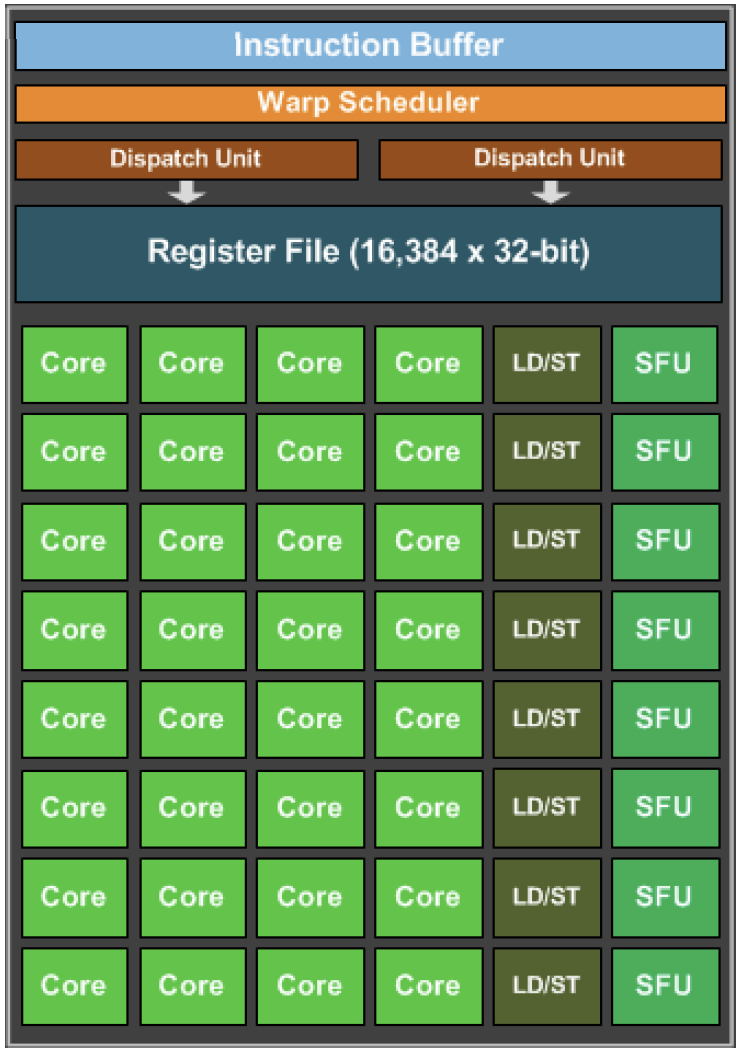
Groups of 32 threads are called a “warp”

They are executed in lock-step, i.e. they all execute the same instruction at the same time

Program:

```
int variable1 = b[0];
int variable2 = c[0];
int variable3 = variable1 + variable2;
a[0] = variable3;
```

Warp execution



Groups of 32 threads are called a “warp”

They are executed in lock-step, i.e. they all execute the same instruction at the same time

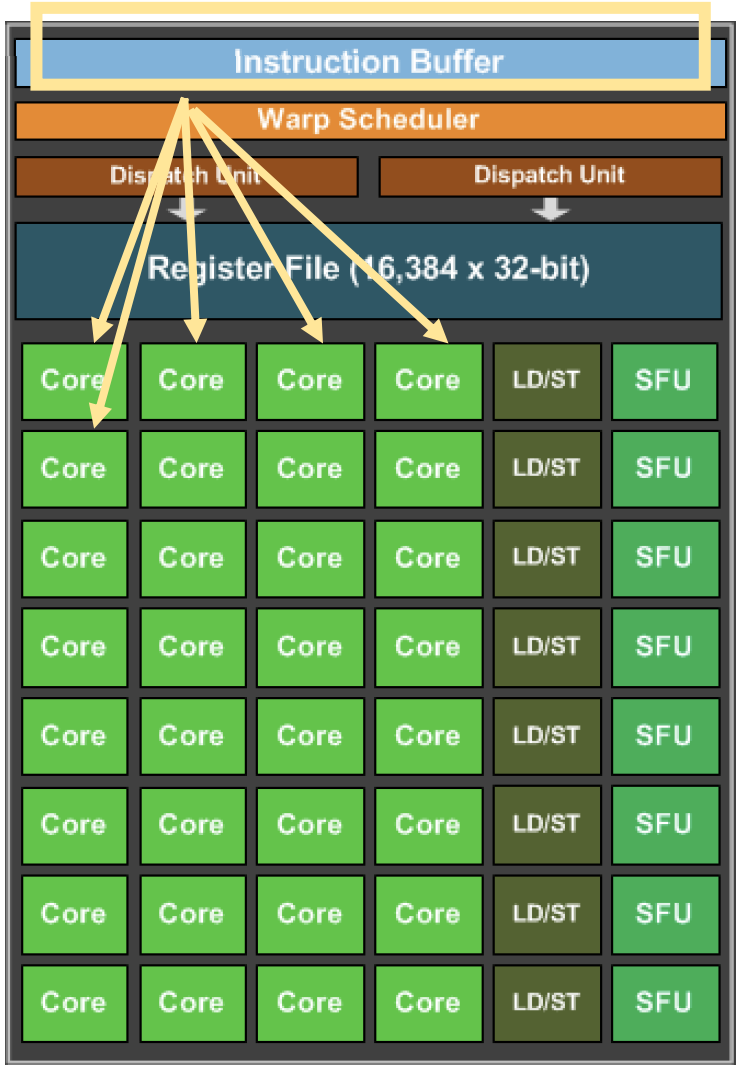
Program:

```
int variable1 = b[0];
int variable2 = c[0];
int variable3 = variable1 + variable2;
a[0] = variable3;
```

Warp execution

Groups of 32 threads are called a “warp”

They are executed in lock-step, i.e. they all execute the same instruction at the same time



instruction is fetched from the buffer and distributed to all the cores.

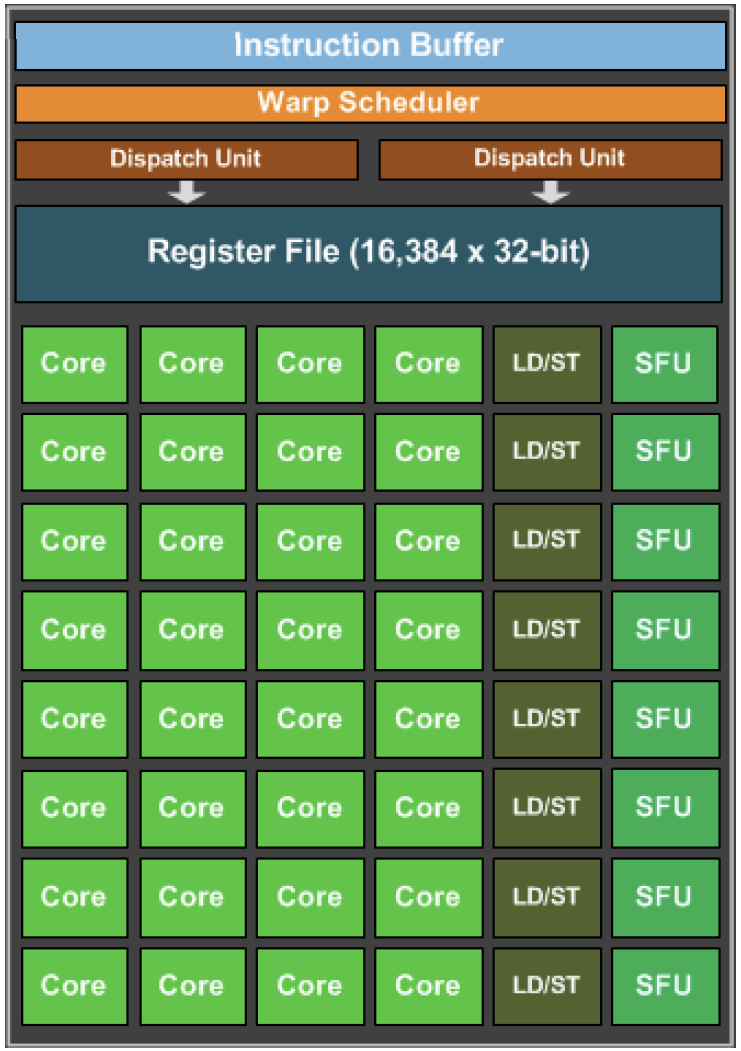
Program:

```
int variable1 = b[0];
int variable2 = c[0];
int variable3 = variable1 + variable2;
a[0] = variable3;
```

Warp execution

Groups of 32 threads are called a “warp”

They are executed in lock-step, i.e. they all execute the same instruction at the same time



All cores need to wait until all cores finish the first instruction

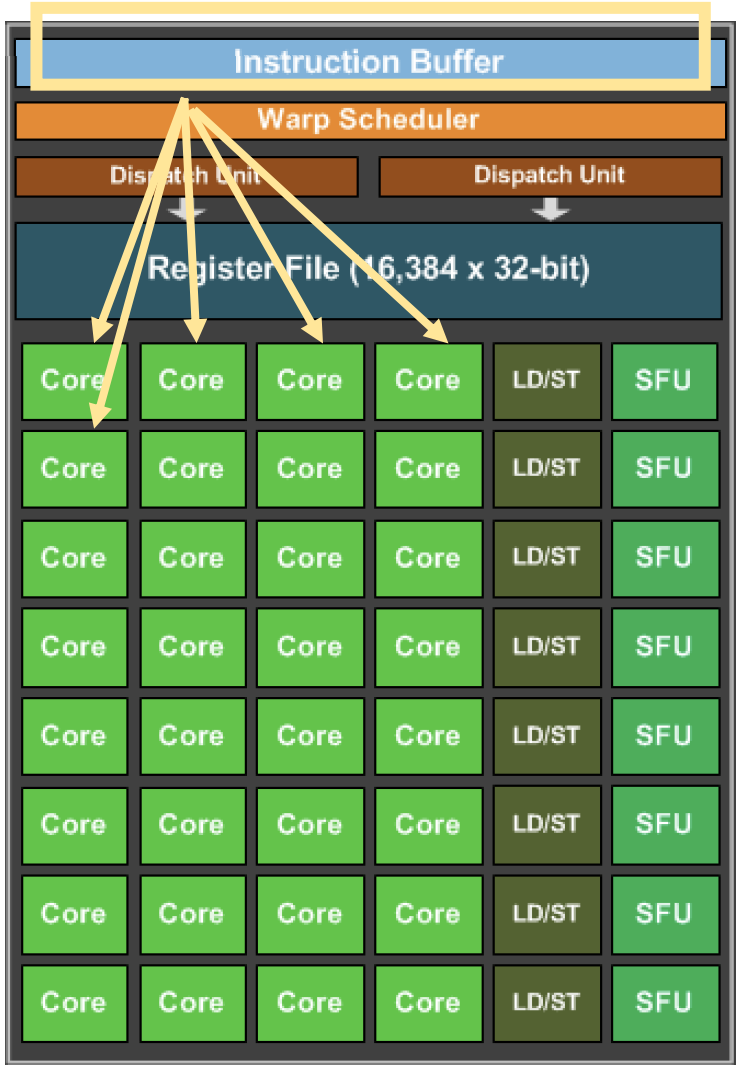
Program:

```
int variable1 = b[0];
int variable2 = c[0];
int variable3 = variable1 + variable2;
a[0] = variable3;
```

Warp execution

Groups of 32 threads are called a “warp”

They are executed in lock-step, i.e. they all execute the same instruction at the same time



Start the next instruction.

Program:

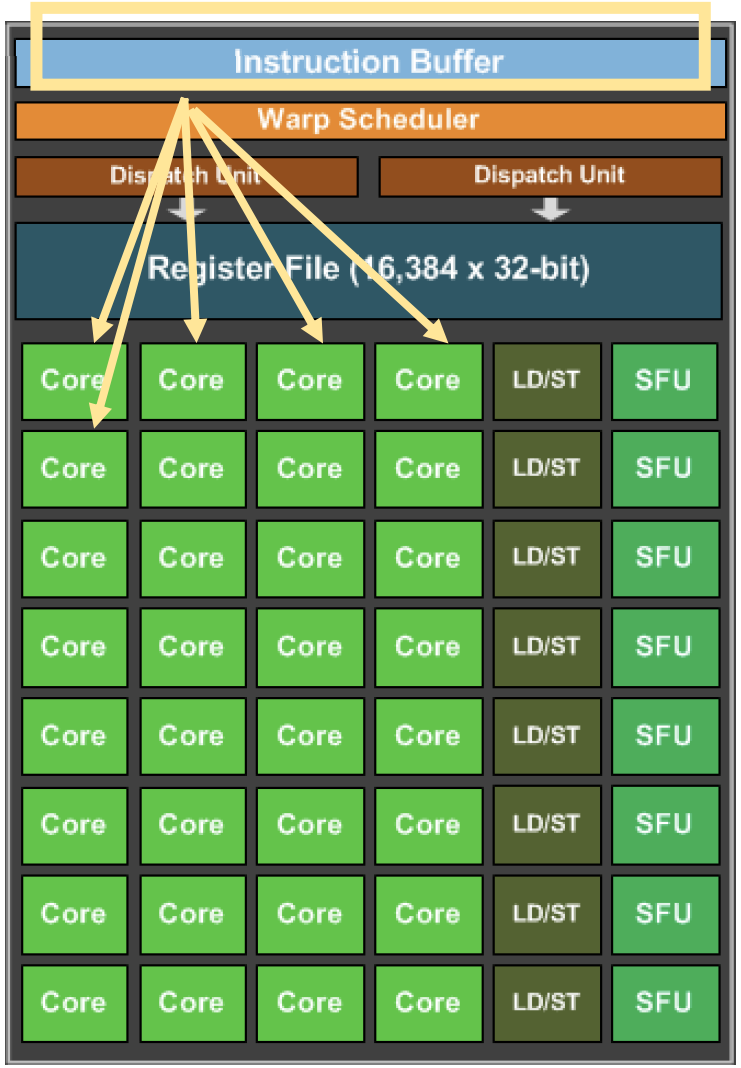
```
int variable1 = b[0];
int variable2 = c[0];
int variable3 = variable1 + variable2;
a[0] = variable3;
```

Why would we have a programming model like this?

Warp execution

Groups of 32 threads are called a “warp”

They are executed in lock-step, i.e. they all execute the same instruction at the same time



Start the next instruction.

Program:

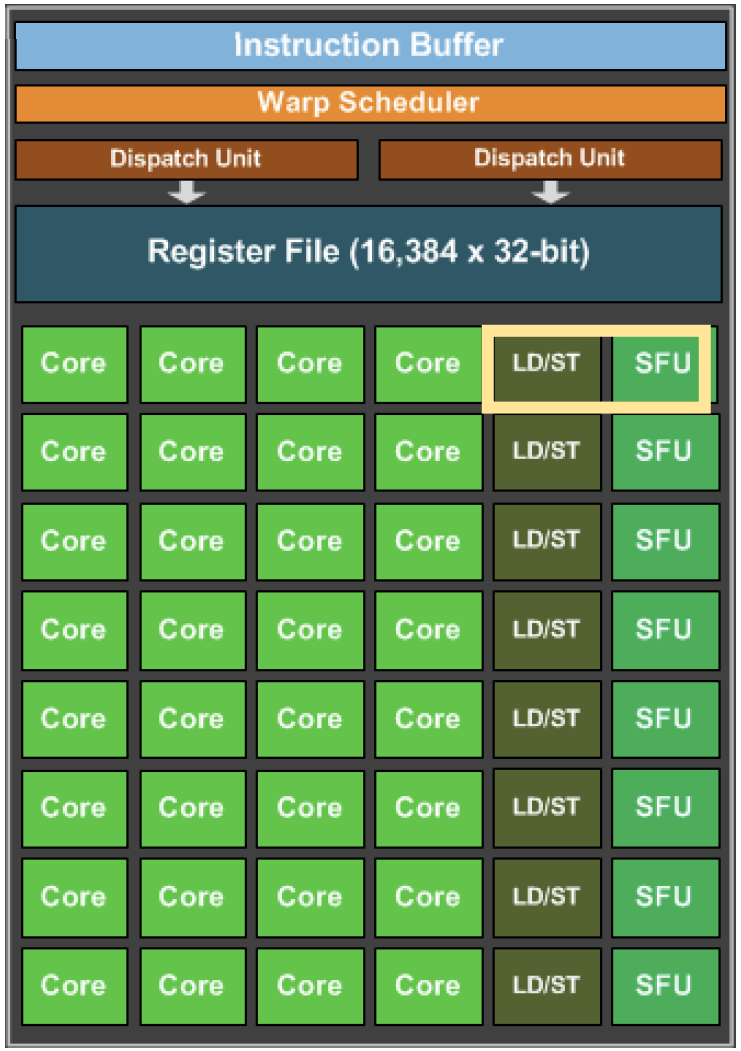
```
int variable1 = b[0];
int variable2 = c[0];
int variable3 = variable1 + variable2;
a[0] = variable3;
```

Why would we have a programming model like this?
More cores (share program counters)
Can be efficient to share hardware resources

Warp execution

Groups of 32 threads are called a “warp”

They are executed in lock-step, i.e. they all execute the same instruction at the same time



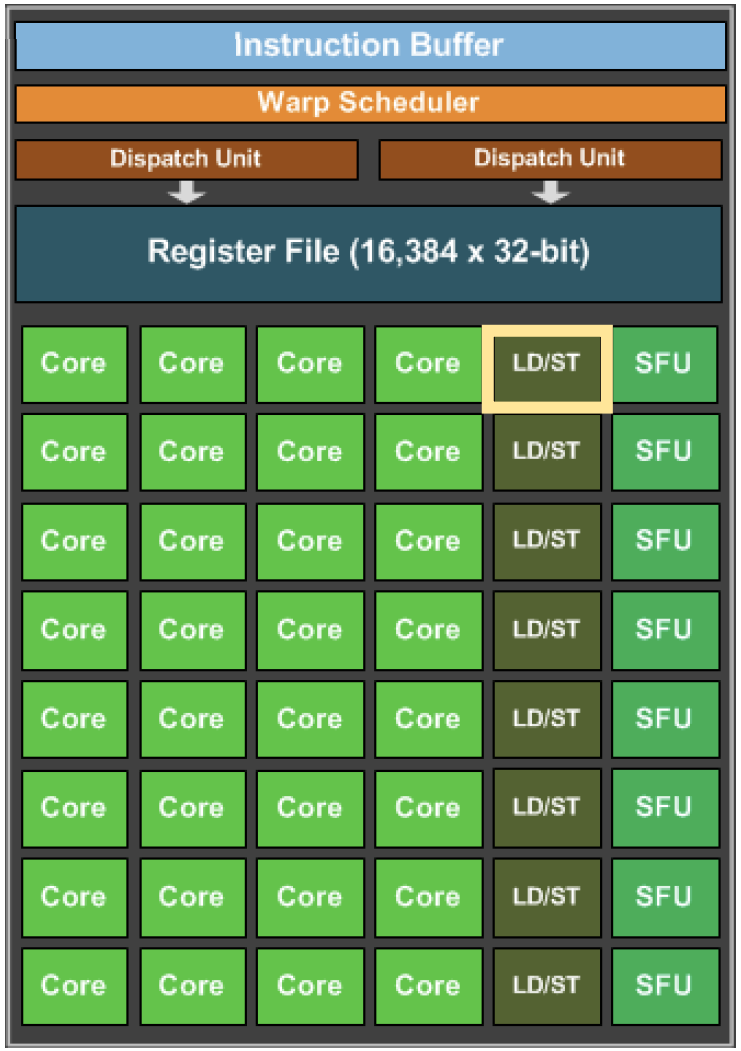
Cores share expensive HW units (load/store and special functions)

Program:

```
int variable1 = b[0];
int variable2 = c[0];
int variable3 = variable1 + variable2;
a[0] = variable3;
```

Warp execution

Lets look closer at memory

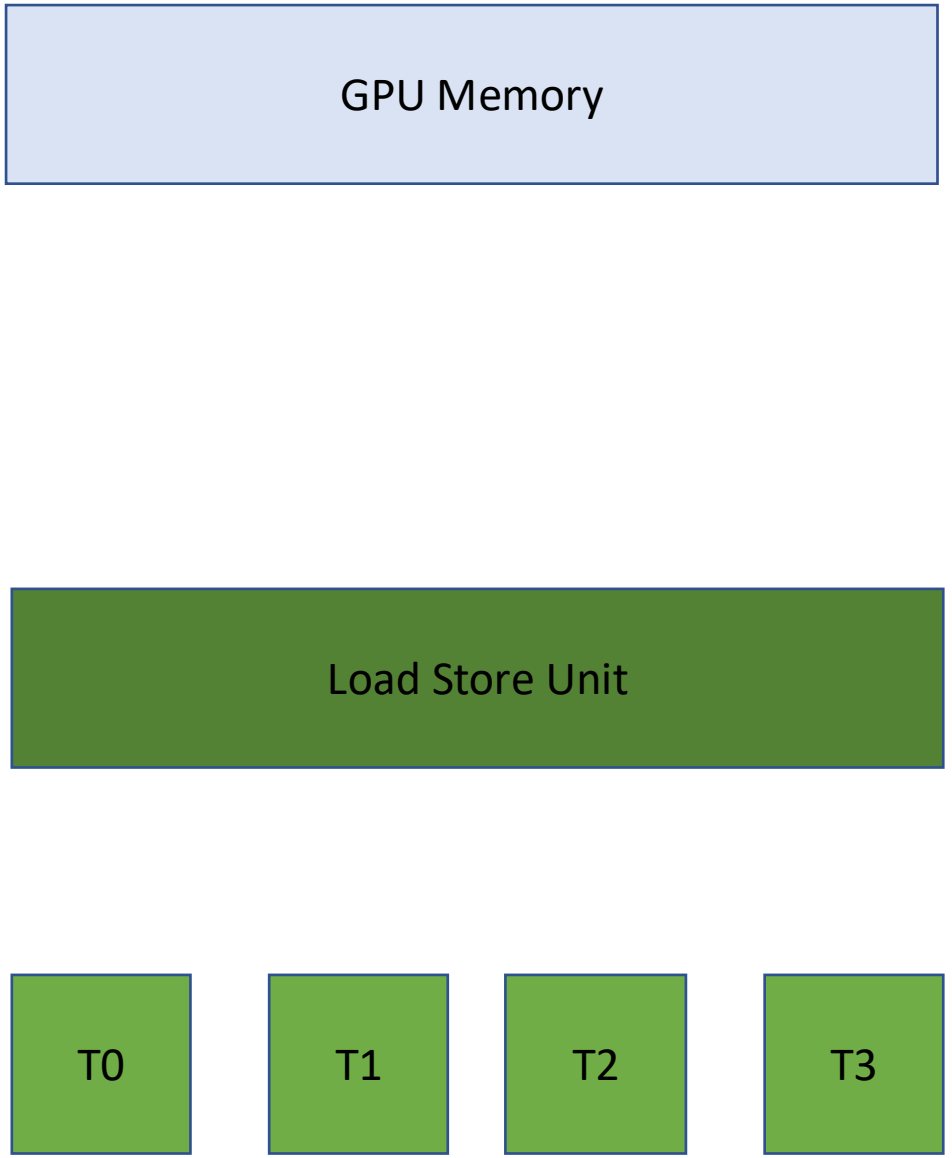


Program:

```
int variable1 = b[0];
int variable2 = c[0];
int variable3 = variable1 + variable2;
a[0] = variable3;
```

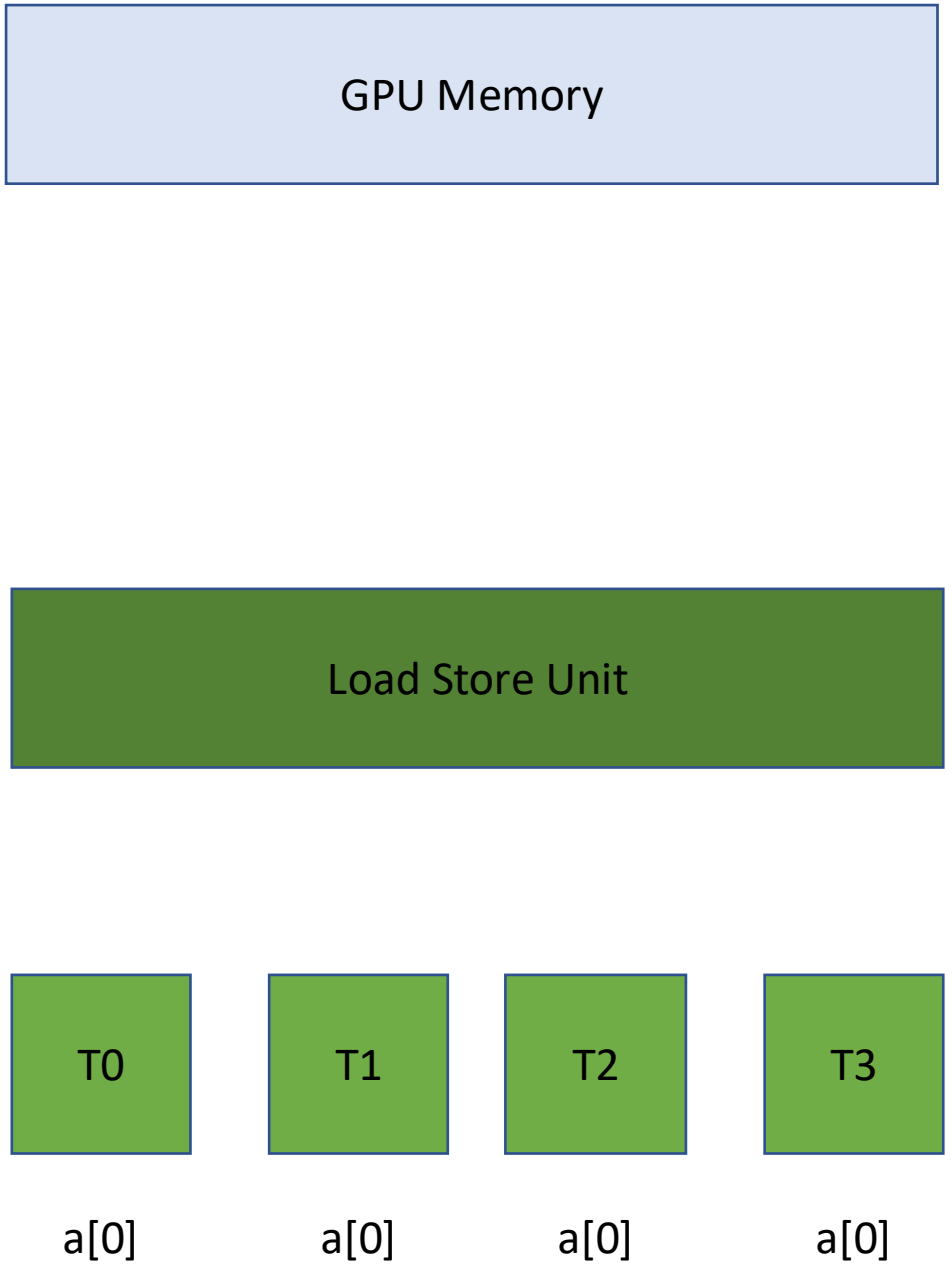
4 cores are accessing memory. what happens if they access the same value?

4 cores are accessing memory. What can happen



4 cores are accessing memory. What can happen

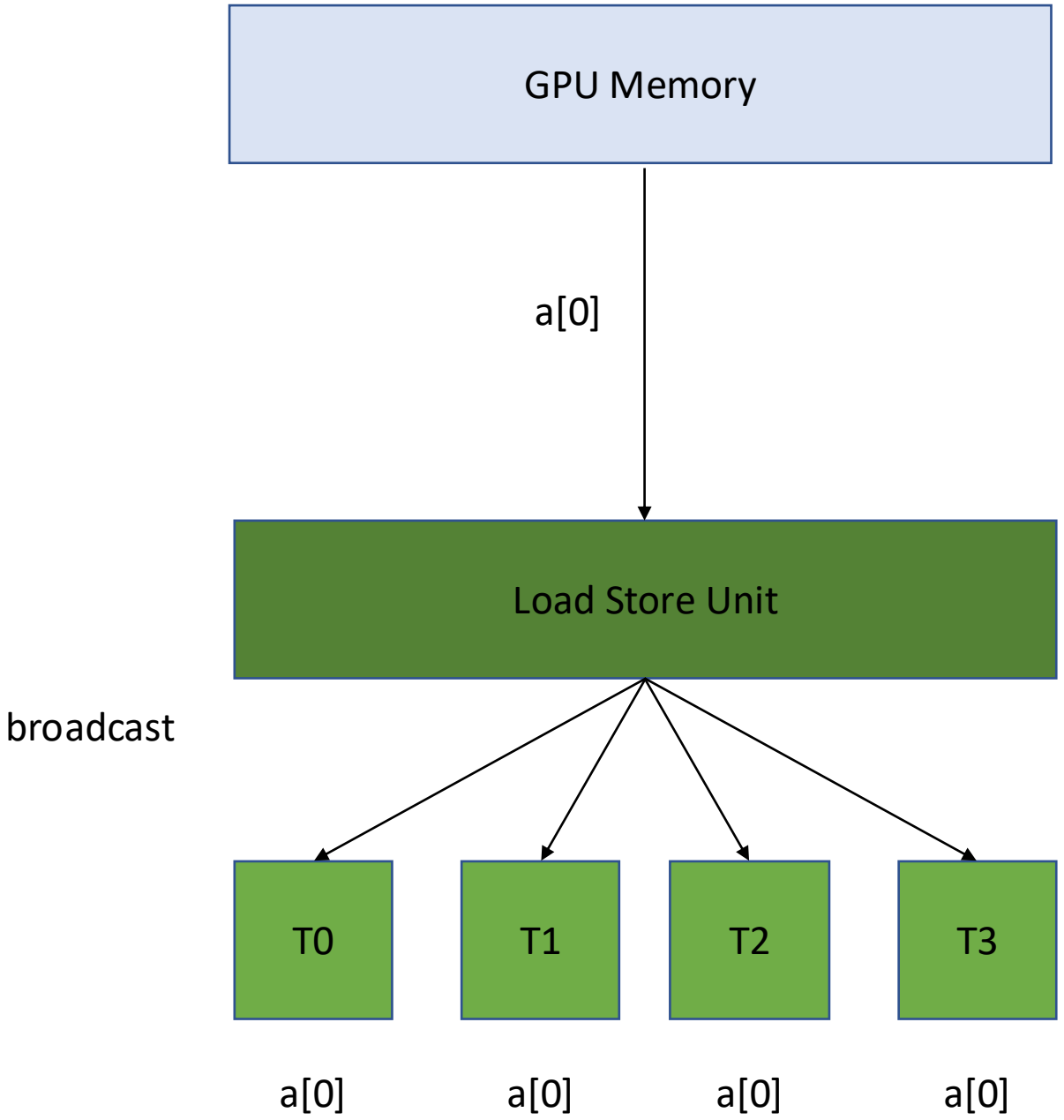
All read the same value



4 cores are accessing memory. What can happen

All read the same value

This is efficient: the load store unit can ask for the value and then broadcast it to all cores.

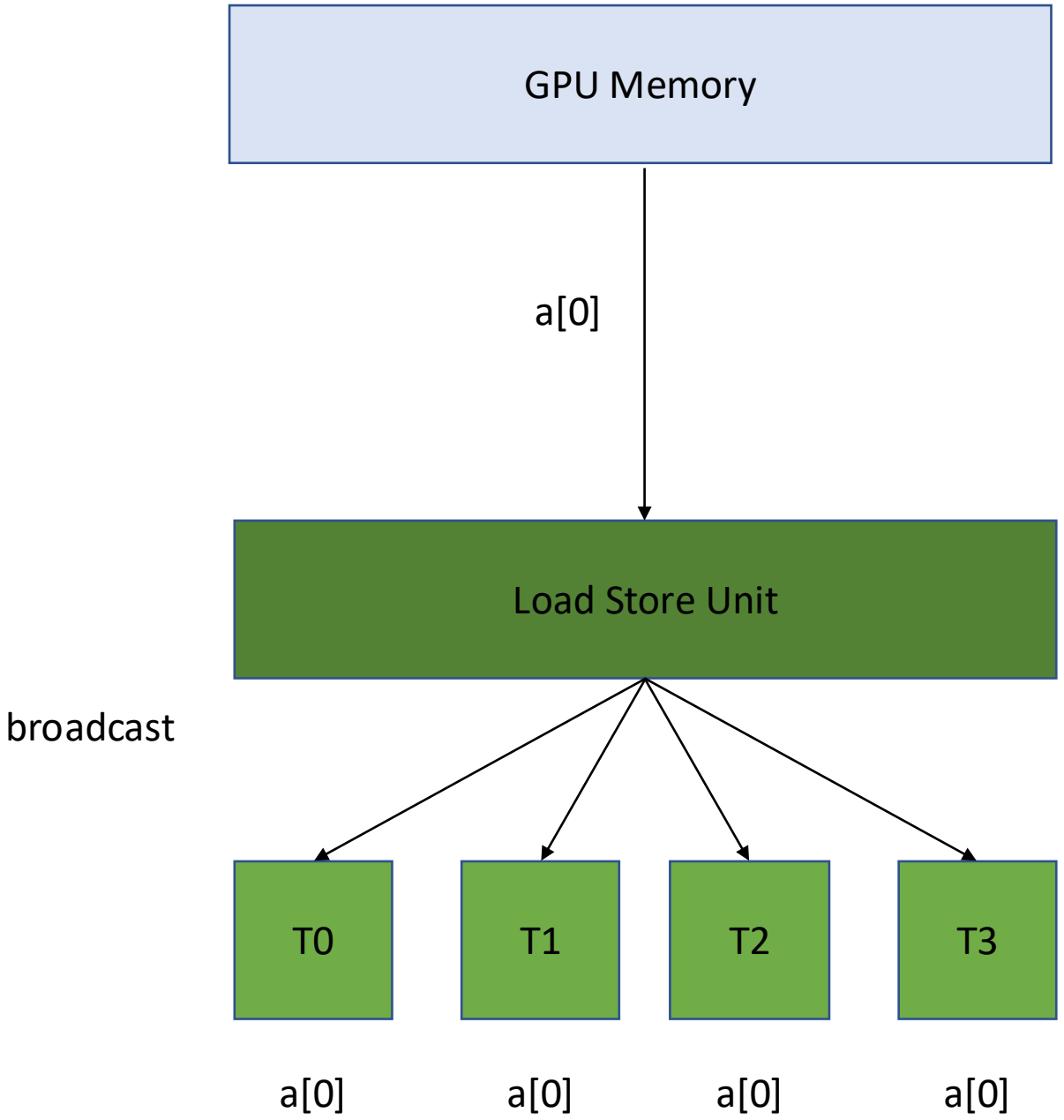


4 cores are accessing memory. What can happen

All read the same value
This is efficient: the load store unit can ask for the value and then broadcast it to all cores.

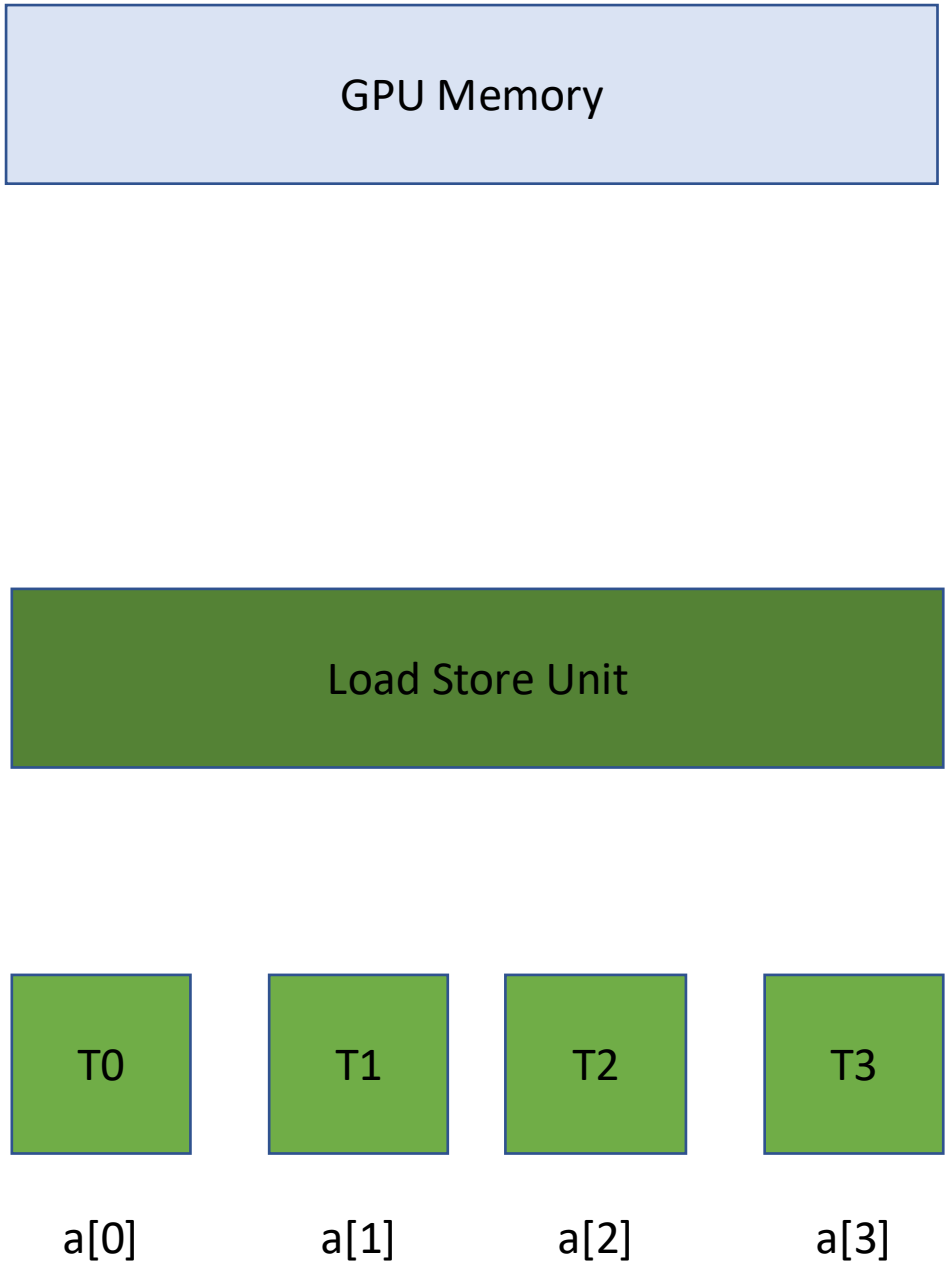
1 request to GPU memory

Efficient, but probably not too common.



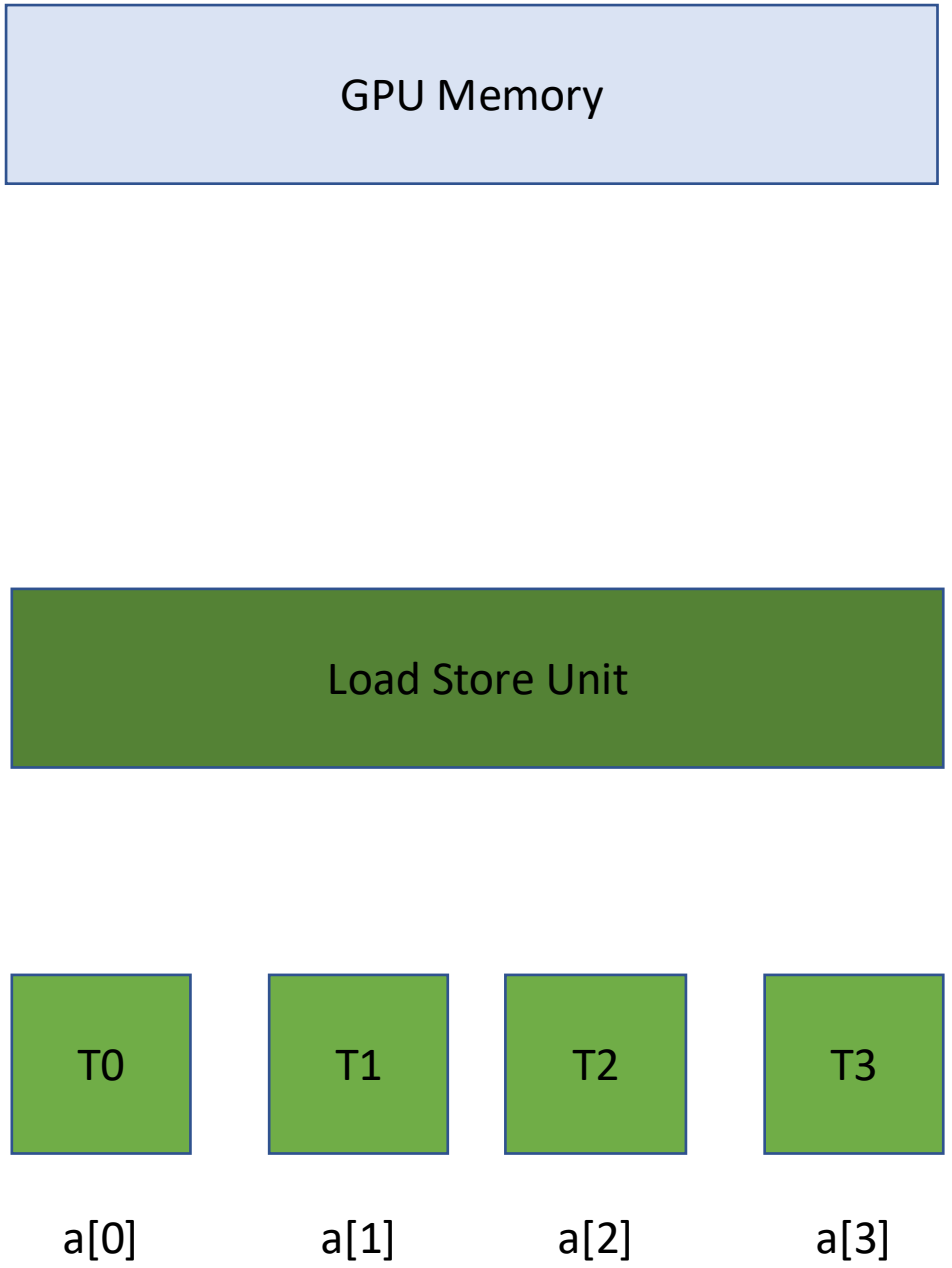
4 cores are accessing memory. What can happen

Read contiguous values



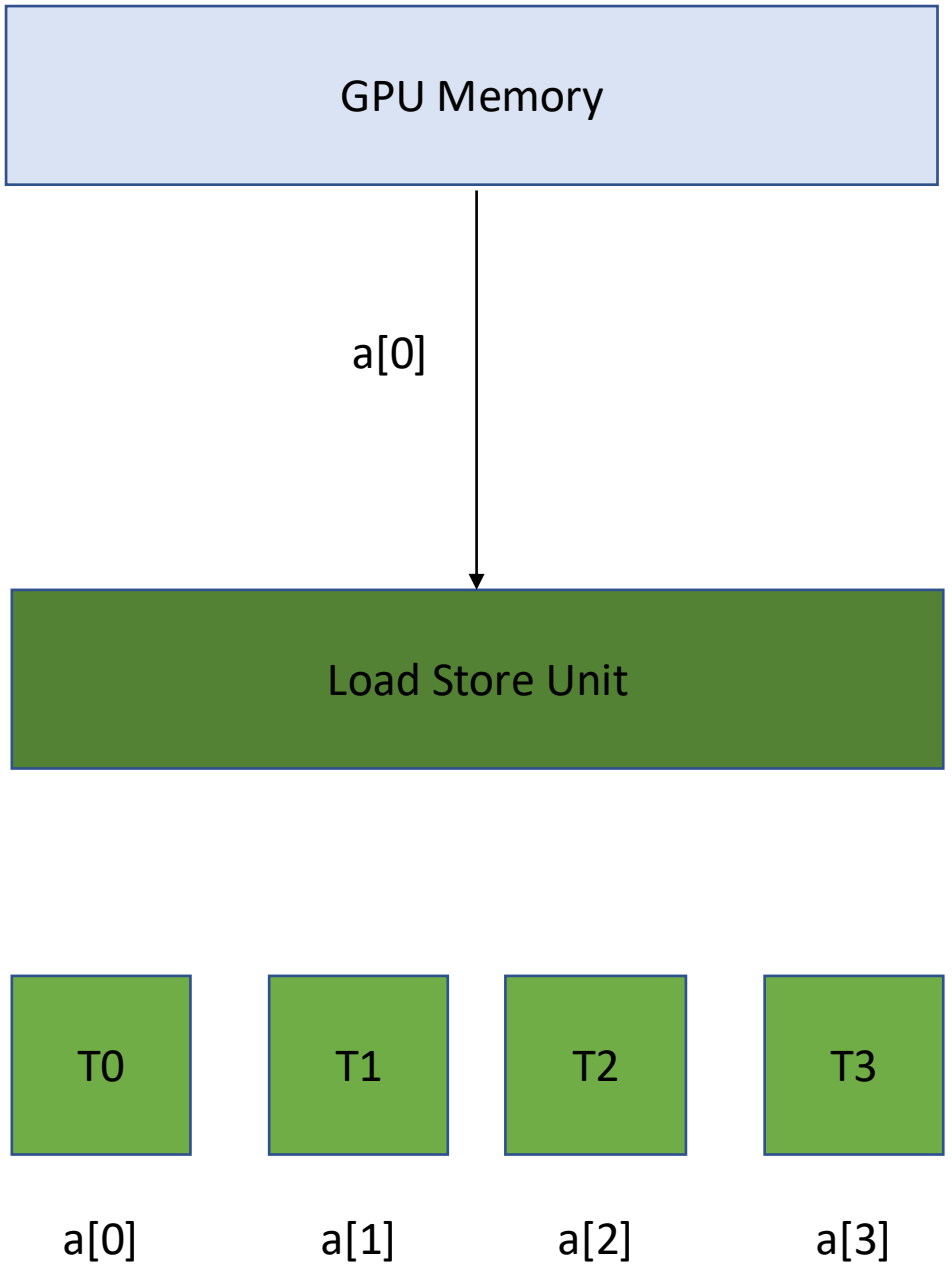
4 cores are accessing memory. What can happen

Read contiguous values
Like the CPU cache, the Load/Store Unit reads in memory in chunks. 16 bytes



4 cores are accessing memory. What can happen

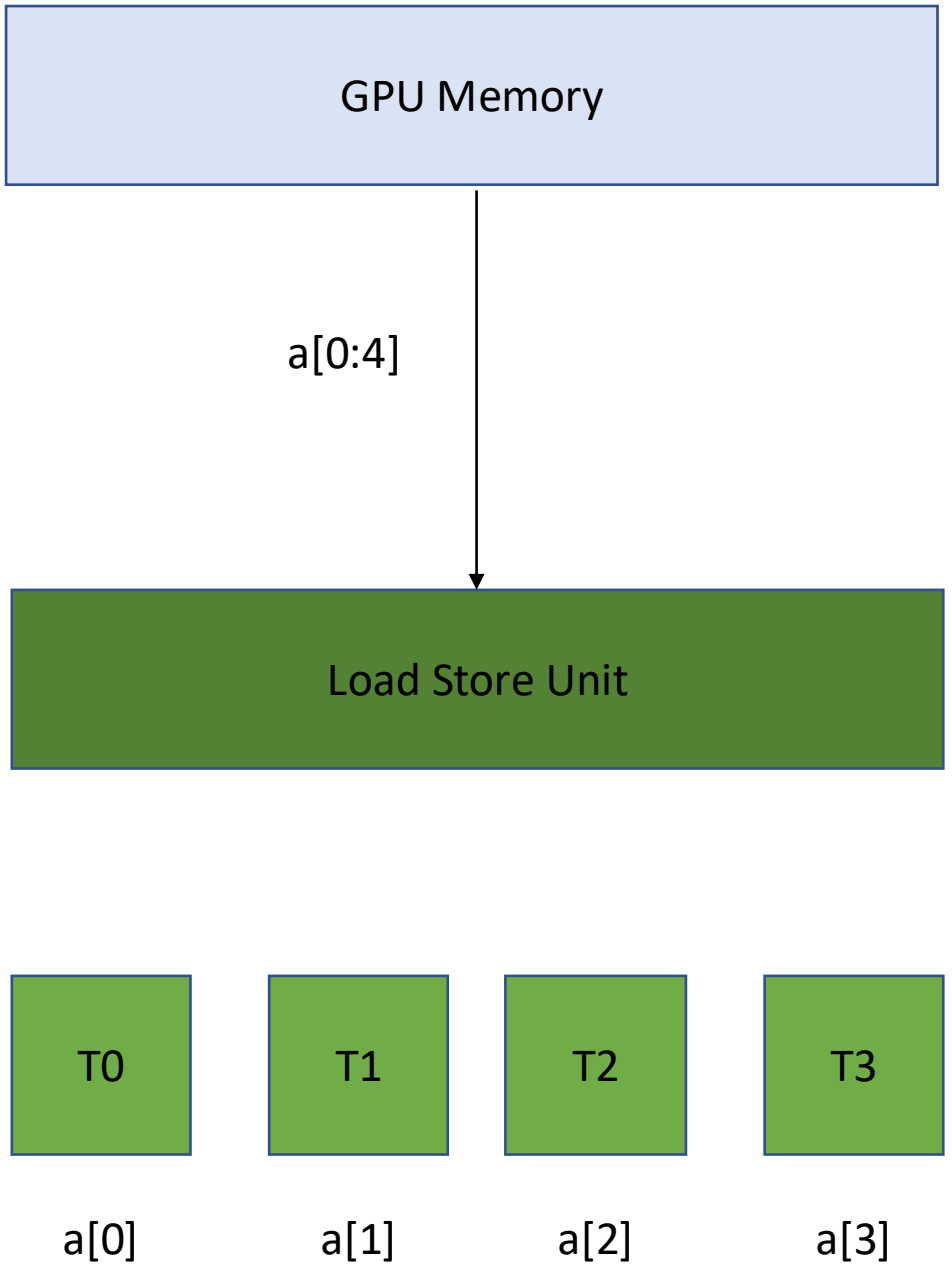
Read contiguous values
Like the CPU cache, the Load/Store Unit reads in memory in chunks. 16 bytes



4 cores are accessing memory. What can happen

Read contiguous values
Like the CPU cache, the Load/Store Unit reads in memory in chunks. 16 bytes

Can easily distribute the values to the threads

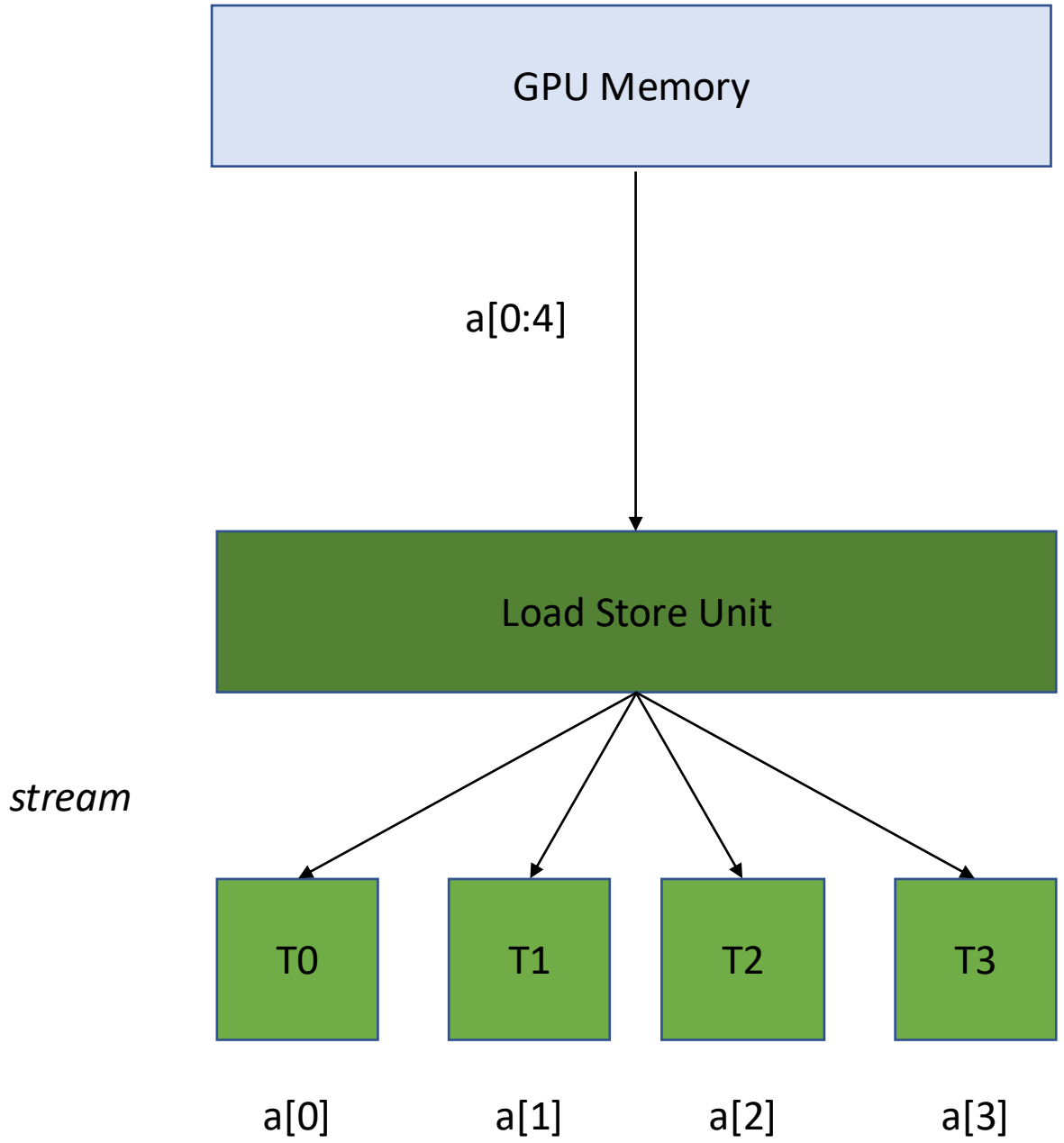


4 cores are accessing memory. What can happen

Read contiguous values
Like the CPU cache, the Load/Store Unit reads in memory in chunks. 16 bytes

Can easily distribute the values to the threads

1 request to GPU memory

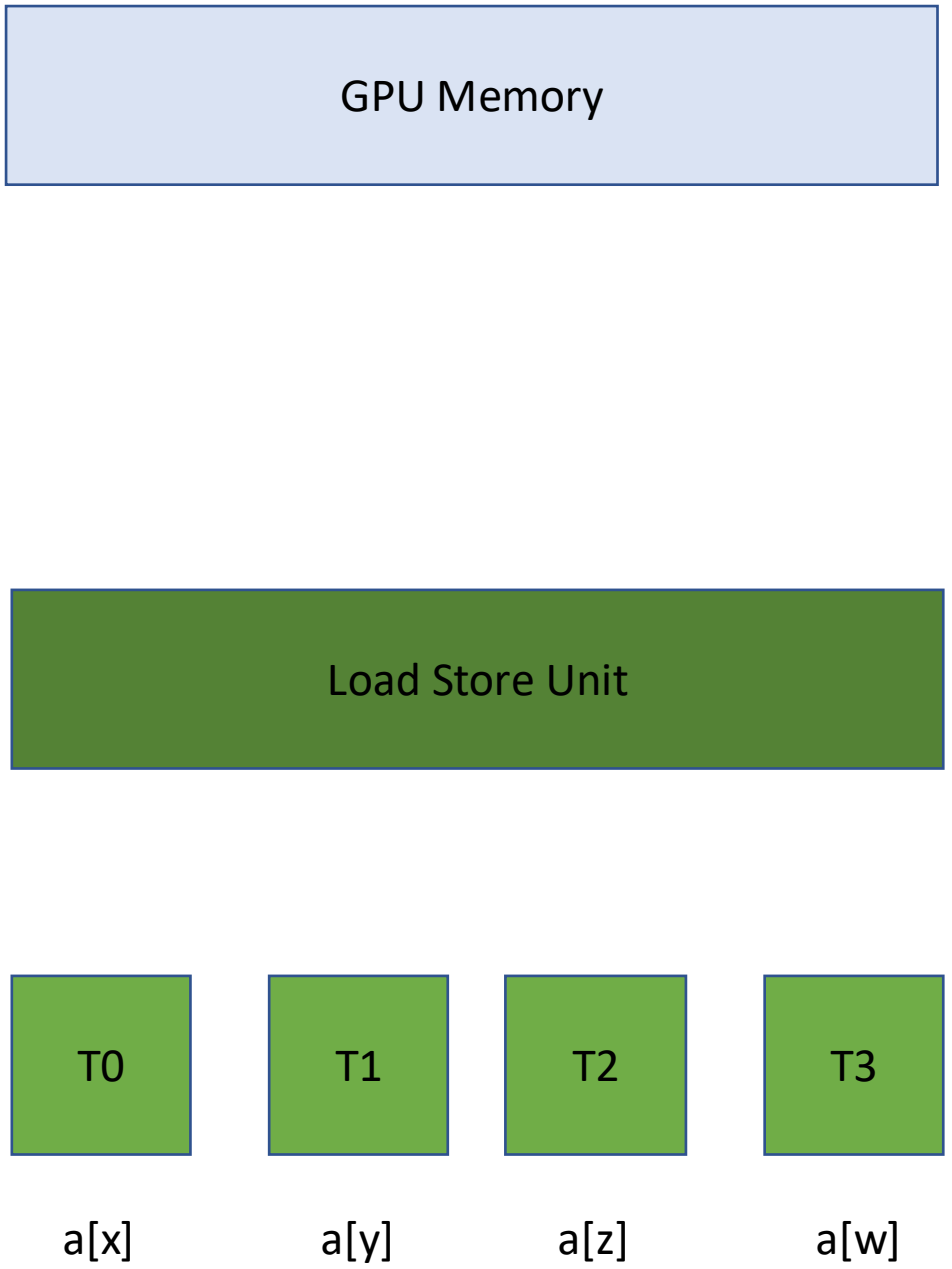


4 cores are accessing memory. What can happen

Read non-contiguous values

Not good!

Accesses are Serialized.
You need 4 requests to GPU memory

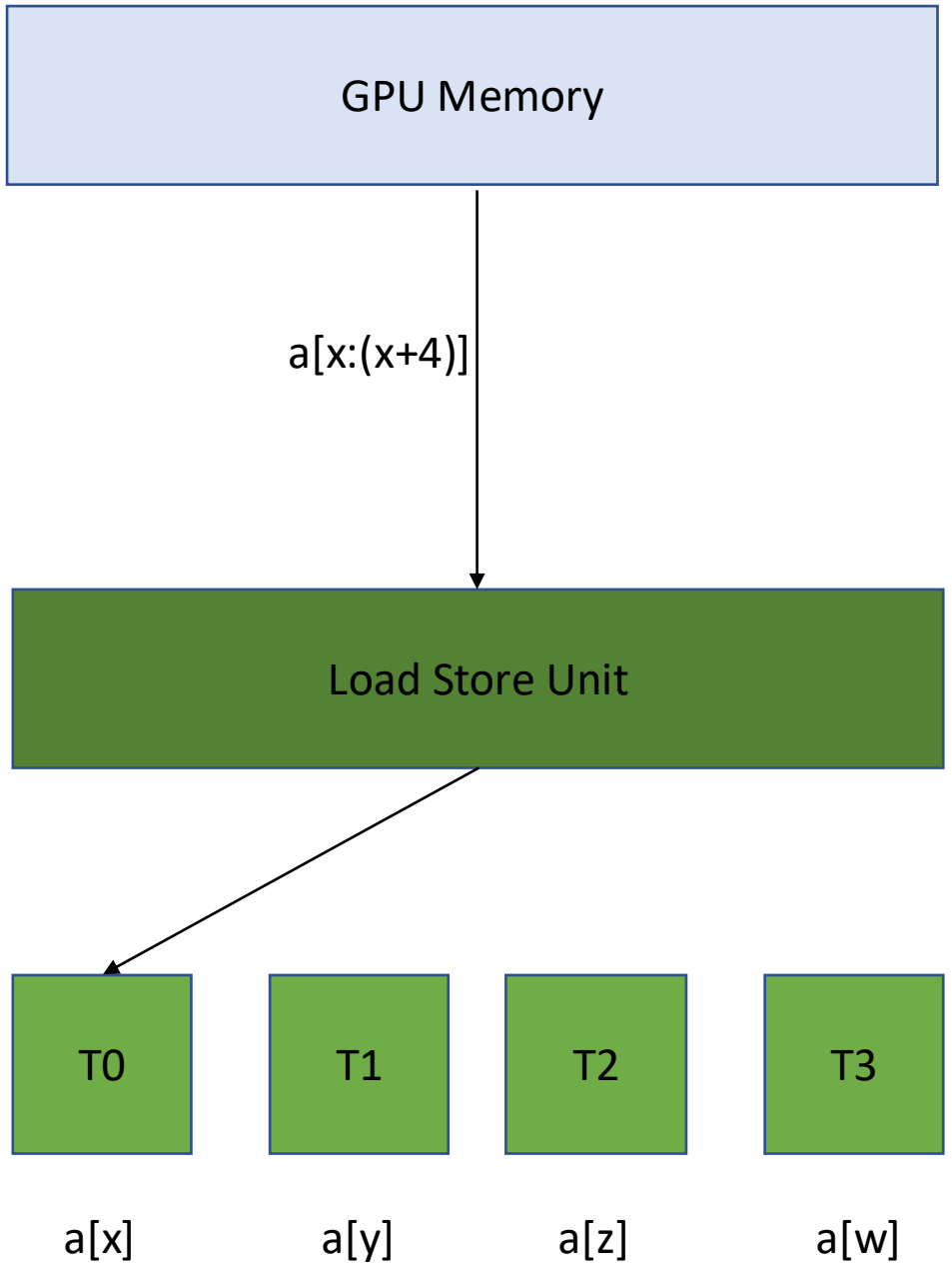


4 cores are accessing memory. What can happen

Read non-contiguous values

Not good!

Accesses are Serialized.
You need 4 requests to GPU memory

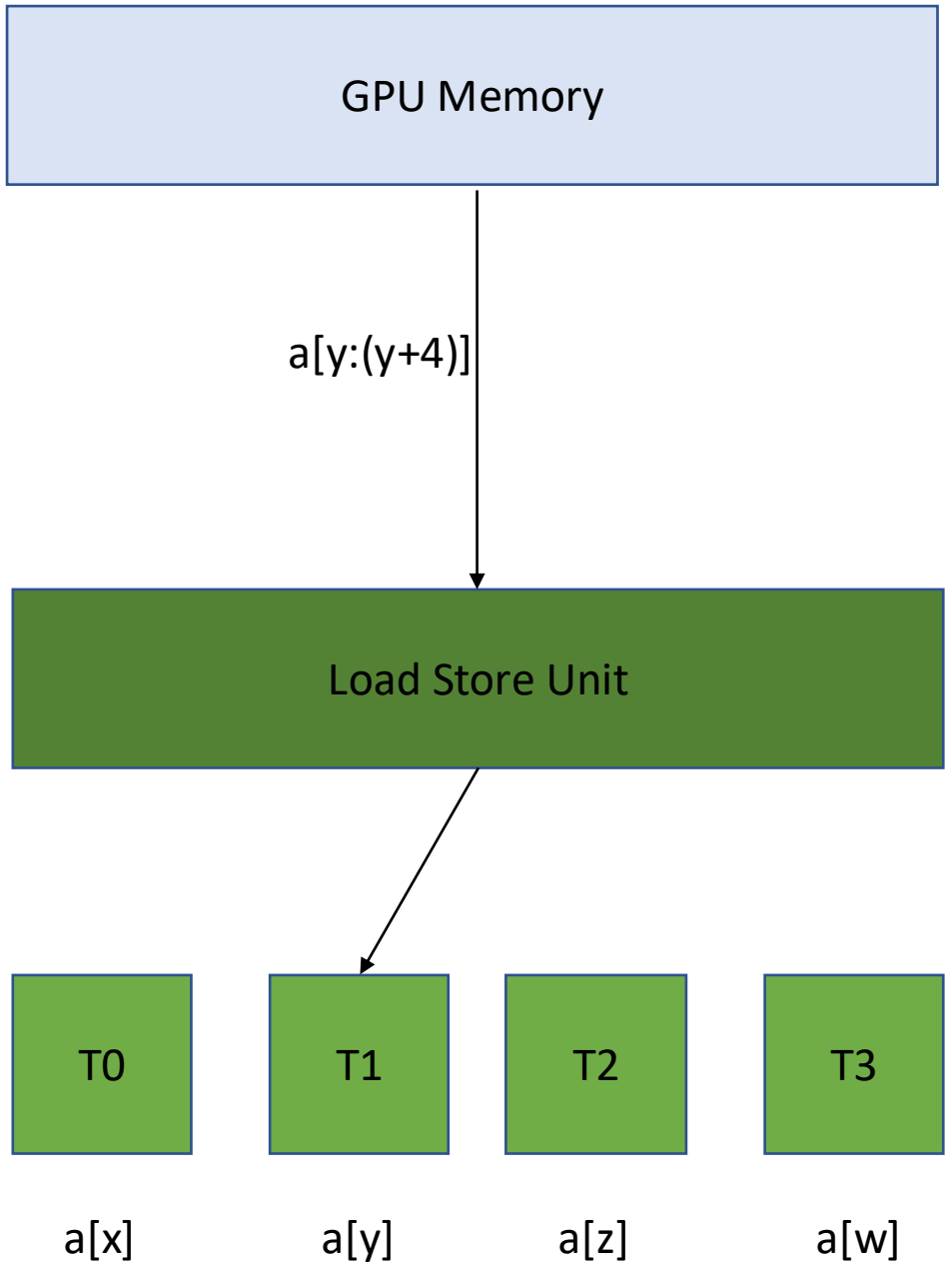


4 cores are accessing memory. What can happen

Read non-contiguous values

Not good!

Accesses are Serialized.
You need 4 requests to GPU memory

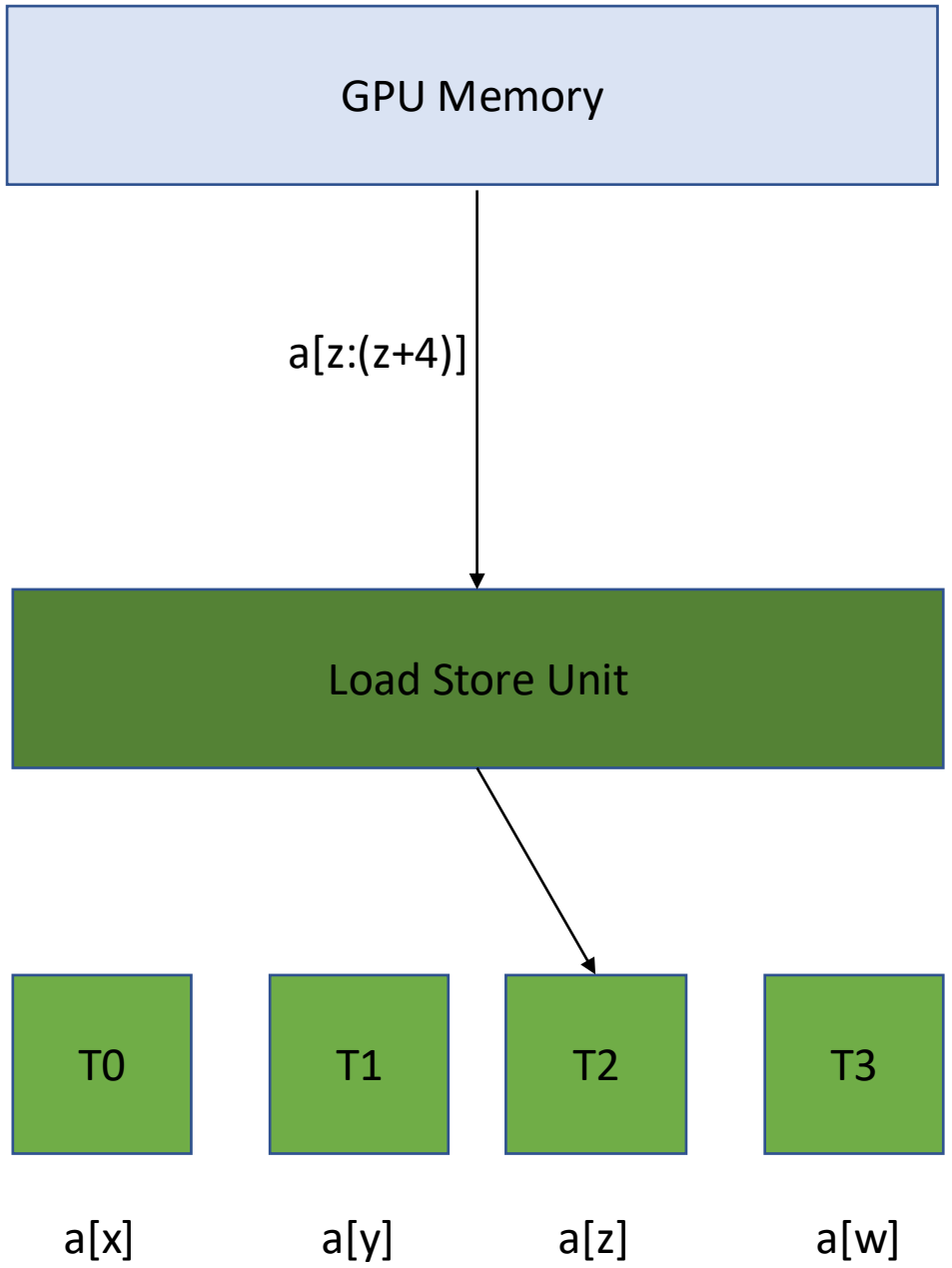


4 cores are accessing memory. What can happen

Read non-contiguous values

Not good!

Accesses are Serialized.
You need 4 requests to GPU memory

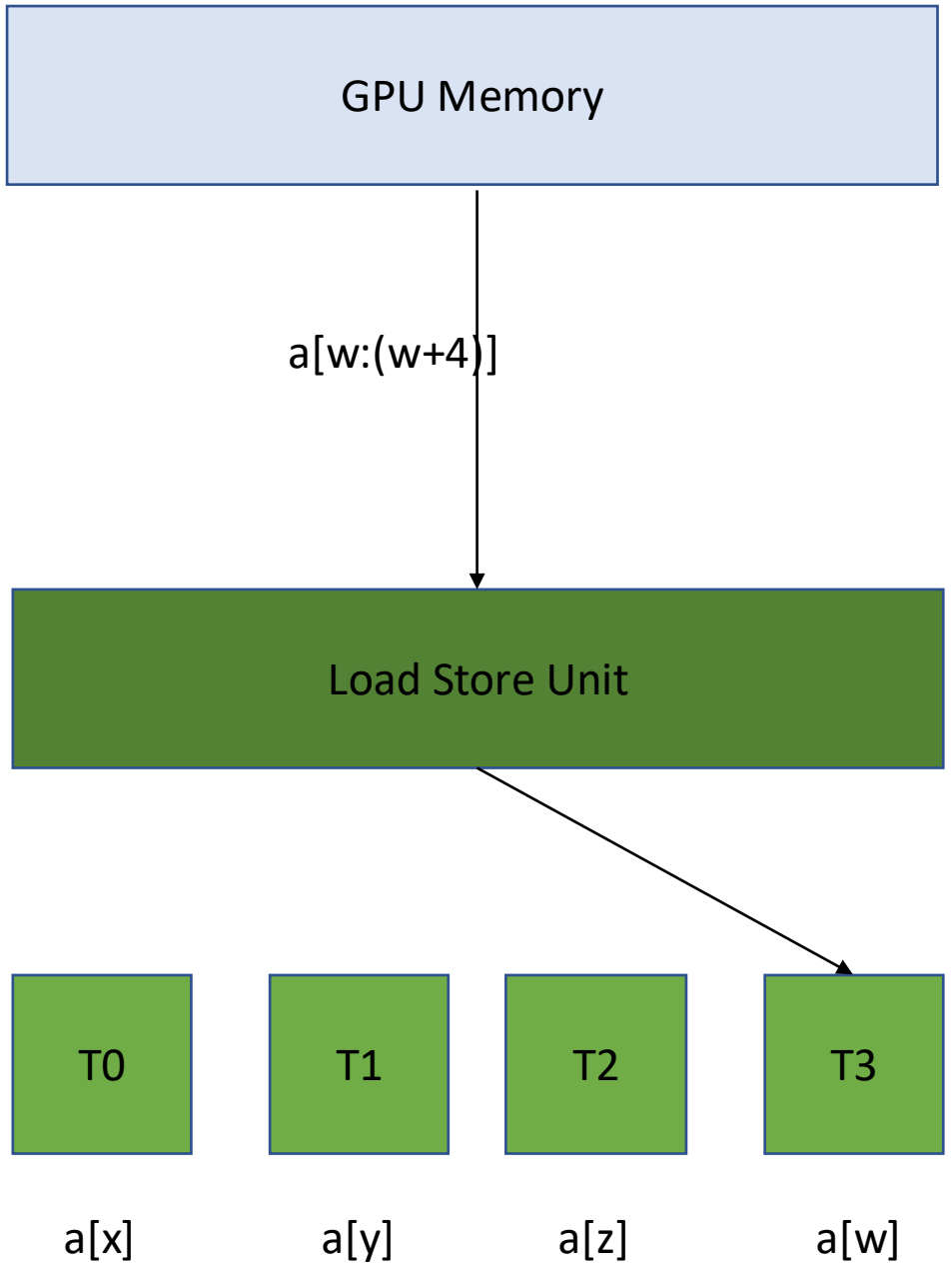


4 cores are accessing memory. What can happen

Read non-contiguous values

Not good!

Accesses are Serialized.
You need 4 requests to GPU memory



Quiz 12.

Like the CPU cache, the Load/Store Unit reads in memory in chunks. Is this correct?

Select one

A. True

B. False

Scan to answer

forms.gle/c8hpnGUtrui63zBcA

chart: docs.google.com/forms/d/e/1FAIpQLSc53ivr_o0JWDVCwQwmlR4_7xFgn_3t4vDOA2QgQjEa9rBsbQ/viewanalytics

Go back to our program

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    int chunk_size = size/blockDim.x;
    int start = chunk_size * threadIdx.x;
    int end = start + chunk_size;
    for (int i = start; i < end; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

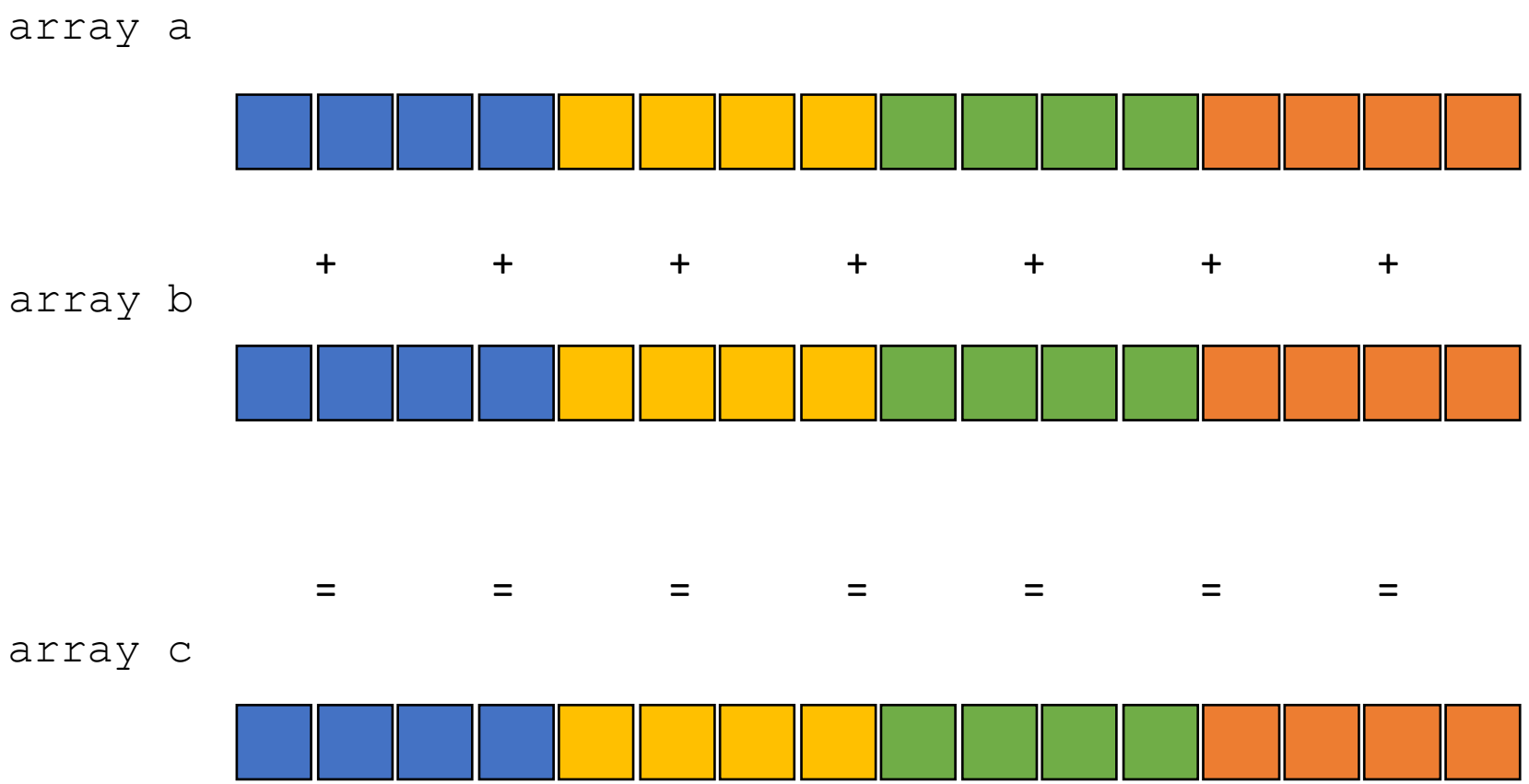
calling the function

```
vector_add<<<1,1024>>>(d_a, d_b, d_c, size);
```

Chunked Pattern

Computation can easily be divided into threads

Thread 0 - Blue
Thread 1 - Yellow
Thread 2 - Green
Thread 3 - Orange

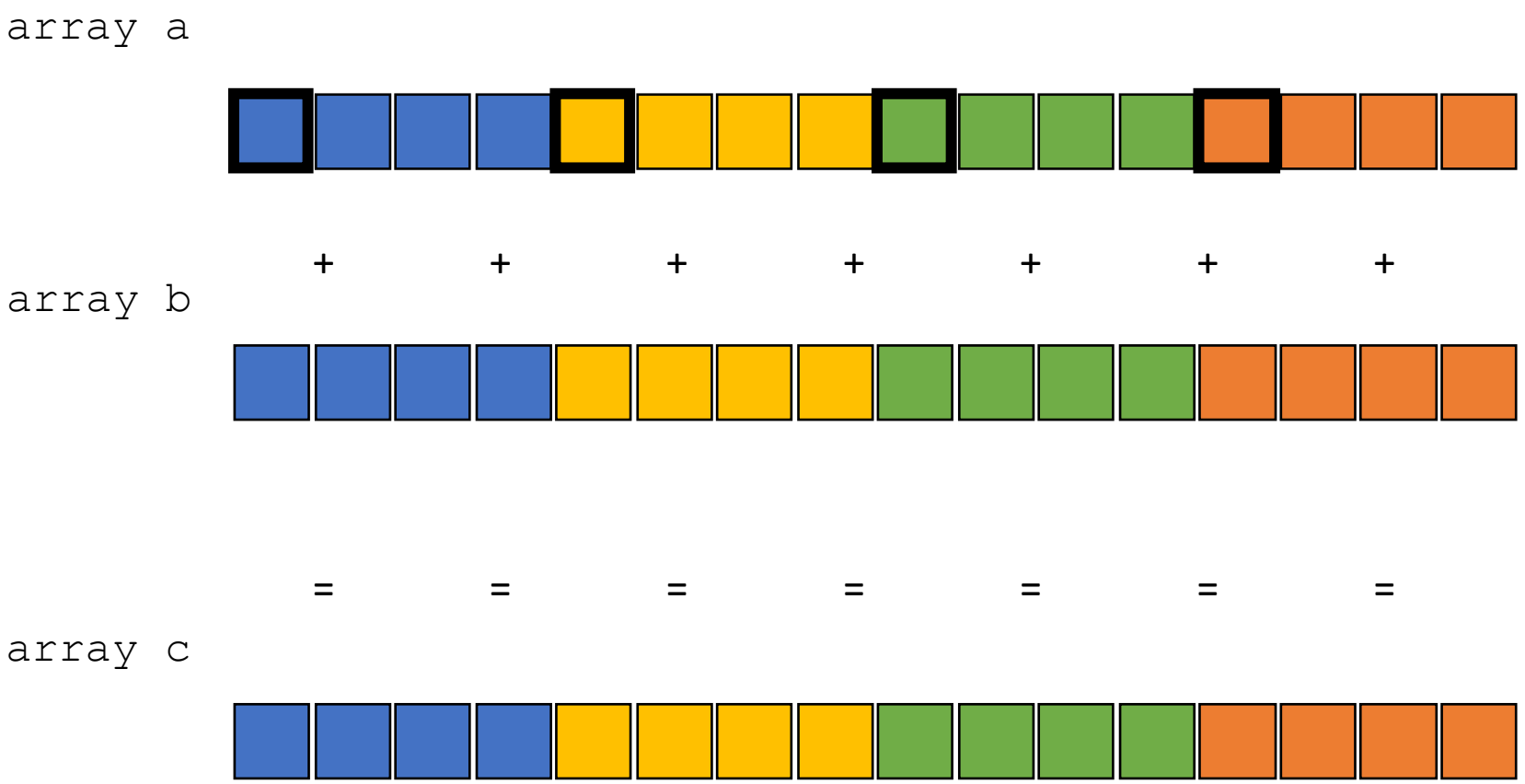


Chunked Pattern

the first element accessed by the 4 threads sharing a load store unit. What sort of access is this?

Computation can easily be divided into threads

Thread 0 - Blue
Thread 1 - Yellow
Thread 2 - Green
Thread 3 - Orange

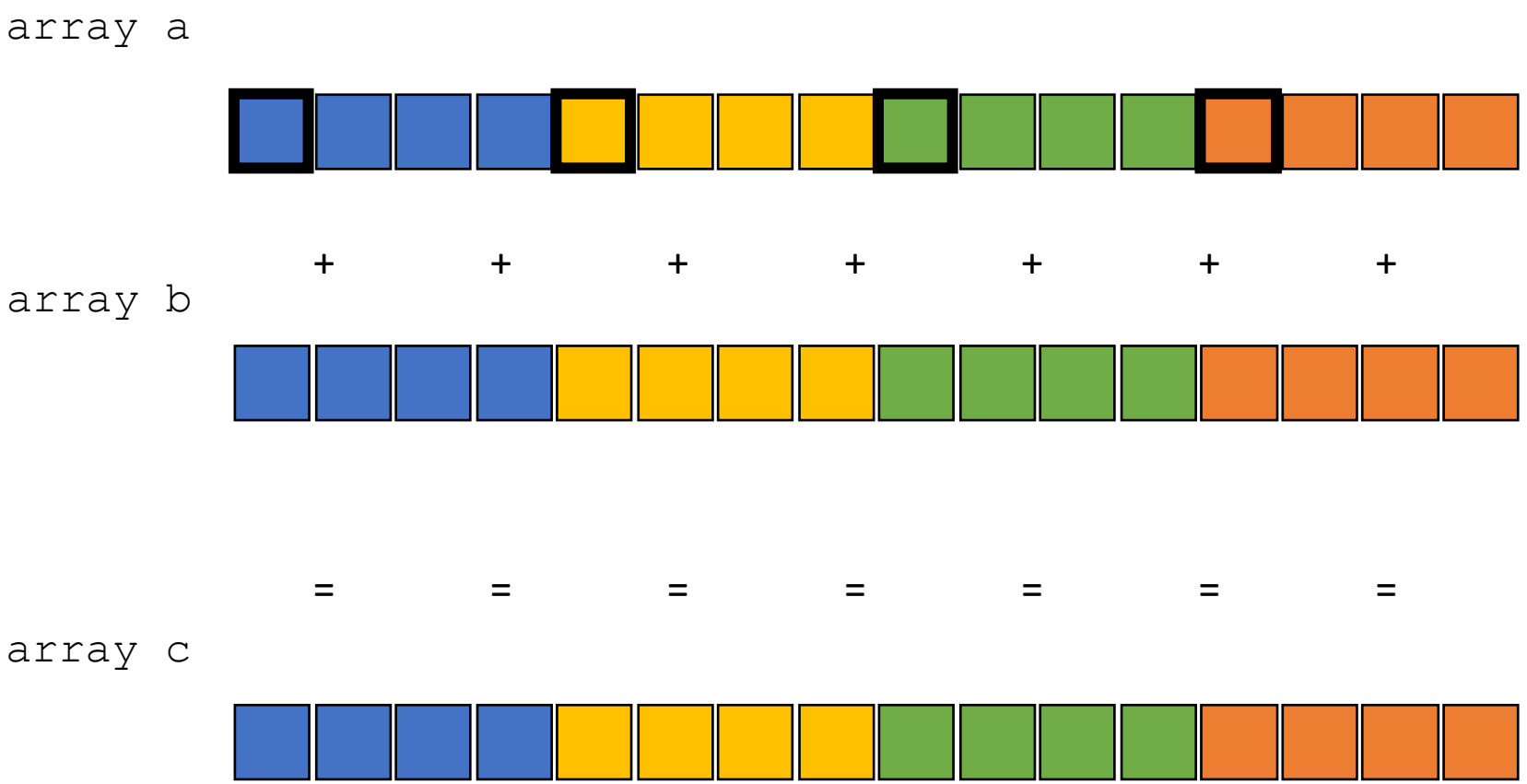


Chunked Pattern

the first element accessed by the 4 threads sharing a load store unit. What sort of access is this?

Computation can easily be divided into threads

Thread 0 - Blue
Thread 1 - Yellow
Thread 2 - Green
Thread 3 - Orange

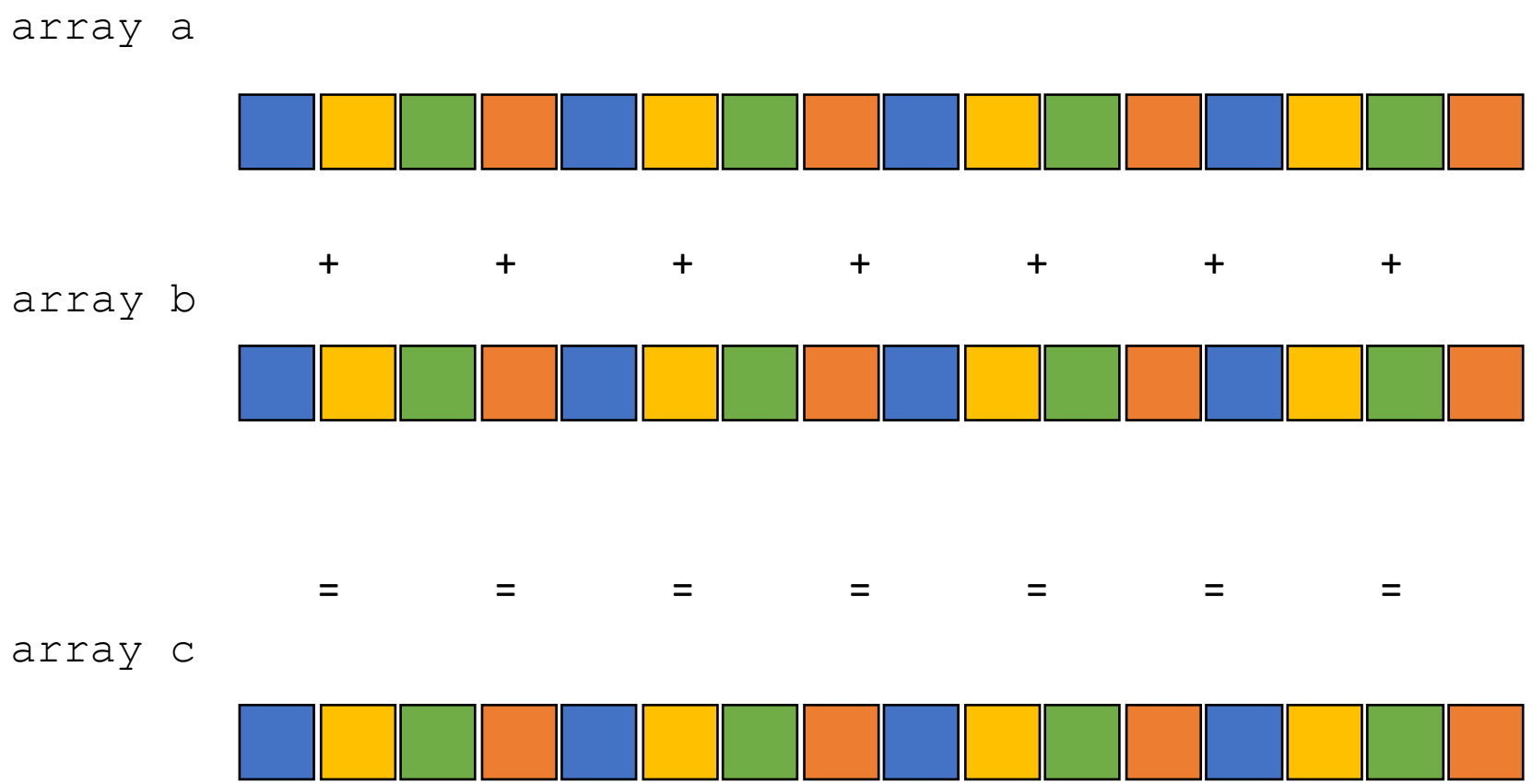


How can we fix this

Stride Pattern

Computation can easily be divided into threads

Thread 0 - Blue
Thread 1 - Yellow
Thread 2 - Green
Thread 3 - Orange

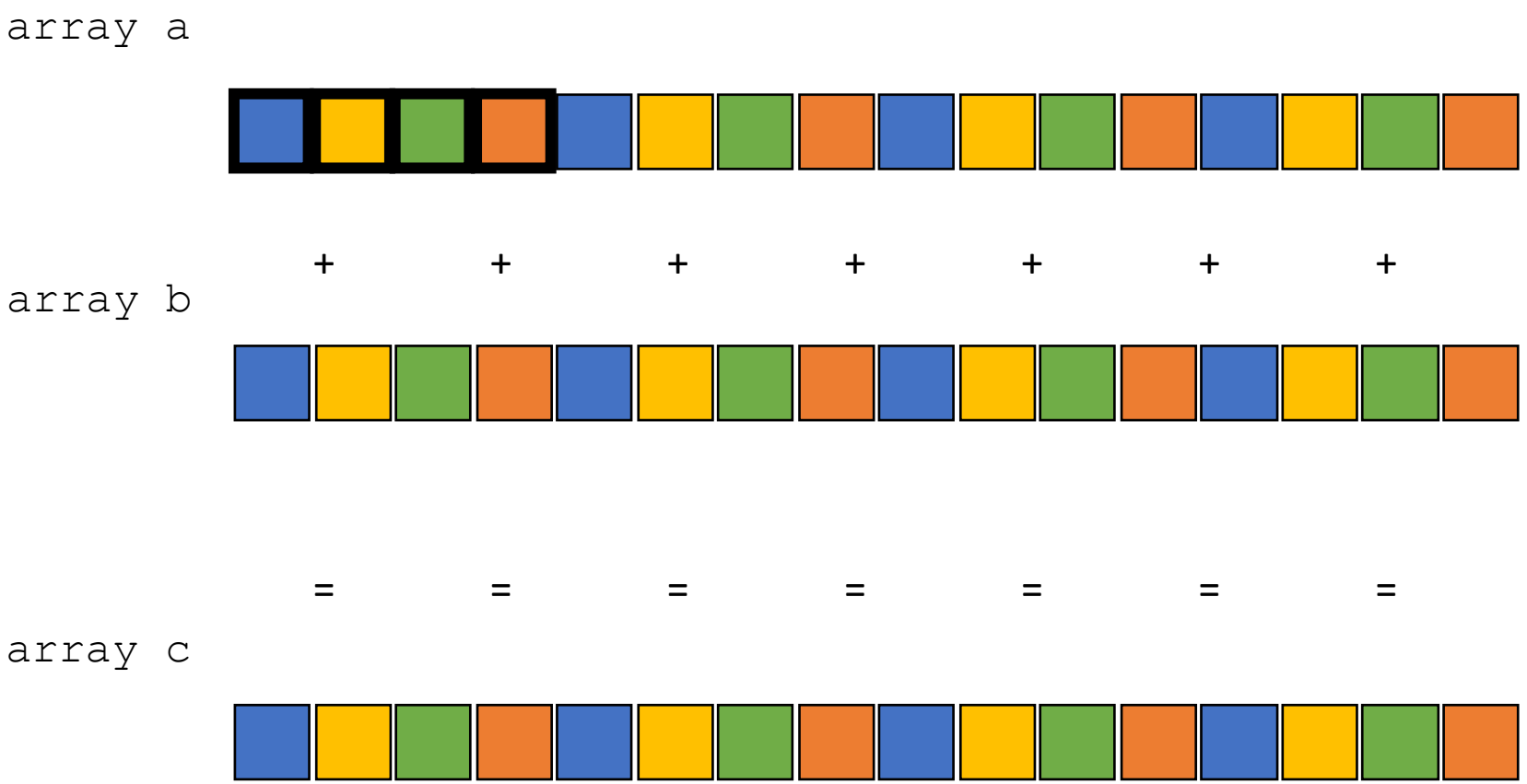


Stride Pattern

What sort of pattern is this?

Computation can easily be divided into threads

Thread 0 - Blue
Thread 1 - Yellow
Thread 2 - Green
Thread 3 - Orange



Go back to our program

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    int chunk_size = size/blockDim.x;
    int start = chunk_size * threadIdx.x;
    int end = start + chunk_size;
    for (int i = start; i < end; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

Chunked pattern
Let's change this to a stride pattern

calling the function

```
vector_add<<<1,1024>>>(d_a, d_b, d_c, size);
```

blockIdx.x is the block id.
blockDim.x is the number of threads per block.
threadIdx.x is the thread id within block.

Go back to our program

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
  for (int i = threadIdx.x; i < size; i+=blockDim.x) {
     d_a[i] = d_b[i] + d_c[i];
  }
}
```

calling the function

```
vector_add<<<1,1024>>>(d_a, d_b, d_c, size);
```

blockIdx.x is the block id.
blockDim.x is the number of threads per block.
threadIdx.x is the thread id within block.

Coalesced memory accesses

This is about 4x faster.

Quiz 12.

What can accelerate memory accesses on GPUs?

Free response: a few sentences on the form

Scan to answer

forms.gle/u2GrZU9t4BPo1iES9

chart: docs.google.com/forms/d/e/1FAIpQLSfqLApVqPTdL8tkgO51FhYpmg-18blEHVSUIyKfQ_NwODqnow/viewanalytics

Quiz 12.

What can accelerate memory accesses on GPUs?

Free response: a few sentences on the form



Scan to answer

Memory coalescing accelerates execution.

forms.gle/u2GrZU9t4BPo1iES9

chart: docs.google.com/forms/d/e/1FAIpQLSfqLApVqPTdL8tkgO51FhYpmg-18blEHVSUIyKfQ_NwODqnow/viewanalytics

Coalesced memory accesses

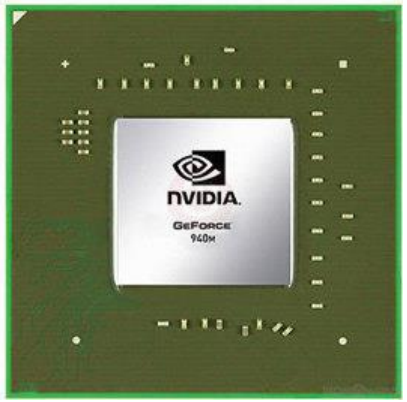
This is about 4x faster.



What else can we do?

Final Round

Tiny GPU in an embedded system



Fight!



The CPU in my professor workstation



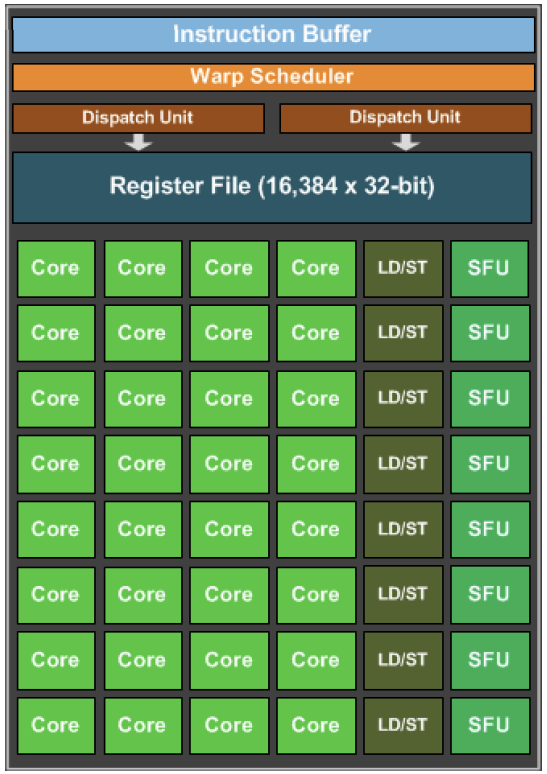
Nvidia Jetson Nano (whole chip, CPU + GPU)
2 Billion transistors
10 TDP
Est. $99

Intel i7-9700K
2.16 Billion transistors
95 TDP
Est. $316

https://www.techpowerup.com/gpu-specs/geforce-940m.c2648
https://www.alibaba.com/product-detail/Intel-Core-i7-9700K-8-Cores_62512430487.html
https://www.prolast.com/prolast-elevated-boxing-rings-22-x-22/

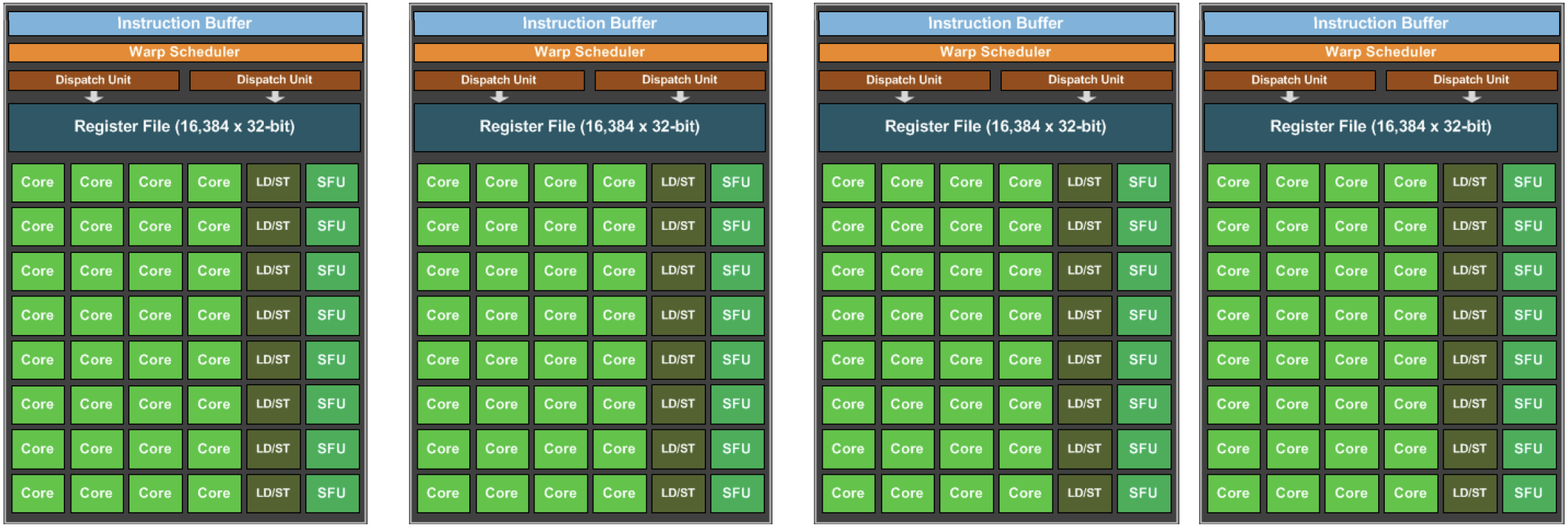
Multiple streaming multiprocessors

We've been talking only about 1 streaming multiprocessor, most GPUs have multiple SMs big ML GPUs have 32.



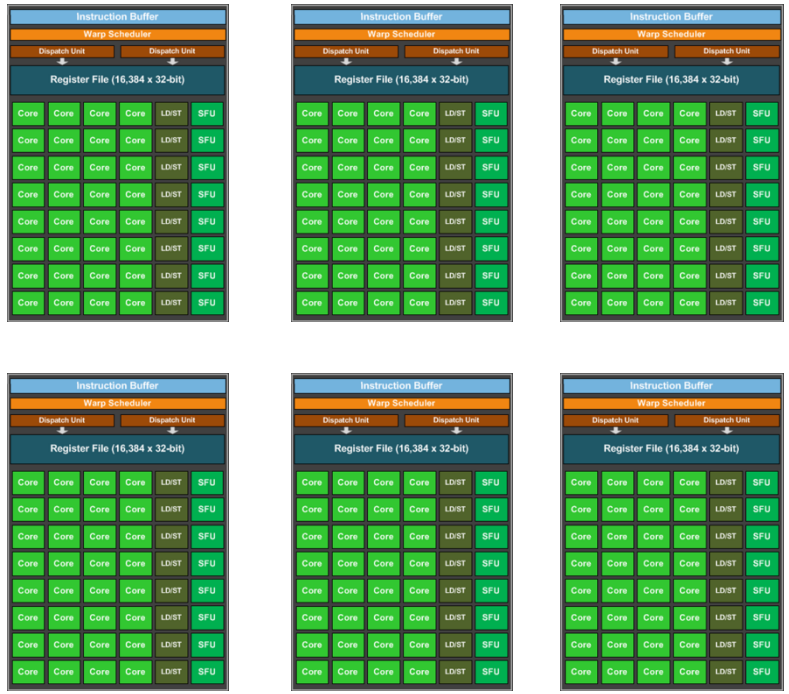
Multiple streaming multiprocessors

We've been talking only about 1 streaming multiprocessor, most GPUs have multiple SMs big ML GPUs have 32. This little GPU has 4



Multiple streaming multiprocessors

CUDA provides virtual streaming multiprocessors called **blocks**
They hide memory latency
Very efficient at launching and joining **blocks.**
No limit on blocks: launch as many as you need to map 1 thread to 1 data element



Go back to our program

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
  for (int i = threadIdx.x; i < size; i+=blockDim.x) {
     d_a[i] = d_b[i] + d_c[i];
  }
}
```

calling the function

```
vector_add<<<1,1024>>>(d_a, d_b, d_c, size);
```

Launch with many thread blocks

blockIdx.x is the block id.
blockDim.x is the number of threads per block.
threadIdx.x is the thread id within block.

Go back to our program

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  d_a[i] = d_b[i] + d_c[i];
}
```

calling the function

```
vector_add<<<1024,1024>>>(d_a, d_b, d_c, size);
```

```
#define SIZE (1024*1024)
```

Launch with many thread blocks

Need to recalculate some thread ids.

Now we have 1 thread for each element

blockIdx.x is the block id.
blockDim.x is the number of threads per block.
threadIdx.x is the thread id within block.

Quiz 12. Q4

Why do we need to calculate int i = blockIdx.x * blockDim.x + threadIdx.x?

Select one

A. To get the index in the matrix we want to compute in that thread

B. To get the whole matrix we want to compute in that thread

C. To get the index in the matrix we want to compute in that warp



Scan to answer

forms.gle/nCPhnjB89Kb1iF8t6

chart: docs.google.com/forms/d/e/1FAIpQLSdPYUn4_ybwmH3S_r8JaguP6oDhihRmp0GREkT-t_5T_nlGKA/viewanalytics

Final round

With 10x energy efficiency, GPU is as fast as CPU.
With the same energy efficiency, GPU is about 33x as CPU.

2D or 3D

```
dim3 block(16, 16);                          // 256 threads per block
                                             // z defaulting to 1
dim3 grid((W + 15) / 16, (H + 15) / 16);     // enough blocks to cover W x H
                                             // A grid is all the blocks created
                                             //   by one kernel launch.
blur<<<grid, block>>>(in, out, W, H);

__global__ void blur(float *in, float *out, int W, int H) {
  int x = blockIdx.x * blockDim.x + threadIdx.x;
  int y = blockIdx.y * blockDim.y + threadIdx.y;
  if (x < W && y < H)
    out[y * W + x] = ...;
}
```

thread, warp (32 threads, scheduled as a unit), block (up to 1024 threads, 32 warms), grid, SM.

Consequences of Warps, Stride Instructions

```
__global__ void vector_add(float * d_a, float * d_b, float * d_c, int size) {
  int global_id = (blockIdx.x * blockDim.x) + threadIdx.x;
  if (global_id % 4 == 0)
      d_a[global_id] = d_b[global_id] + d_c[global_id];
  else if (global_id % 4 == 1)
      d_a[global_id] = d_b[global_id] * d_c[global_id];
  else if (global_id % 4 == 2)
      d_a[global_id] = d_b[global_id] / d_c[global_id];
  else if (global_id % 4 == 3)
      d_a[global_id] = d_b[global_id] - d_c[global_id];
}
```

Arrray:
First quarter: +
Second quarter: *
Third quarter: /
Fourth quarter: -

Consequences of Warps, Chunked Instructions

```
__global__ void vector_add_chunked(float * d_a, float * d_b, float * d_c, int size) {
  int global_id = (blockIdx.x * blockDim.x) + threadIdx.x;
  if (global_id < (size/4)*1)
      d_a[global_id] = d_b[global_id] + d_c[global_id];
  else if (global_id < (size/4)*2)
      d_a[global_id] = d_b[global_id] * d_c[global_id];
  else if (global_id < (size/4)*3)
      d_a[global_id] = d_b[global_id] / d_c[global_id];
  else if (global_id < (size/4)*4)
      d_a[global_id] = d_b[global_id] - d_c[global_id];
}
```

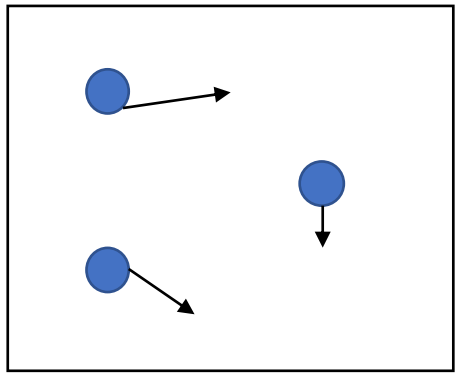
What is the right way to program GPUs?

- Still an open question!

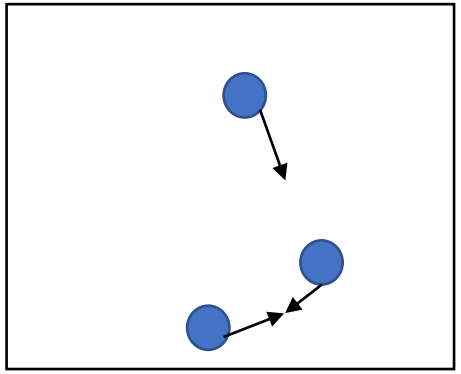
Homework 4 - first look

- Your assignment:
  - N-body simulation

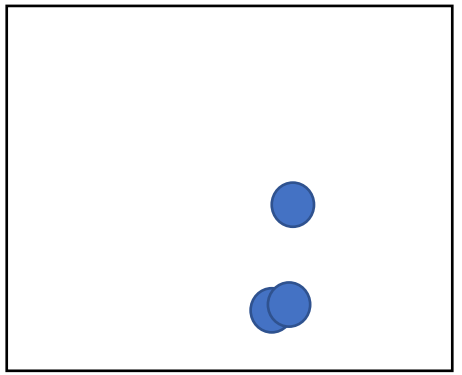
- Each particle interacts with every other particle



time = 0



time = 1

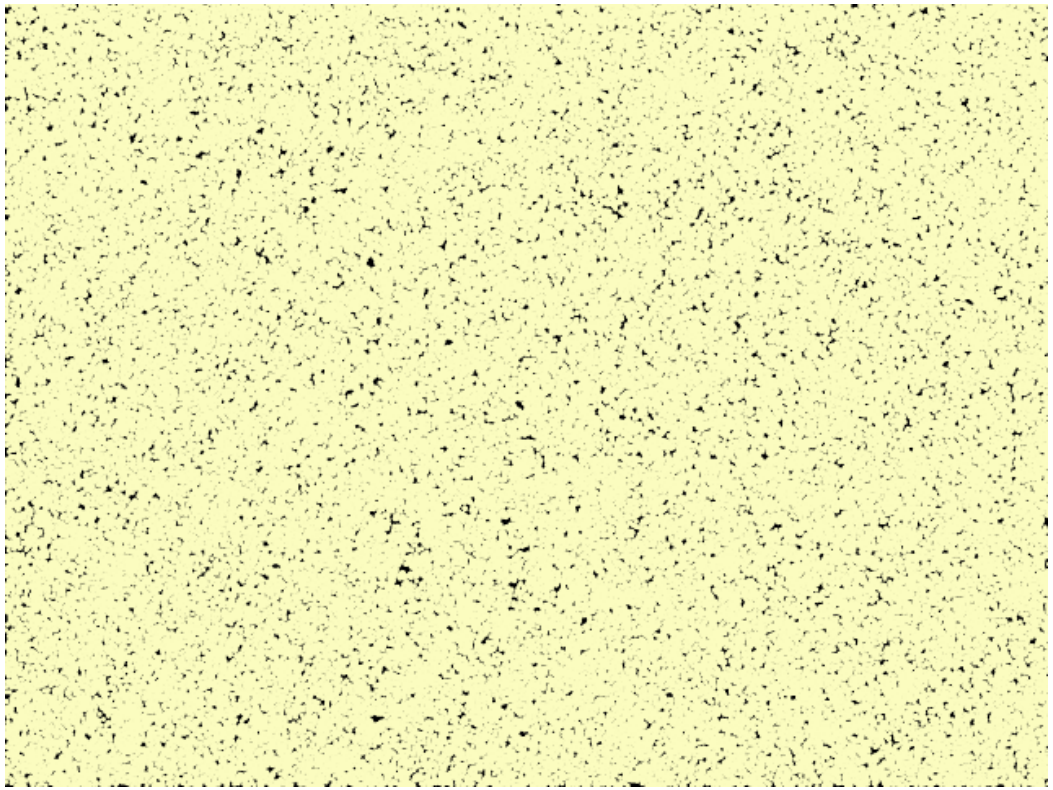


time = 2

Examples

- Gravity

- Boids (bird-oid object)
  An artificial life algorithm which simulates the flocking behavior of birds, and related group motion.
  https://en.wikipedia.org/wiki/Boids



Your homework

- N-body require a little bit of physics background so we will do something simpler.
  - If you want to explore with physics, please feel free

- **Local attraction clustering**:
  - For each particle: find your closest neighbor
  - You can take one step in the x direction and one step in the y direction towards your closest neighbor.

Your homework

- Part 1 of your homework will do this on a single javascript thread

Your homework

- Looks good, but with more particles, things start to go slower...

Your homework

- Looks good, but with more particles, things start to go slower...
- Part 2 of the homework is to implement with multiple CPU threads using javascript webworkers
  - Should get a linear speedup
- Part 3 is to implement with webGPU
  - Should get a BIG speedup!
- You need to explore how many particles you can simulate while keeping a 60 FPS (Frame Per Second) rate.